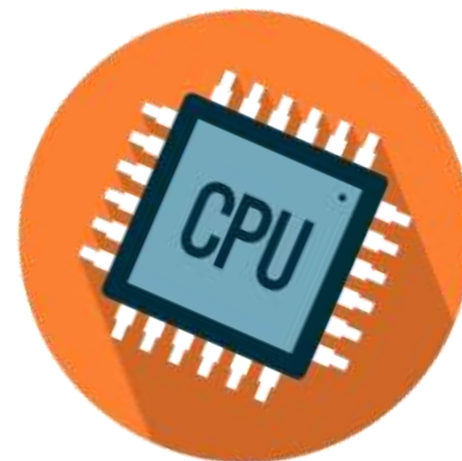




计算机组成原理

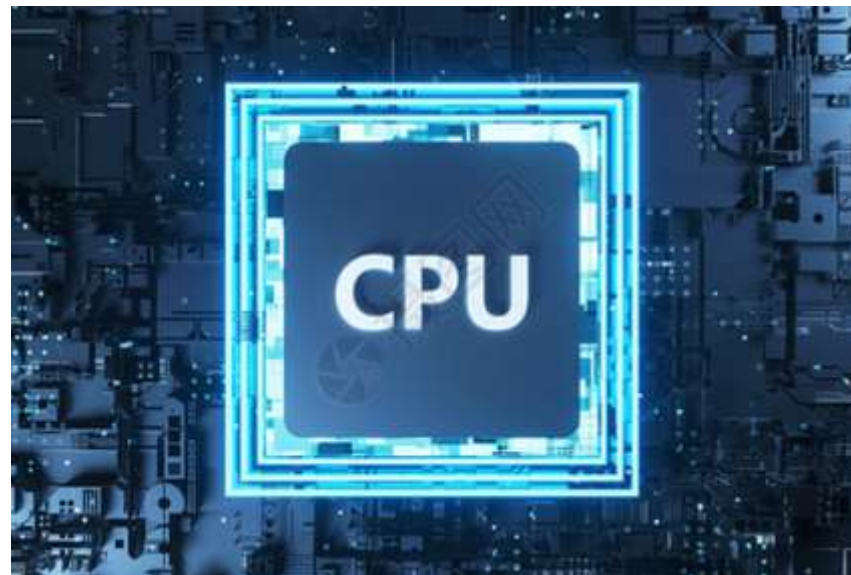


六、中央处理器



CPU的发展历程

- 1971年Intel的第一块CPU-4004，集成了2300个晶体管，功能有限，速度很慢，是8位处理器。
- 1978年Intel的16位CPU i8086，主频4.77MHz，集成30万个晶体管。
- 1979年，IBM的PC机首次使用CPU i8086，开启了PC机的第一代CPU使用记录。



CPU的发展历程

- 1982年Intel推出了80286CPU，集成了13.4万个晶体管，时钟频率达到20MHz。
- 1985年Intel推出了80386CPU，32位处理器，包含27.5万个晶体管，时钟频率33MHz。
- 1989年，推出了80486CPU，120万个晶体管，频率66MHz，性能比80386提高4倍。

CPU的发展历程

- 1983年Intel推出了Pentium CPU，主频200MHz，集成310万个晶体管。
- 1997年，推出Pentium MMX，主频233MHz。MMX表示“多媒体扩展”，为了增强CPU在音像、图形、通信应用方面的技术。
- 1999年Intel推出了Pentium III，主频1GHz，具有增强3D运算、动画、影像等功能

CPU的发展历程

- 2000年Intel推出了Pentium IV，主频1.5GHz，2001年达到2GHz，2002年达到2.4GHz
- 2004年分别推出了Pentium 4 560、550、540、530和520，主频分别是3.6GHz，3.4GHz，3.2GHz，3.0GHz，2.8GHz
- 2005年Pentium 4 571主频为3.8GHz
- 随着技术的发展，CPU的主频越来越快，计算机的运算速度也越来越快。

年份	CPU 型号	频率
1978	8086/8088	5-10 MHz
1982	80286	6-12 MHz
1985	Intel386™	16-33 MHz
1989	Intel486™DX	25-50 MHz
1993	Pentium®	60-233 MHz
1997	Pentium® II	233-450 MHz
1999	Pentium® III	450M-1G Hz
2000	Pentium® 4	1.4-3.2 GHz

国产CPU发展历程

正式起步

1965年，中国自主研发与国际水平相近的晶体管109机；1968年研发了小规模集成电路106机

差距扩大

1975年，受限于国内经济条件，国际技术封锁等原因，我国丧失了第四代计算机系统的研发机会

坎坷发展

1990年，国家对国产CPU的支持力度明显下降，主要科研支持计划都未将其列入，直接导致了CPU设计能力的基本丧失

再度起航

2006年，十五期间，国家启动发展国产CPU的泰山计划，863计划也提出自主研发CPU。2006年核高基专项启动，飞速发展

沸腾之路

目前国产CPU产业已初具规模，涌现出一批领军企业；移动CPU设计能力快速成长，已处于全球领先地位；PK体系2.0版本，生态日趋完善

www.huaon.com

资料来源：公开资料，华经产业研究院整理

CPU（中央处理器）行业相关政策

时间	政策	发布主体	内容
2023. 10	《算力基础设施高质量发展行动计划》	工信部等六部门	聚焦CPU、GPU、操作系统、存储等关键产品，推动关键技术试点验证，形成标杆应用产品与方案，构建软硬件相互适配、协调发展的生态体系。
2022. 05	《国务院关于印发扎实稳住经济一揽子政策措施的通知》	国务院	鼓励平台企业加快人工智能、云计算、区块链、操作系统、处理器等领域技术研发突破。
2022. 03	《关于做好2022年享受税收优惠政策的集成电路企业或项目、软件企业清单制定工作有关要求的通知》	发改委	重点集成电路设计领域包括：（一）高性能处理器和FPGA芯片；（二）存储芯片；（三）智能传感器；（四）工业、通信、汽车和安全芯片；（五）EDA、IP和设计服务。
2021. 12	《“十四五”国家信息化规划》	中央网络安全和信息化委员会	加快集成电路关键技术攻关，推动计算芯片、存储芯片等创新，加快集成电路设计工具、重点装备和高纯靶材等关键材料研发，推动绝缘栅双极型晶体管（IGBT）、微机电系统（MEMS）等特色工艺突破。

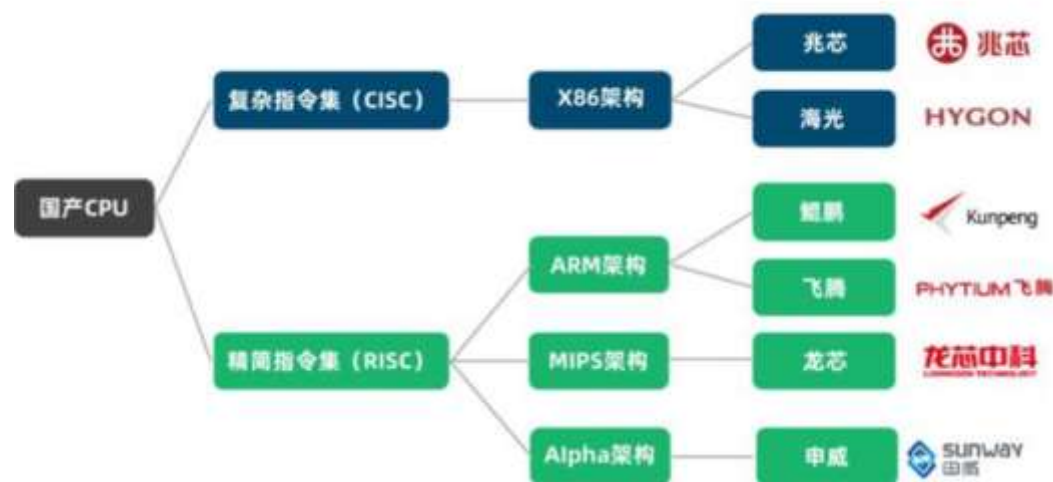
资料来源：政府官网，华经产业研究院整理

对比指标	海光	龙芯	鲲鹏	飞腾	兆芯	申威
合作方/资方	AMD/中科院曙光	中科院研究所	华为	天津飞腾/CEC	VIA/上海国资委	江南计算所/CETC
指令集体系	X86 (AMD)	LongISA2.0+MIPS	ARMv8	ARMv8	X86 (VIA)	ALPHA, SW-64
架构来源	IP 授权	指令集授权+自研	指令集授权	指令集授权	IP 授权	指令集授权+自研
代表产品	海光 1 号 海光 2 号 海光 3 号 海光 4 号	龙芯 1 号 龙芯 2 号 龙芯 3 号	鲲鹏 920	腾云 S 系列 腾锐 D 系列 腾锐 E 系列	ZX-C ZX-D KX-5000 KX-6000 KH-20000	SW1600 SW1610 SW26010
产品覆盖领域	服务器	桌面、服务器	服务器、桌面、嵌入式	服务器、桌面、嵌入式	服务器、桌面、嵌入式	服务器、桌面
应用市场	党政+商用	党政市场	党政+商用	党政+商用	党政+商用	军方+党政
优势	X86 最新授权，性能较强，应用生态丰富	起步最早，适配厂商多，自主化程度高	ARM 前景广阔：产品线极其丰富，性能最强；党政+商用市场接受程度高	ARM 前景广阔：产品线丰富，性能不断提升；架构层级授权自主化程度较高	上海地区覆盖广，x86 应用生态丰富	在军方市场占有率高，底层应用、超算为主力方向
劣势	目前自主化程度相对低；股权结构复杂；无桌面授权	MIPS 生态应用匮乏，性能一般，不利于商用市场拓展	受制裁中，未来存在不确定性	产品起步晚，性能相对弱	早期的 x86 内核层级授权，市场开拓不足	超算为主要方向，商用产品开发不足

厂商	服务器：中科曙光	台式机：曙光、联想、方正、同方等； 服务器：云海麒麟、五舟科技、清华同方、长城等； 笔记本：方正、同方、山东超越、北京计算机研究所等	服务器：华为	台式机：长城； 笔记本：长城； 服务器：清华同方、浪潮、联想、长城等	台式机：联想、同方； 笔记本：联想； 服务器：云海麒麟、火星舰、联想、秉时	服务器：ZoomServer、云海麒麟、联想、方正、宝德等
实际应用	国家级超算项目	玲珑、远舰、福舰、北斗导航卫星	华为服务器	天河一号、天河二号、天河三号	笔记本、服务器、火星舰存储系统	神威蓝光、神威-太湖之光
代工厂	格罗方德、三星	意法半导体	台积电	台积电	台积电	中芯国际
最小制程	14nm	28nm	7nm	16nm	16nm	28nm



6大国产CPU



六大国产CPU公司一览

	龙芯	鲲鹏	飞腾	海光	兆芯	申威
合作方/资方	中科院研究所	华为	天津飞腾/CEC	AMD/中科曙光	VIA/上海国资委	江南计算所/CETC
指令集体系	LongISA 2.0+MIPS	ARMv8	ARMv8	X86 (AMD)	X86 (VIA)	ALPHA, SW-64
架构来源	指令集授权+自研	指令集授权	指令集授权	IP授权	IP授权	指令集授权+自研
代表产品	龙芯1/龙芯2/龙芯3	鲲鹏920	腾云S系列、腾锐D系列、腾珑E系列	海光1号、2号、3号、4号	ZX-C、ZX-D、KX-5000、KX-6000、KH-20000	申威SW1600、SW1610、SW26010
产品覆盖领域	桌面、服务器	服务器、桌面、嵌入式	服务器、桌面、嵌入式	服务器	服务器、桌面、嵌入式	服务器、桌面
应用市场	党政市场	党政+商用市场	党政+商用市场	党政+商用	党政+商用	军方+党政
厂商	台式机：曙光、联想、方正、同方等 服务器：云海麒麟、五舟科技、清华同方、长城等 笔记本：方正、同方、山东超越、北京计算机研究所等	服务器：华为	台式机：长城 笔记本：长城 服务器：清华同方、浪潮、联想、长城等	服务器：中科曙光	台式机：联想、同方 笔记本：联想 服务器：云海麒麟、火星舱、联想、秉时	服务器：ZoomServer、云海麒麟、联想、方正、宝德等 笔记本：方正等
实际应用	玲珑、逸珑、福珑、北斗导航卫星	华为服务器	天河一号、天河二号、天河三号	国家级超算项目	笔记本、服务器、火星舱存储系统	神威蓝光、神威-太湖之光
代工厂	慧法半导体	台积电	台积电	格罗方德、三星	台积电	中芯国际
最小制程	28nm	7nm	16nm	14nm	16nm	28nm

资料来源：华经情报网，智东西，方正证券研究所整理

本章主要内容

- 6.1 中央处理器概述
- 6.2 指令周期
- 6.3 数据通路及指令操作流程
- 6.4 时序与控制
- 6.5 硬布线控制器
- 6.6 微程序控制器
- 6.7 异常与中断处理



CPU概述

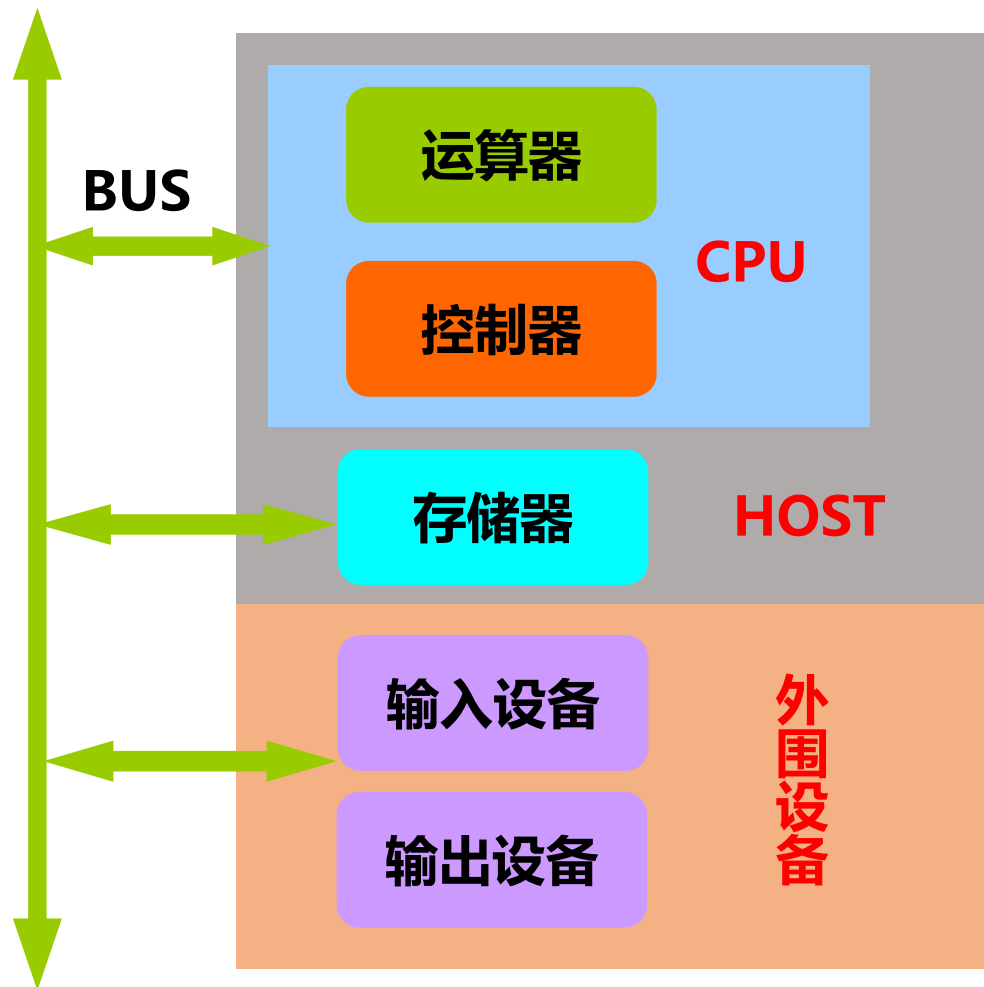
■ 冯诺依曼计算机

◆ 运算器、控制器

◆ 存储程序、程序控制

■ 计算机工作的过程就是运行程序的过程，实质上就是由控制器根据程序对应的机器指令序列逐条执行指令的过程

□ CPU上电复位后即开始不断地进行取指令、执行指令



CPU主要功能

■ 取指令并执行指令的部件-----CPU

◆ 数据处理: 算术/逻辑运算

运算器

◆ 程序控制: 程序中指令执行顺序控制

◆ 操作控制: 将机器指令翻译成执行部件所需操作控制信号

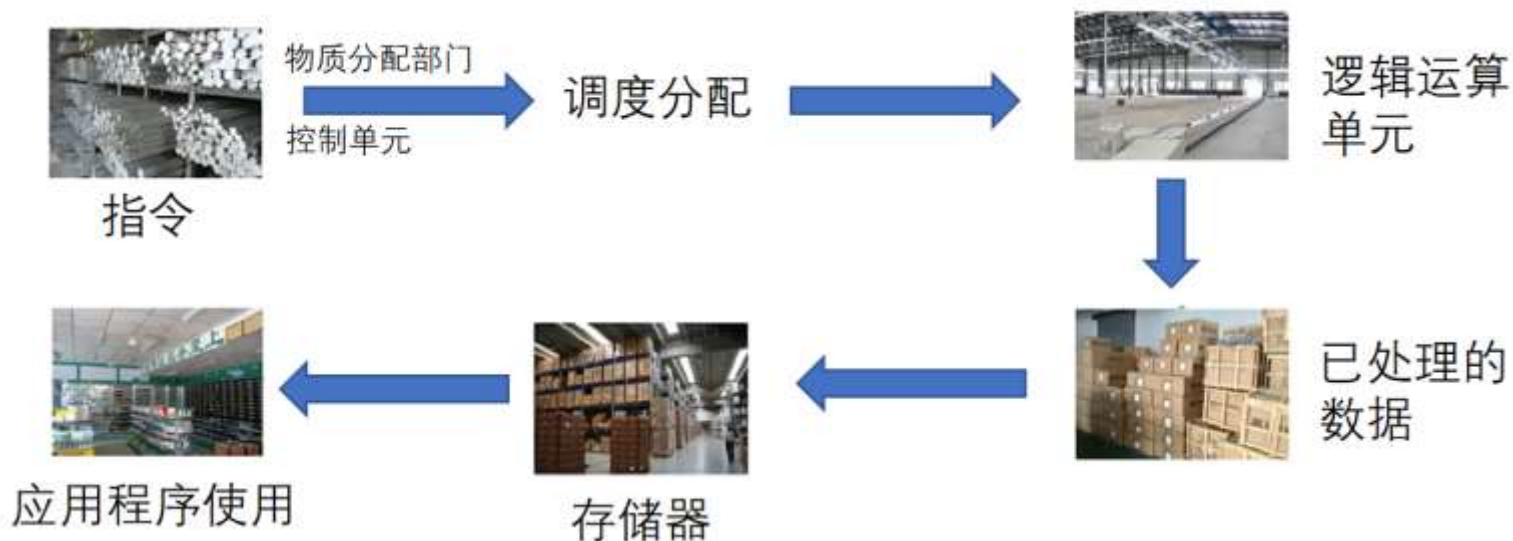
◆ 时序控制: 控制操作信号的产生时间、持续时间

◆ 异常控制: 异常处理, 外设交互

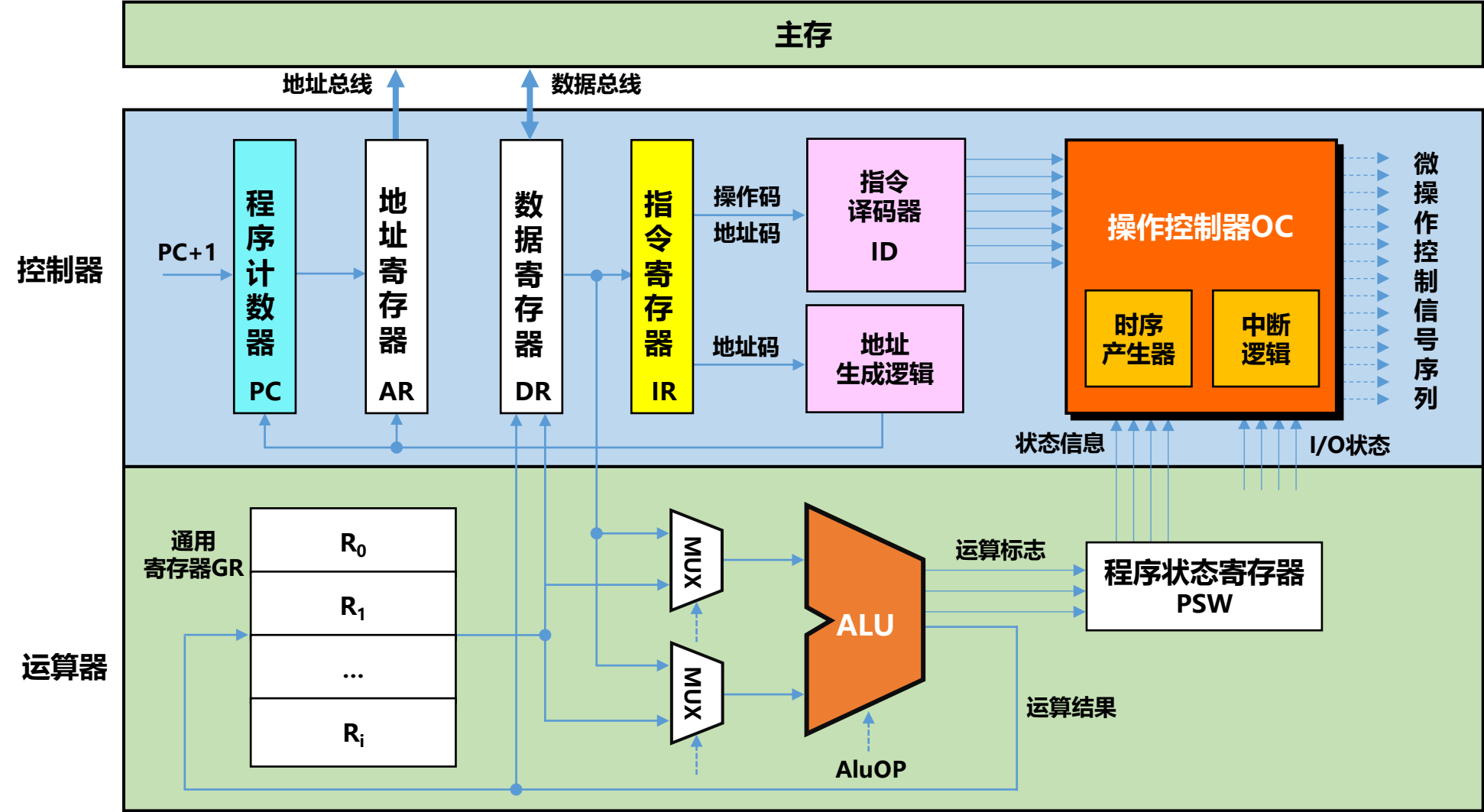
控制器

CPU主要功能

- CPU的工作原理就像一个工厂对产品的加工过程
 - 进入工厂的原料（指令）
 - 经过物资分配部门（控制单元）的调度分配
 - 被送往生产线（逻辑运算单元）
 - 生产出成品后（处理后的数据）
 - 再存储再仓库（存储器）中
 - 最后等着拿到市场去销售（交由应用程序使用）



简单CPU模型



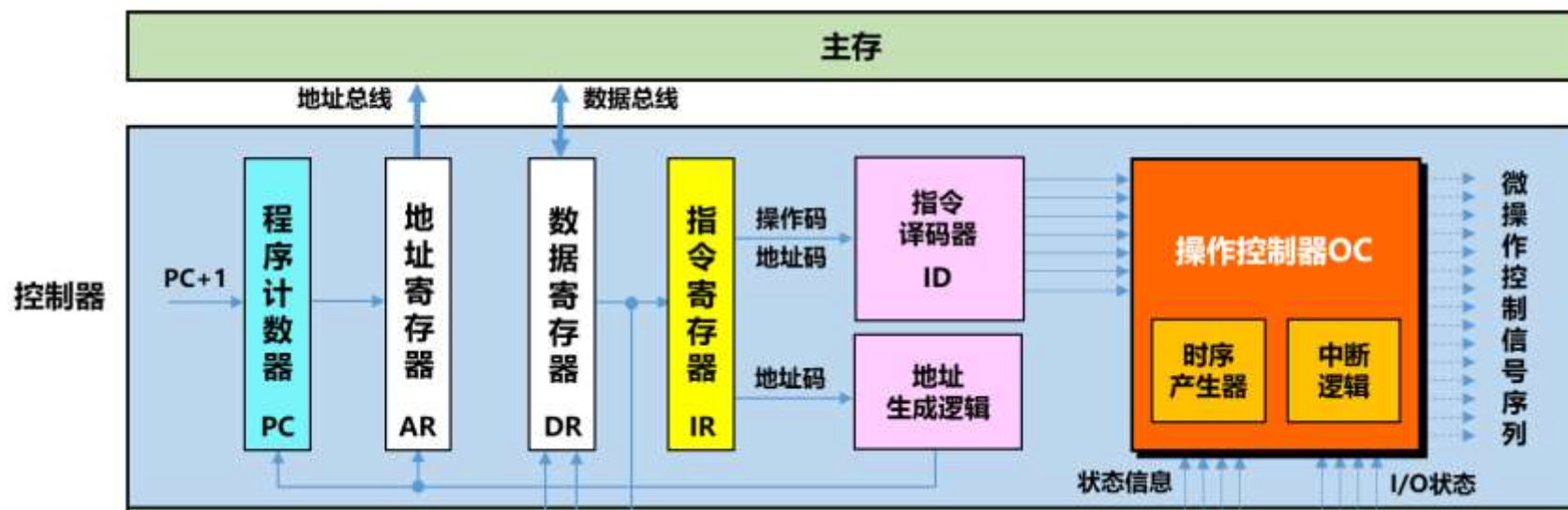
CPU中主要寄存器

- PC (Program Counter)-----程序计数器 X86: IP MIPS: PC
- IR (Instruction Register)-----指令寄存器 [可选]
- AR (Address Register)-----地址寄存器 MAR [可选]
- DR (Data Register)-----数据缓冲寄存器 MDR [可选]
- AC (Accumulate Count)-----累加寄存器 [可选]
- PSW (Program Status Word)-----程序状态字 [可选]
 - X86: EFLAGS MIPS: 无

CPU中主要寄存器

■ PC (Program Counter)-----程序计数器

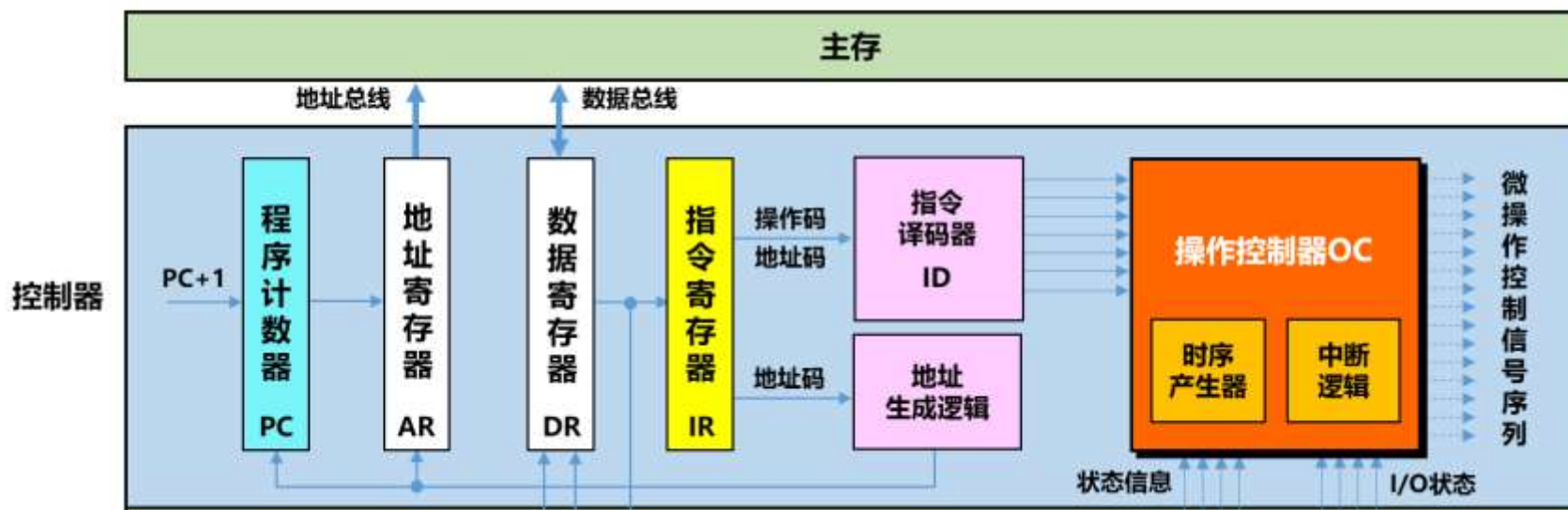
- 保存将要执行的指令的字节地址
- PC 位宽与主存地址总线位宽相同
- CPU 取指令时，会利用PC 的内容作为地址访问主存，并将从主存取出的指令字送入指令寄存器中，然后还需要修改PC的值以形成下一条指令的地址



CPU中主要寄存器

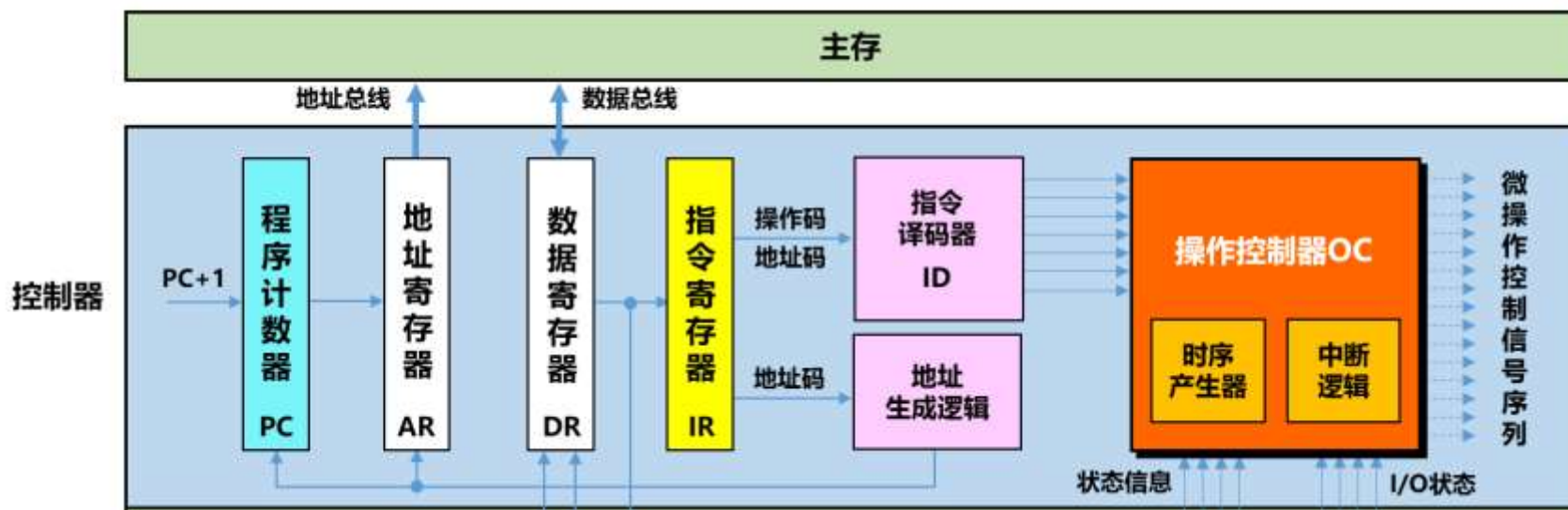
■ 存储器地址寄存器AR), 简称为地址寄存器

- 保存CPU访问主存的单元地址，无论CPU是取指令还是存取数据，都必须先将要访问地址送入AR，直到读写操作完成。
- AR位宽和主存地址总线位宽相同。
- AR并不是必需的，部分计算机中可直接将访存地址加载在地址总线上实现访存。



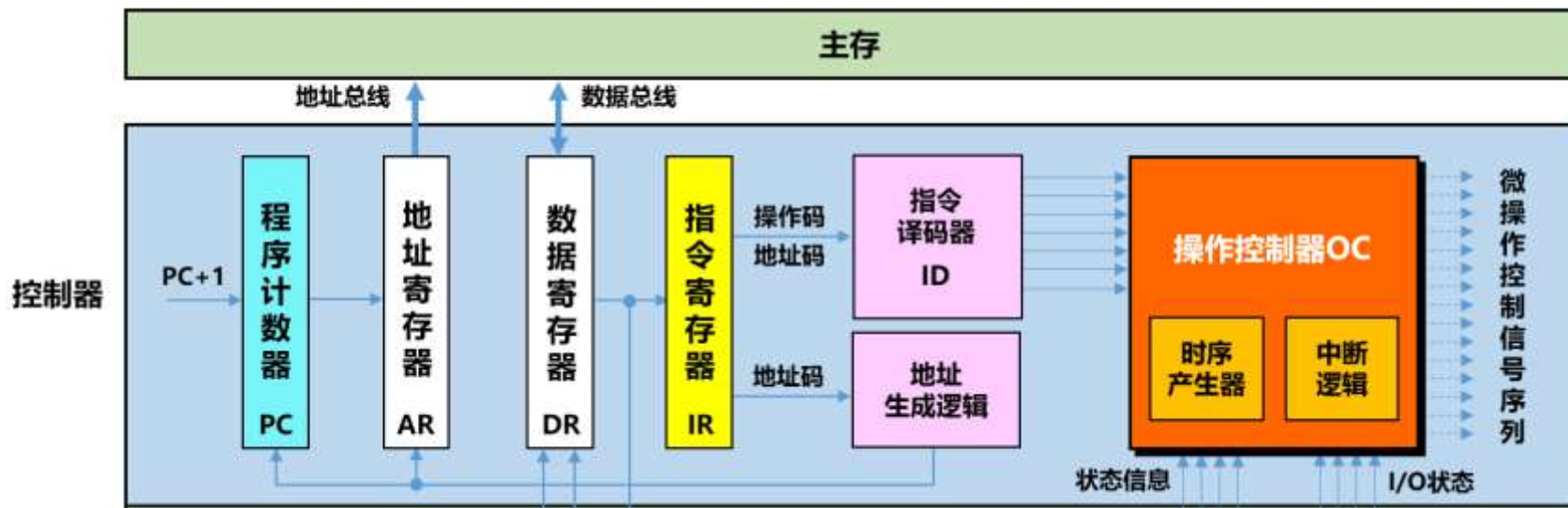
CPU中主要寄存器

- 存储器数据寄存器(Memory Data Register, DR), 简称为数据寄存器
 - 存放从主存中读出的数据或准备写入主存的数据
 - 其数据位宽与机器字长相同。
 - 作为CPU和主存之间的缓冲寄存器也可用于存放运算器ALU 的操作数、运算结果或中间结果, 以减少访问主存的次数
 - DR作为存储器的访问接口也不是必需的, 具体与CPU结构有关。



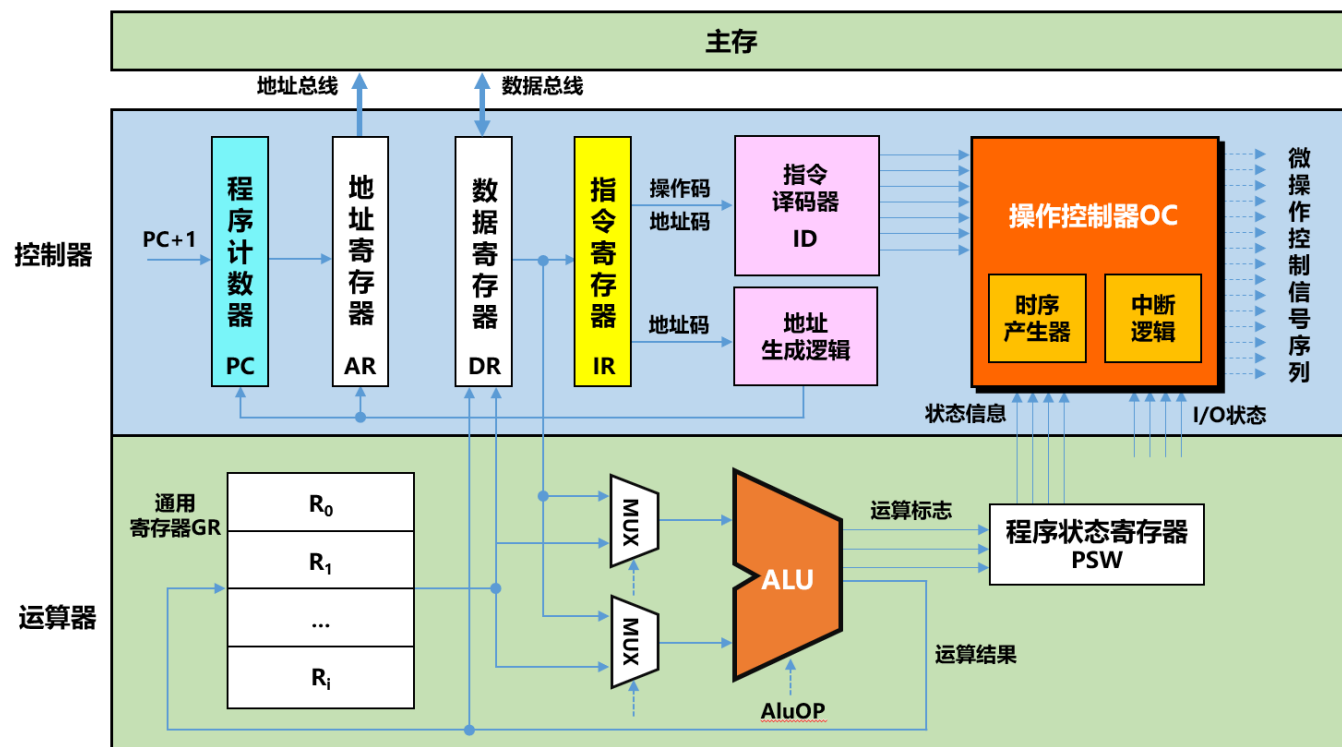
CPU中主要寄存器

- 指令寄存器(Instruction Register, IR)
 - 保存当前正在执行的指令
 - 从主存中取出的指令字存放在IR 中，其位宽和指令字相同
 - 指令字由指令译码器(ID)翻译成若干个指令译码信号(每一个指令译码信号表示一条不同的指令，同一时刻只有一个信号有效)
 - 指令字中的地址码部分由地址生成逻辑对寻址方式进行译码并生成目标地址或数据，根据寻址方式的不同将目标地址送入程序计数器PC、地址寄存器AR或运算部件。



CPU中主要寄存器

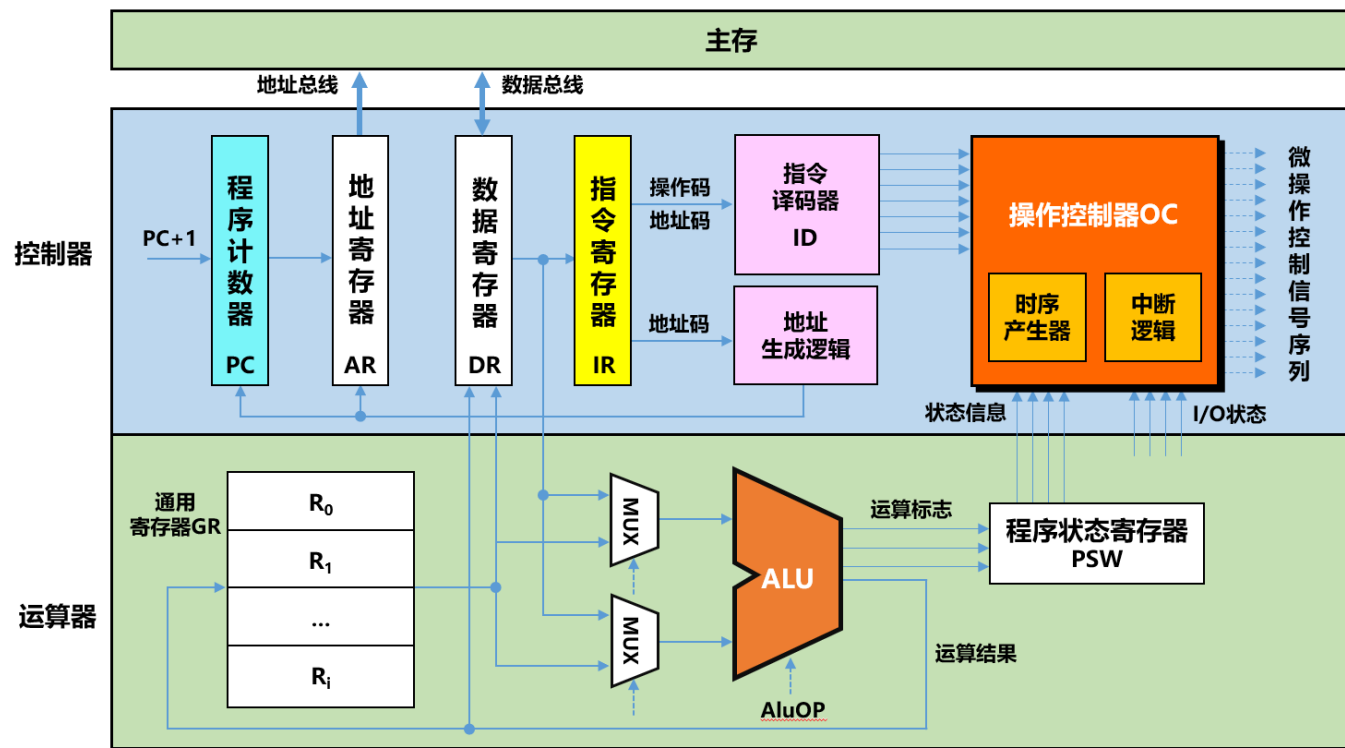
- 通用寄存器组(General Registers, GR), 运算器内部的若干寄存器, 又称寄存器堆
 - ALU的累加器、变址寄存器、基址寄存器、地址指针、数据缓冲器, 用于存放操作数、中间结果以及各种地址信息等
 - 增加通用寄存器的数量, 既可减少访问主存的次数, 从而提高CPU 的处理效率, 又可以方便汇编编程以及编译器生成代码。



CPU中主要寄存器

■ 程序状态字寄存器(Program Status Word/Register, PSW/PSR)

- 保存由算术运算指令、逻辑运算指令、测试指令等建立的各种条件标志
- 常见的状态信息包括进位标志(C)、溢出标志(V)、结果为负数标志(S)及结果为零标志(Z)等



操作控制器

- 取指令，将机器指令译码并生成执行部件所需的控制信号序列，控制信号按序送至各执行部件控点，引起逻辑门开闭，建立正确的数据通路，从而完成指令功能。
 - 硬布线控制器（时序逻辑型）（硬件实现）
 - 微程序控制器（存储程序型）（软件实现）

时序产生器

- 时序产生器：对完成指令而执行的微操作控制信号进行时间调制，严格规定各信号的产生时间和持续时间。
 - 执行加法指令时，必须先将操作数送到 ALU的输入端，然后给出ALU执行加法的运算选择控制信号，待加法操作完成后，才能将结果送往目的地
- 对什么时刻送操作数、什么时刻执行加法操作、什么时刻送出结果也有规定。

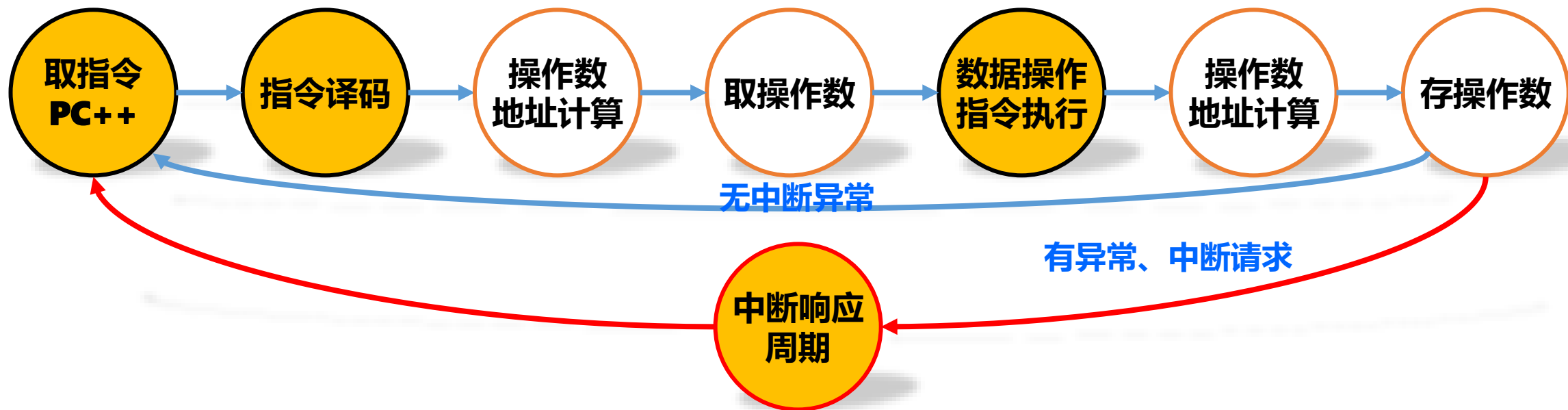
本章主要内容

- 6.1 中央处理器概述
- **6.2 指令周期**
- 6.3 数据通路及指令操作流程
- 6.4 时序与控制
- 6.5 硬布线控制器
- 6.6 微程序控制器
- 6.7 异常与中断处理



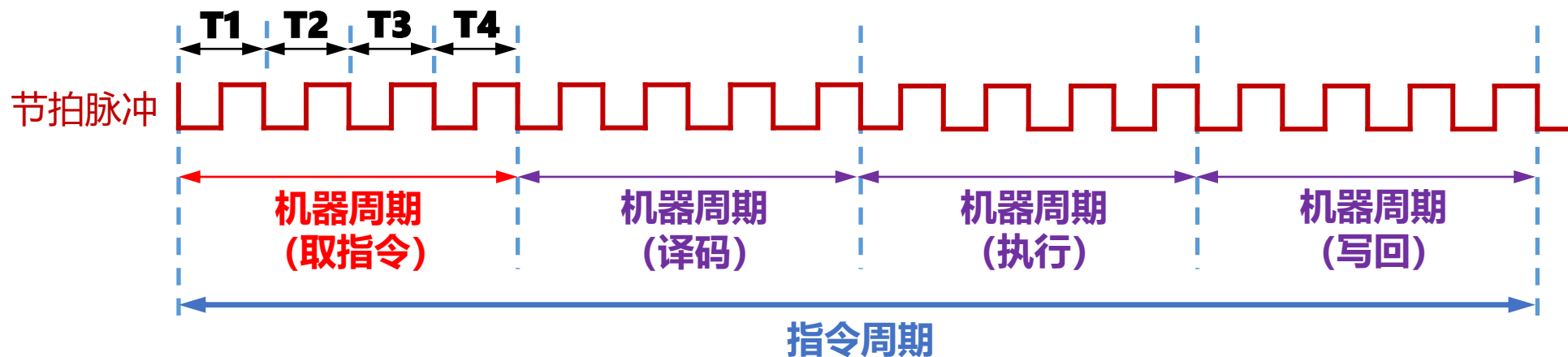
指令执行一般流程

- 计算机工作的过程就是运行程序的过程，控制器依据事先编制好并存放在主存中的程序控制各功能部件协调工作
- 取指令、执行指令反复循环
- 指令功能、寻址方式不同，数据通路不同，执行时间不同
- 访存指令、寄存器运算指令、加法指令、除法指令



指令周期基本概念

- 时钟周期 = 节拍脉冲 = 震荡周期 能完成一次微操作
- 机器周期 = CPU周期 从主存读出一条指令的最短时间 可完成 复杂操作
- 指令周期：从主存取一条指令并执行指令的时间
 - ◆ 取指令周期、执行指令周期 （间址周期、中断周期）
 - ◆ 指令周期包含n个机器周期，机器周期包含k个节拍



指令周期同步

■ 不同指令功能不同，复杂度不同，如何进行时间控制？

◆ 定长指令周期： 早期三级时序系统

◆ 机器周期数**固定**，节拍数**固定**，按**机器周期**同步，mips单周期

◆ 变长指令周期： 现代时序系统

◆ 机器周期数**可变**，节拍数**可变**，按**时钟周期**同步，mips多周期

◆ 定长指令周期只是一种特例

本章主要内容

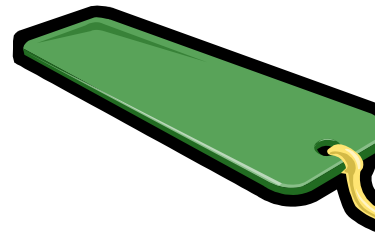
- 6.1 中央处理器概述
- 6.2 指令周期
- **6.3 数据通路及指令操作流程**
- 6.4 时序与控制
- 6.5 硬布线控制器
- 6.6 微程序控制器
- 6.7 异常与中断处理



数据通路 DataPath

- 数据通路-----数据在功能部件间传送的路径（数据流）
 - ◆通路的建立由控制信号控制，受时钟驱动（控制流）
 - ◆不同指令、同一指令在执行的不同阶段的数据通路不同
 - ◆分类：共享通路（总线）、专用通路
 - ◆指令执行流程、执行效率
 - ◆微操作控制信号的时序安排

数据通路的功能部件



■ 数据处理单元：

- 组合逻辑电路构成，其输出只与当前的输入有关，负责对数据进行加工处理，如ALU、符号扩展单元、译码器等

■ 状态存储单元

- 带有存储功能的单元，如存储器和寄存器。
- 状态单元一般包含输入端、写使能控制端、时钟输入端和输出端。数据只有在写使能信号有效且时钟脉冲到来时才能写入状态单元(边沿触发)。

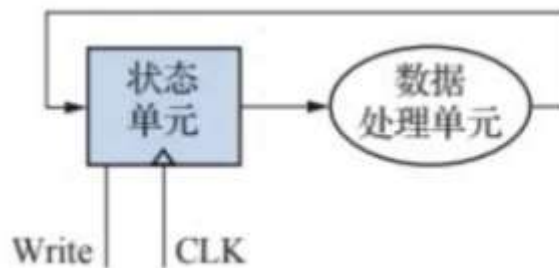
数据通路抽象模型（寄存器传输）

- 通用数据通路模型可以利用一个数据处理单元连接两个状态单元的方式构成



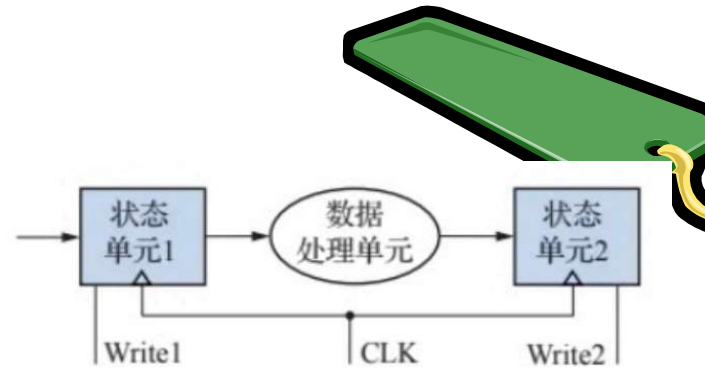
数据处理单元从前一个状态单元接收数据，经过数据处理单元的处理后将结果送后一个状态单元保存

- 简化之后的数据通路模型，其将数据处理单元处理的结果重新传回同一个状态单元中

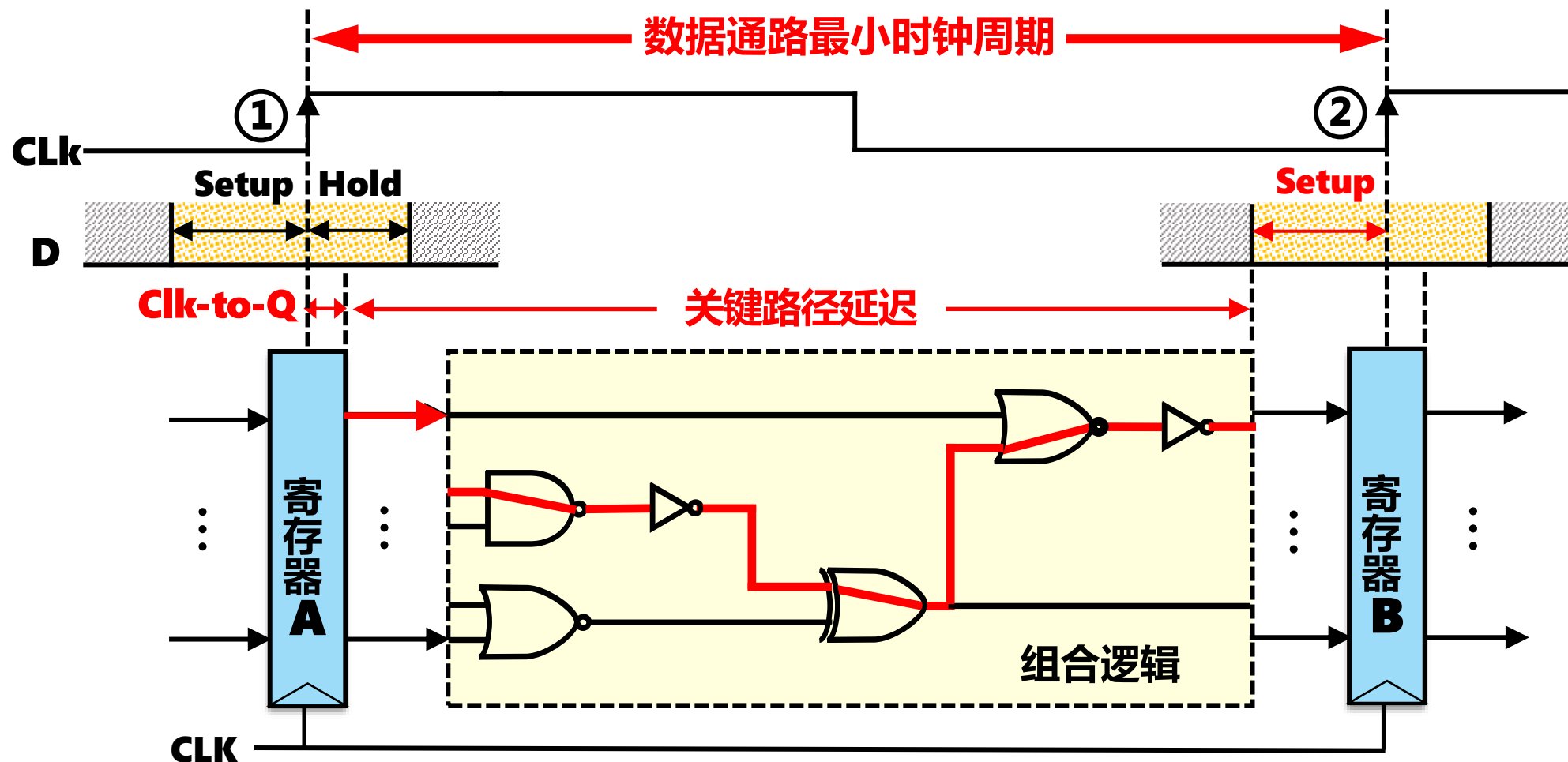


数据通路的时序分析

- 假设状态单元1和状态单元2分别对应寄存器A和B
- **寄存器延迟** $T_{clk_to_q}$: 从时钟上跳沿①时刻到寄存器A输出稳定的时间
- **关键路径延迟** T_{max} : 数据处理单元对寄存器 A 的输出数据进行处理加工时间
- **寄存器建立时间** T_{setup} : 寄存器 B 将寄存器A经过组合逻辑传递过来的数据正确锁存, 在下一个时钟上跳沿到来之前, 还需要让输入数据保持一段稳定时间
- **寄存器保持时间** T_{hold} : 另外时钟上跳沿到来之后输入数据还需要一段稳定时间

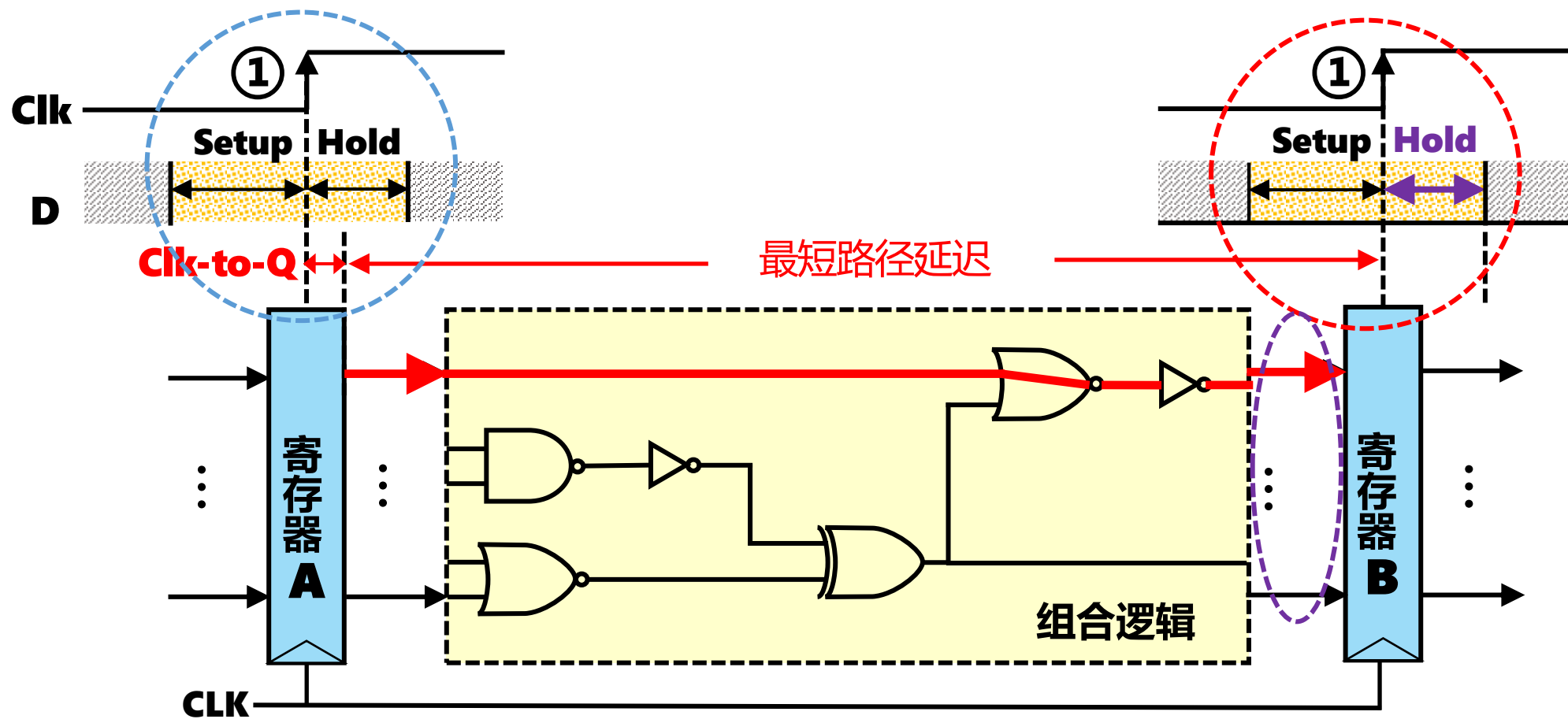


数据通路与时钟周期



■ **最小时钟周期** > **Clk_to_Q** + **关键路径时延** + **Setup Time**

保持时间违例



- A寄存器锁存的新值经过寄存器延迟 $T_{clk_to_q}$ 及组合逻辑最短路径延迟 T_{min} 后就会到达寄存器B 的输入端
- 寄存器 B 要正确锁存，时钟上跳沿②之前的输入数据还需要保持一段稳定时间 T_{hold} ，寄存器保持时间应该满足

$$Clk_to_Q + \text{最短路径时延} > \text{Hold_Time}$$

数据通路分类

■ 共享通路（总线型）

- ◆ 主要部件都连接在公共总线上，各部件间通过总线进行数据传输
- ◆ 结构简单，实现容易，但并发性较差，需分时使用总线，效率低

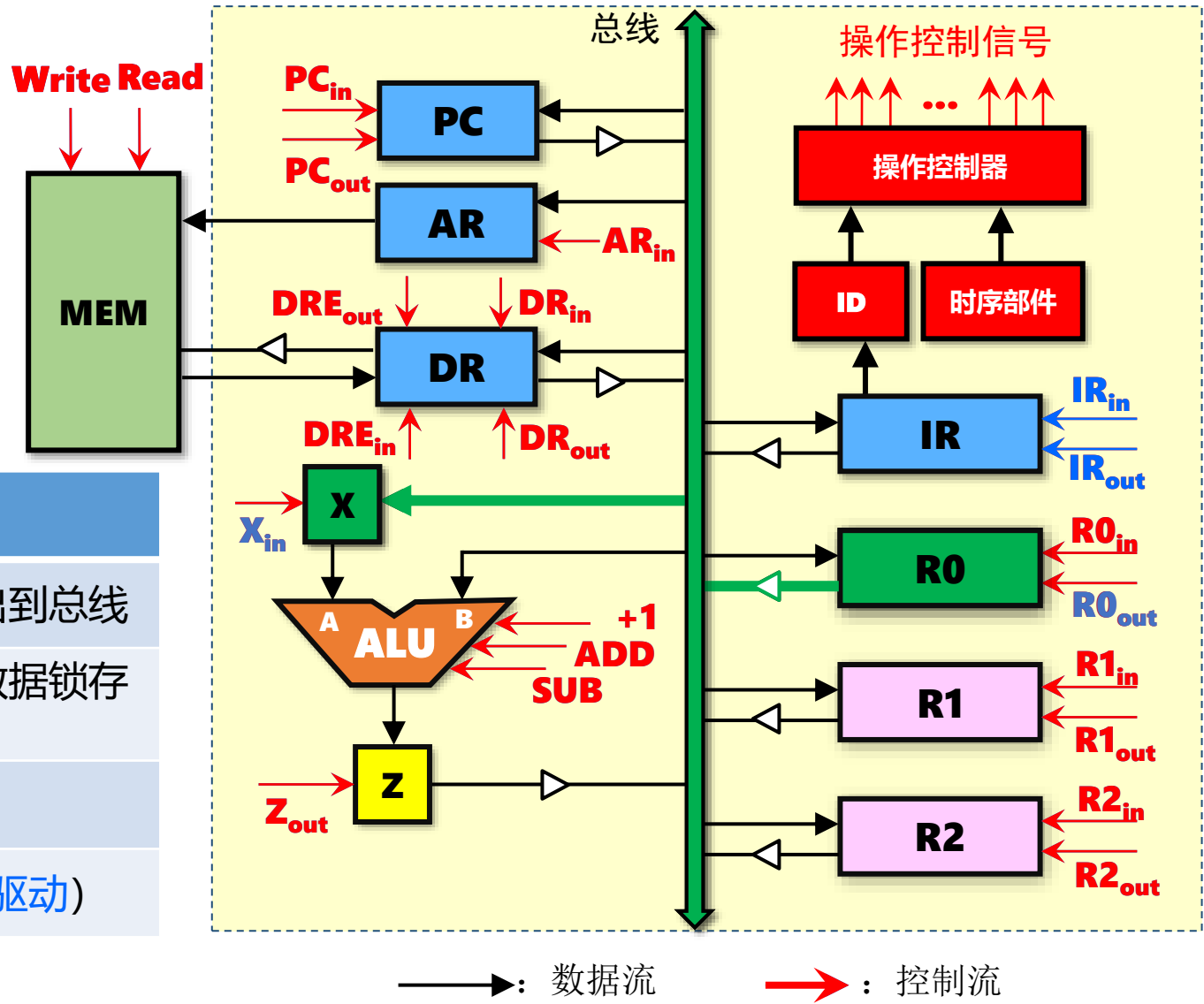
■ 专用通路

- ◆ 并发度高，性能佳，设计复杂，成本高
- ◆ 可以看作多总线结构

单总线结构CPU实例

- 主要部件都连接在总线上
- 各部件间通过总线进行传输
- 可以有多个模块同时接收数据，但某一时刻只能有一个模块向总线发送数据

控制信号	作用说明
IR_{out} 、 PC_{out} 、... $R1_{out}$	控制三态门将寄存器值输出到总线
IR_{in} 、 PC_{in} 、... $R1_{in}$	控制寄存器使能端将总线数据锁存 (时钟驱动)
$+1$ 、 ADD 、 SUB	运算控制信号
$Write$ 、 $Read$	内存读写控制信号 (时钟驱动)



单总线结构MIPS CPU 典型指令

#	指令	指令功能 (RTL描述)
1	<code>lw rt,imm(rs)</code>	$R[rt] \leftarrow M[R[rs] + \text{SignExt}(imm)]$, 读取
2	<code>sw rt,imm(rs)</code>	$M[R[rs] + \text{SignExt}(imm)] \leftarrow R[rt]$, 写入
3	<code>beq rs,rt,imm</code>	$\text{if}(R[rs] == R[rt]) \text{ PC} \leftarrow \text{PC} + 4 + \text{SignExt}(imm) \ll 2$, 条件分支
4	<code>addi rt,rs,imm</code>	$R[rt] \leftarrow R[rs] + \text{SignExt}(imm)$, 立即数加
5	<code>add rd,rs,rt</code>	$R[rd] \leftarrow R[rs] + R[rt]$, R型

lw指令数据通路

- 从主存读取一个32位的存储器字，汇编代码为lw rt,imm(rs)

$M[PC++]\rightarrow IR$

- 访存地址为rs字段对应的寄存器加16位有符号立即数imm，变址寻址，

lw从对应的主存单元取出4个字节送入rt寄存器中保存



lw指令数据通路

- 取指周期
- 计算周期
- 执行周期

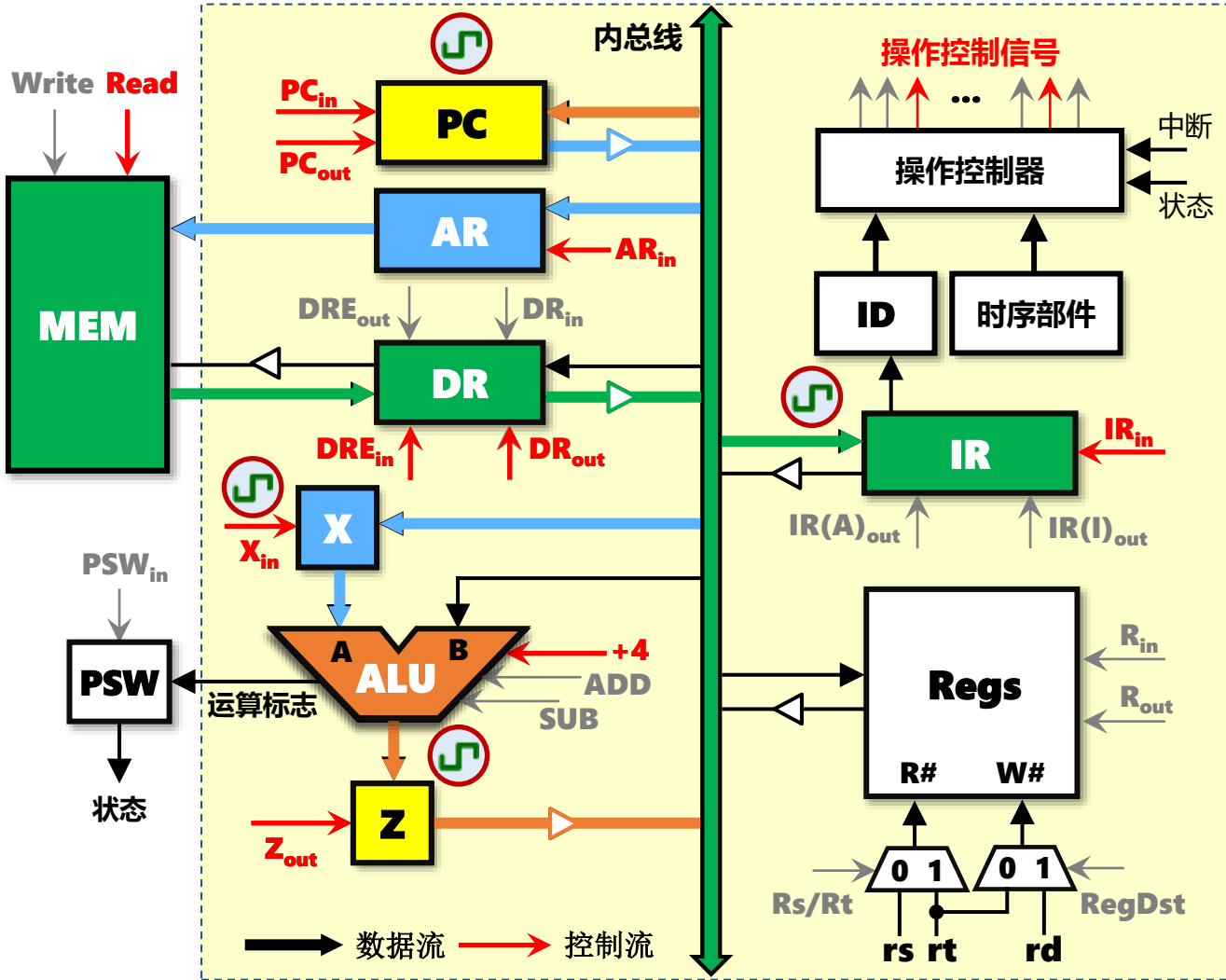
Iw指令数据通路

■ 取指周期

$M[PC++] \rightarrow IR$

节拍	数据通路 (数据流)	控制信号(控制流)
T1	PC→AR, PC→X	PC _{out} , AR _{in} , X _{in}
T2	X+4→Z	+4
T3	Z→PC, M[AR]→DR	Z _{out} , PC _{in} , DRE _{in} , Read
T4	DR→IR	DR _{out} , IR _{in}

T1: 将程序计数器PC内容送入AR, 同时送入暂寄存器X
T2: 将PC值加4并送入暂寄存器Z
T3: 将暂寄存器Z内容回送到PC, 同时读AR内容对应主存单元的值并送入DR
T4: 将DR内容送入IR, 完成取指令



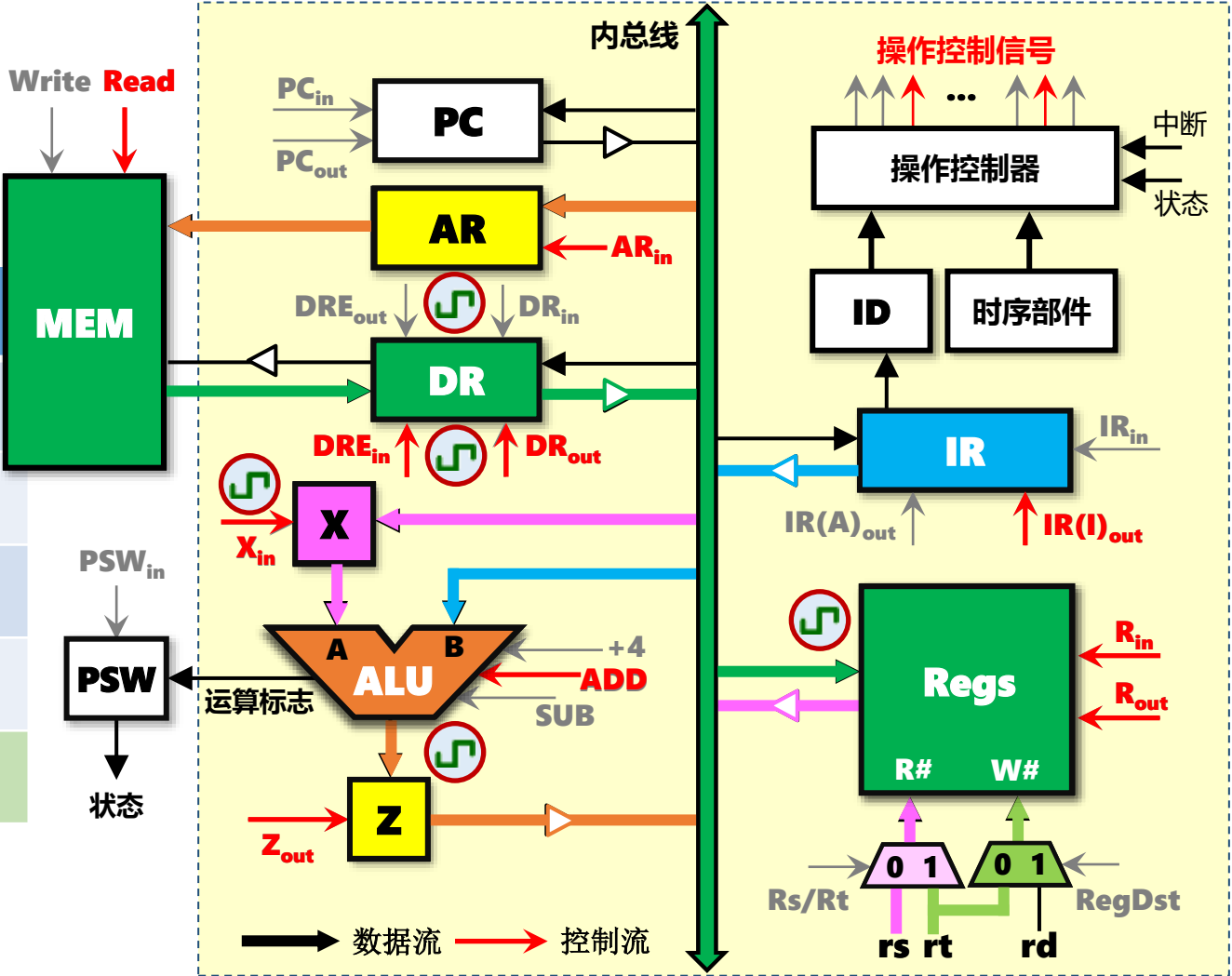
LW 指令执行数据通路 $Lw\ rt, imm(rs)$



$M[R[rs] + imm] \rightarrow R[rt]$

节拍	数据通路 (数据流)	控制信号(控制流)
T5	$R[rs] \rightarrow X$	R_{out}, X_{in}
T6	$X + imm \rightarrow Z$	$IR(I)_{out}, ADD$
T7	$Z \rightarrow AR$	Z_{out}, AR_{in}
T8	$M[AR] \rightarrow DR$	$DRE_{in}, Read$
T9	$DR \rightarrow R[rt]$	DR_{out}, R_{in}

- 计算周期** (T5、T6) 与 **执行周期** (T7—T9)
- T5: 将rs寄存器内容送入暂存器X, 准备计算访存地址
 - T6: 将IR中的立即数符号扩展为32位并送入ALU做加法运算
 - T7: 将Z中暂存的访存地址送入AR
 - T8: 读AR内容对应主存单元的值并送入DR
 - T9: 将DR内容送入寄存器rt



sw指令数据通路

- 在主存中写入一个32 位的存储器字，汇编代码为 sw rt,imm(rs)

$$R[rt] \rightarrow M[R[rs] + imm]$$

- 其访存地址是rs寄存器加16位有符号立即数imm，sw指令将**rt寄存器**的值写入该地址对应的主存单元



sw指令数据通路

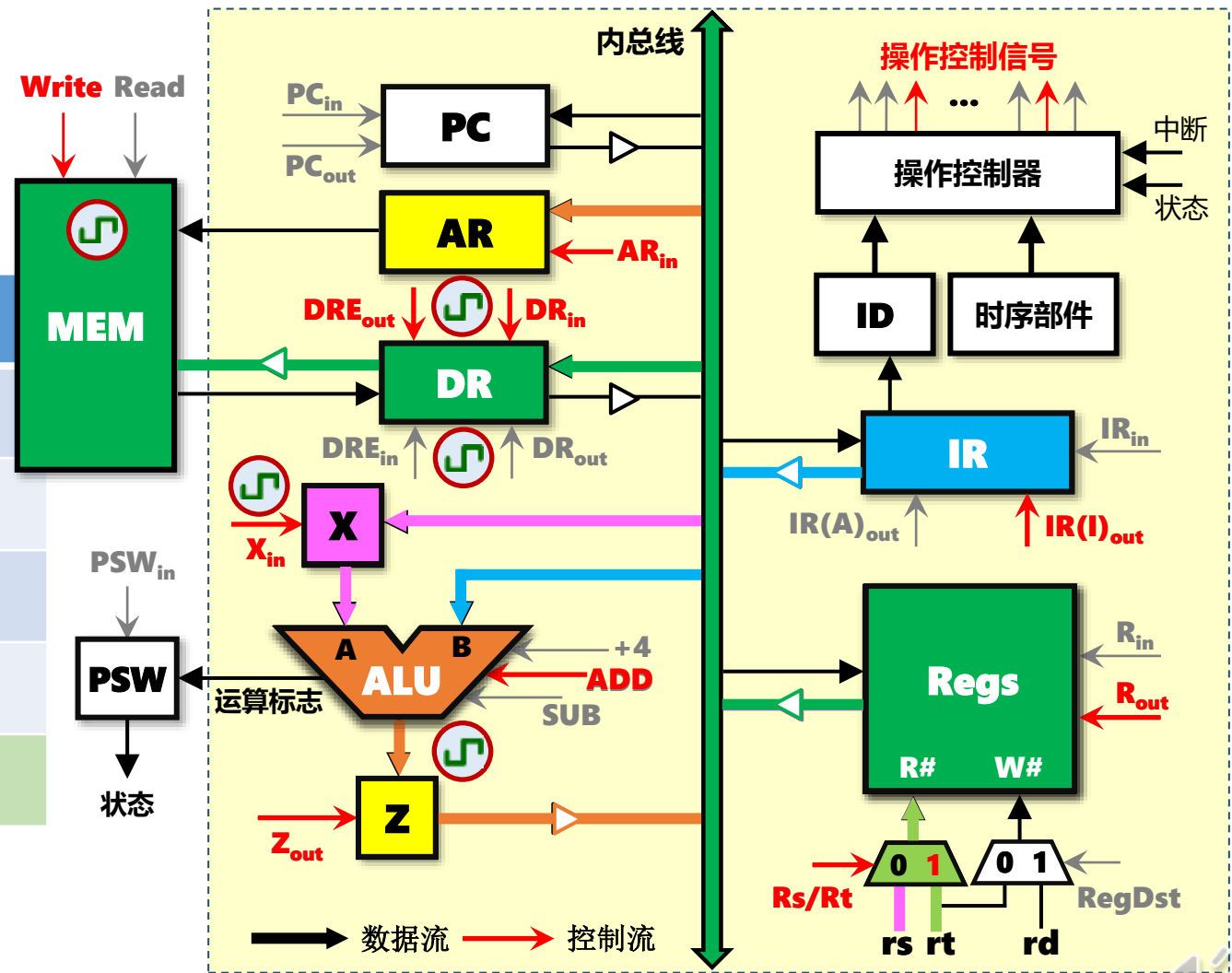
- 取指周期
- 计算周期
- 执行周期

SW 指令执行数据通路 $sw\ rt, imm(rs)$

前两个周期与lw相同

节拍	数据通路 (数据流)	控制信号(控制流)
T5	$R[rs] \rightarrow X$	R_{out}, X_{in}
T6	$X + imm \rightarrow Z$	$IR(I)_{out}, ADD$
T7	$Z \rightarrow AR$	Z_{out}, AR_{in}
T8	$R[rt] \rightarrow DR$	$R_{out}, Rs/Rt, DR_{in}$
T9	$DR \rightarrow M[AR]$	$DRE_{out}, Write$

- T7: 将Z中暂存的访存地址送入AR
- T8: 将rt寄存器内容送入DR
- T9: 将DR内容写入AR内容所指的主存单元



beq指令数据通路

- 条件分支指令，汇编代码为 beq rs,rt,imm

if ($R[rs] = R[rt]$)

$PC + 4 + imm \ll 2 \rightarrow PC$

- **比较寄存器rs和rt的值，如果相等则进行分支跳转**
- imm 的值表示分支目标地址相对下一条指令也就是PC+4的指令条数，所以计算分支目标地址时应该将PC的值(取指令阶段已更新为PC+4)与**imm符号扩展为32位再左移两位后**的值相加，这里左移两位的目的是计算字节偏移量



beq指令数据通路

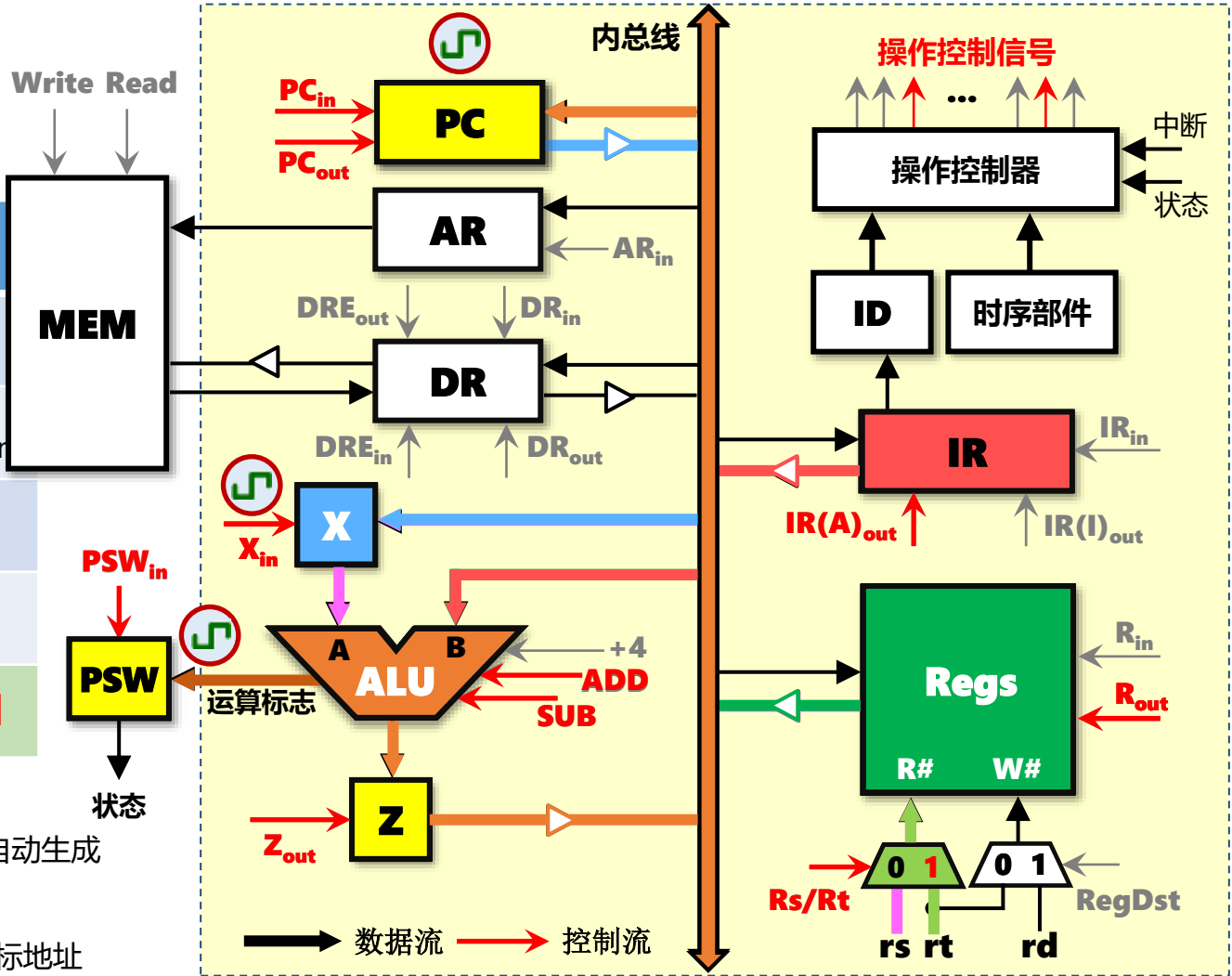
- 取指周期
- 计算周期
- 执行周期

BEQ 指令数据通路 *beq rs,rt,imm*

取指令周期与lw相同

节拍	数据通路 (数据流)	控制信号(控制流)
T5	$R[rs] \rightarrow X$	R_{out}, X_{in}
T6	$X - R[rt] \rightarrow PSW$	$R_{out}, Rs/Rt, SUB, PSW_{in}$
T7	$PC \rightarrow X$	PC_{out}, X_{in}
T8	$IR(A) + X \rightarrow Z$	$IR(A)_{out}, ADD$
T9	If (PSW.equal) $Z \rightarrow PC$	$Z_{out}, PC_{in} = PSW.equal$

T5: 将rs寄存器内容送入暂存器X, 准备进行比较
T6: 将rt寄存器内容送入ALU做减法, 可产生结果为零的标志位。本书中由ALU自动生成 equal标志并送入PSw, 所以不用关心ALU进行何种运算
T7: 将PC内容送入暂存器X
T8: 将IR中的立即数符号扩展为32位后左移两位送入ALU做加法, 计算出分支目标地址
T9: 如果equal标志位为1, 将分支目标地址送入PC



addi指令数据通路

- 立即数加指令，汇编代码为 `addi rt,rs,imm`

$$R[rs] + imm \rightarrow R[rt]$$

- 将rs 寄存器与立即数相加的结果送入rt 寄存器。该指令除取指周期外只需要一个机器周期，即将寄存器rs的内容送入X，再将指令字中的立即数符号扩展成32位后送入ALU进行加法运算，最后将结果送入rt寄存器

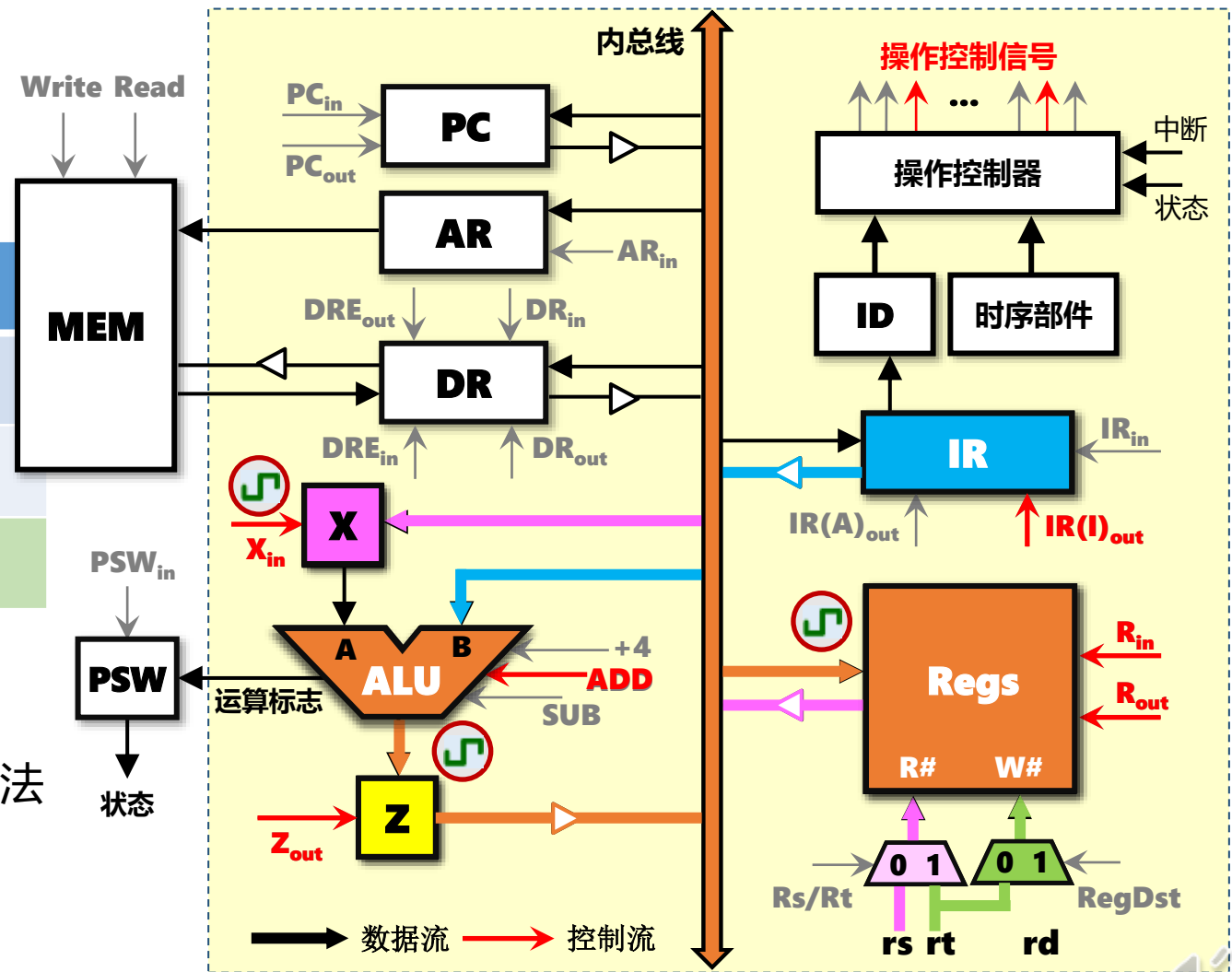


addi指令执行数据通路 *addi rt,rs,imm*

取指周期使用的数据通路与lw指令的数据通路相同

节拍	数据通路 (数据流)	控制信号(控制流)
T5	$R[rs] \rightarrow X$	R_{out}, X_{in}
T6	$X + imm \rightarrow Z$	$IR(I)_{out}, ADD$
T7	$Z \rightarrow R[rt]$	Z_{out}, R_{in}

T5: 将rs寄存器内容送入暂存器，准备进行加法运算
T6: 将IR中的立即数符号扩展成32位后送入ALU做加法
T7: 将暂存器乙的运算结果送入目的寄存器t



add指令数据通路

- R型 MIPS 指令，汇编代码为 add rd,rs,rt

$$R[rs] + R[rt] \rightarrow R[rd]$$

- 将rs 寄存器与rt寄存器相加的结果送入rd寄存器。该指令除取指周期外只需要一个机器周期，即将寄存器rs的内容送入X，再将寄存器rt的内容送入ALU进行加法运算，最后将结果送入rd寄存器

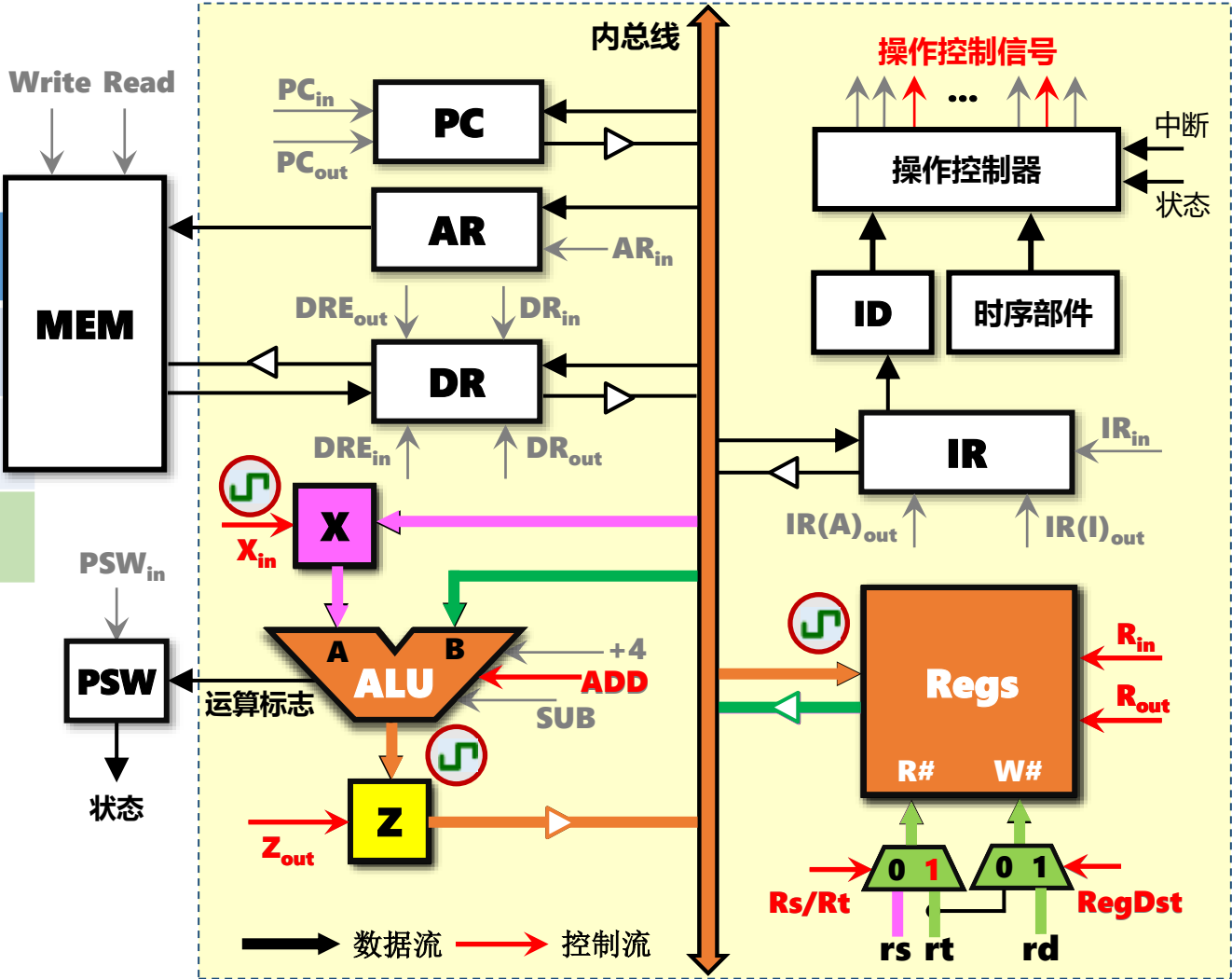


R型运算类指令执行数据通路 *add rd,rs,rt*

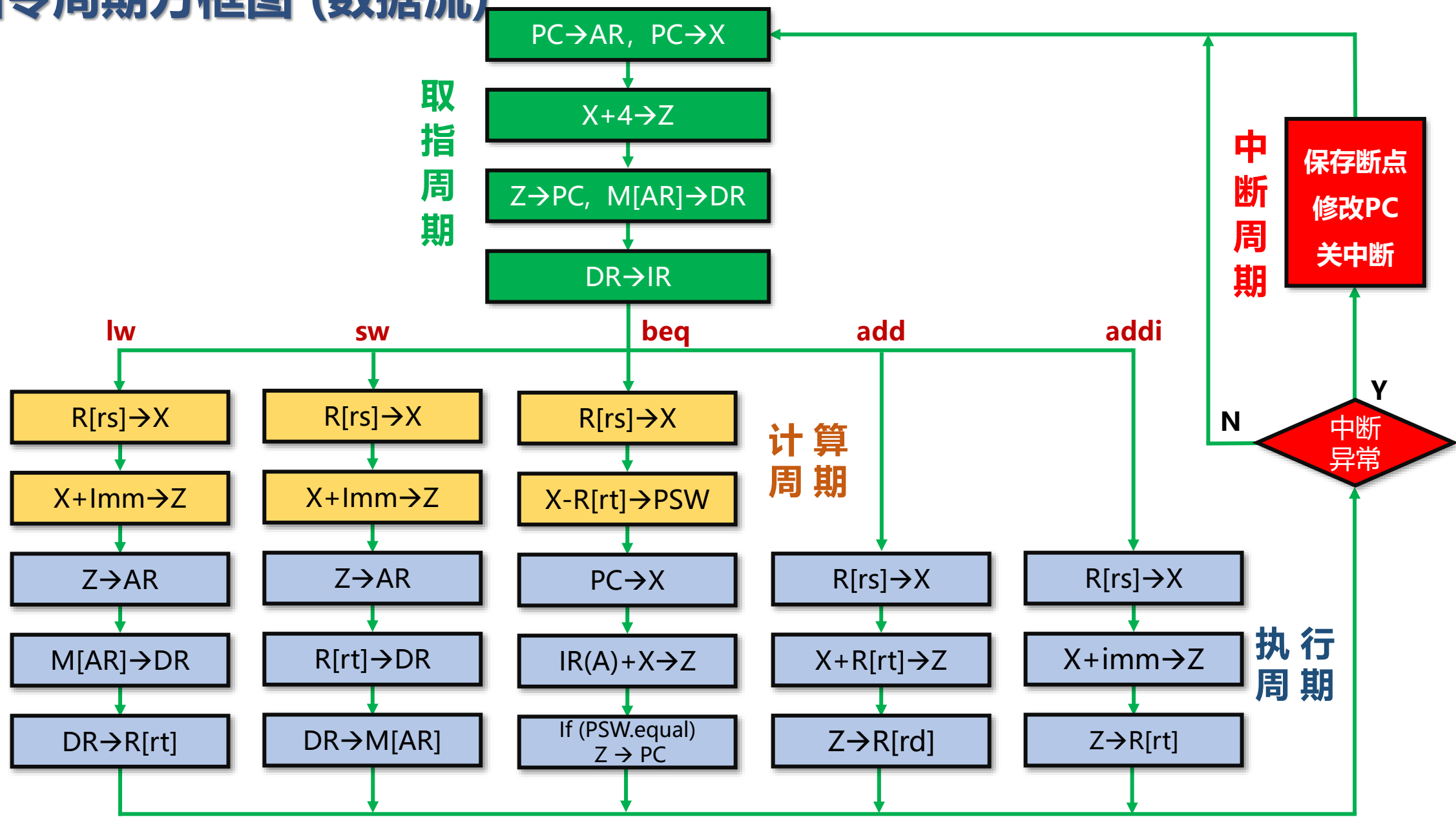
取指周期使用的数据通路与lw指令的数据通路相同

节拍	数据通路 (数据流)	控制信号(控制流)
T5	$R[rs] \rightarrow X$	R_{out}, X_{in}
T6	$X + R[rt] \rightarrow Z$	$Rs/Rt, R_{out}, ADD$
T7	$Z \rightarrow R[rd]$	$Z_{out}, R_{in}, RegDst$

T5: 将rs寄存器内容送入暂存器, 准备进行加法运算
T6: 将rt寄存器内容送入ALU进行加法运算, 结果送入Z
T7: 将暂存器Z的运算结果送入目的寄存器t



指令周期方框图 (数据流)



指令周期方框图 (数据流)

- 单总线结构计算机的最小时钟周期或时钟频率取决于不同机器周期、不同时钟节拍中最慢的关键路径时间延迟
- 单总线结构中最慢的数据通路为访存通路 $M[AR] \rightarrow DR$ 和运算通路 $ALU \rightarrow Z$
- 最小时钟周期

$$\text{最小时钟周期} = \text{Clk_to_Q} + \max(\text{alu}, \text{mem}) + \text{Setup Time}$$

■ 例题6.1

本章主要内容

- 6.1 中央处理器概述
- 6.2 指令周期
- 6.3 数据通路及指令操作流程
- **6.4 时序与控制**
- 6.5 硬布线控制器
- 6.6 微程序控制器
- 6.7 异常与中断处理



- 指令的正确执行依赖于数据通路的建立，而数据通路的建立依赖于操作控制器产生的控制信号。
- 在单周期处理器中，所有控制信号同时产生，没有先后顺序
- 在多周期处理器中，这些控制信号应该具有严格的时间先后顺序，才能保证指令的正确执行。
- **CPU时序系统**: 控制这些信号什么时候产生、持续多长时间就是控制信号的时序调制问题，
时序调制问题

6.4 时序与控制

■ 6.4.1 中央处理器时序

■ 6.4.2 控制方式

■ 6.4.3 时序发生器

单总线结构CPU指令周期（控制流）

取指令周期

节拍	控制信号(4 cycles)
T1	PC _{out} , AR _{in} , X _{in}
T2	+4
T3	Z _{out} , PC _{in} , DRE _{in} , Read
T4	DR _{out} , IR _{in}

■ 控制信号的生命周期

■ 何时开始，何时结束，哪些需要时钟配合

■ 如何进行时序调制？

■ 时序产生器 生成 机器周期，节拍周期，节拍脉冲

■ 控制信号被限定在具体机器周期，具体节拍，受具体节拍控制

计算周期

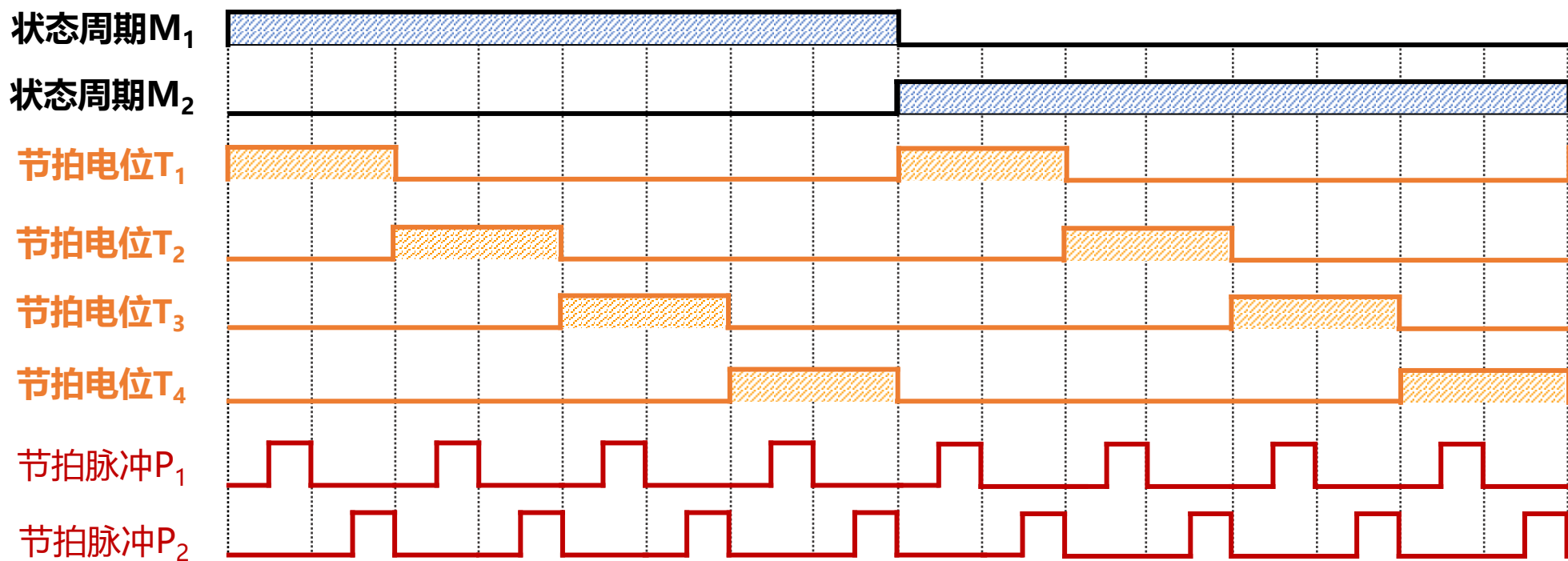
执行周期

节拍	lw (5 cycles)	sw (5 cycles)	beq (5 cycles)	add (3 cycles)	addi (3 cycles)
T1	R _{out} , X _{in}	R _{out} , X _{in}	R _{out} , X _{in}		
T2	IR(I) _{out} , ADD	IR(I) _{out} , ADD	R _{out} , Rs/Rt, SUB, PSW _{in}		
T1	Z _{out} , AR _{in}	Z _{out} , AR _{in}	PC _{out} , X _{in}	R _{out} , X _{in}	R _{out} , X _{in}
T2	DRE _{in} , Read	R _{out} , Rs/Rt, DR _{in}	IR(A) _{out} , ADD	Rs/Rt, R _{out} , ADD	IR(I) _{out} , ADD
T3	DR _{out} , R _{in}	DRE _{out} , Write	Z _{out} , PC _{in} =PSW.equal	Z _{out} , R _{in} , RegDst	Z _{out} , R _{in}

传统三级时序系统

- 状态周期、节拍电位、节拍脉冲
- 状态周期数、节拍电位数、节拍脉冲数可变
- 状态周期用电位来表示当前处于指令执行的哪个机器周期，节拍电位用电位表示当前处于机器周期的第几个节拍

现代时序只有时钟信号



6.4 时序与控制

■ 6.4.1 中央处理器时序

■ 6.4.2 控制方式

■ 6.4.3 时序发生器

控制方式

■ 同步控制

- 机器周期数、节拍数固定
- 机器周期数固定，节拍数不固定
- 大多数指令机器周期固定

3机器周期*4节拍

3机器周期，取指/计算/执行 4/2/3节拍

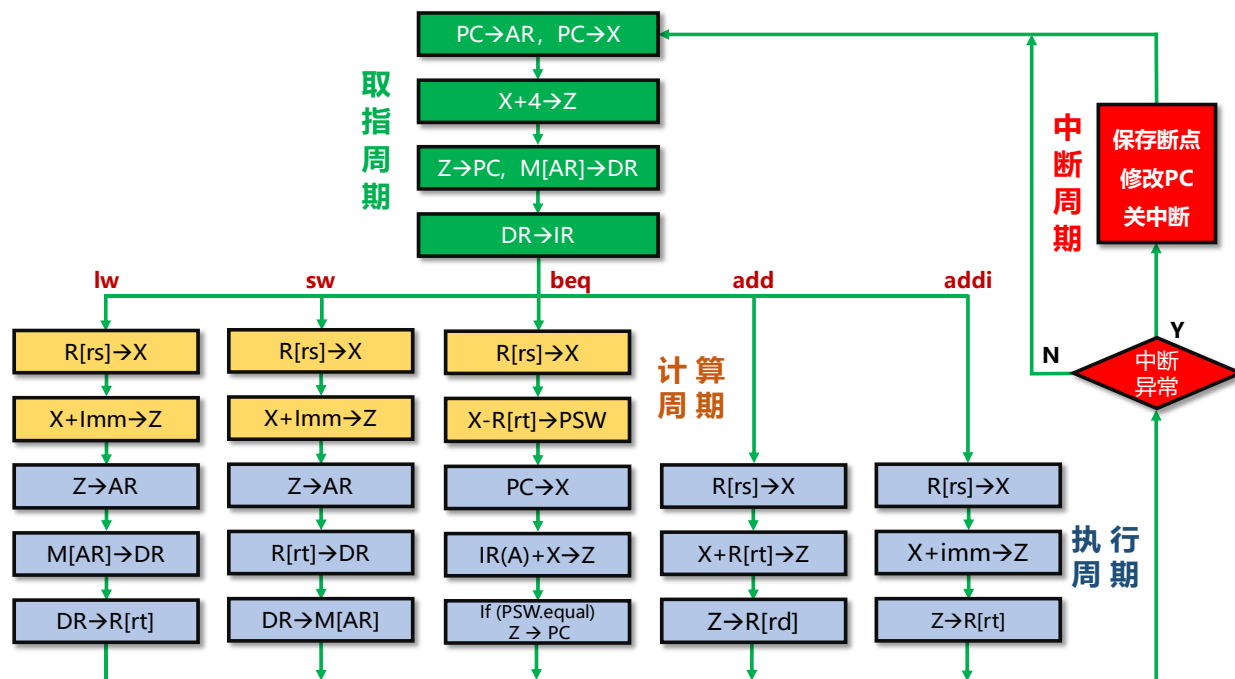
机器周期，节拍数按需分派

■ 异步控制

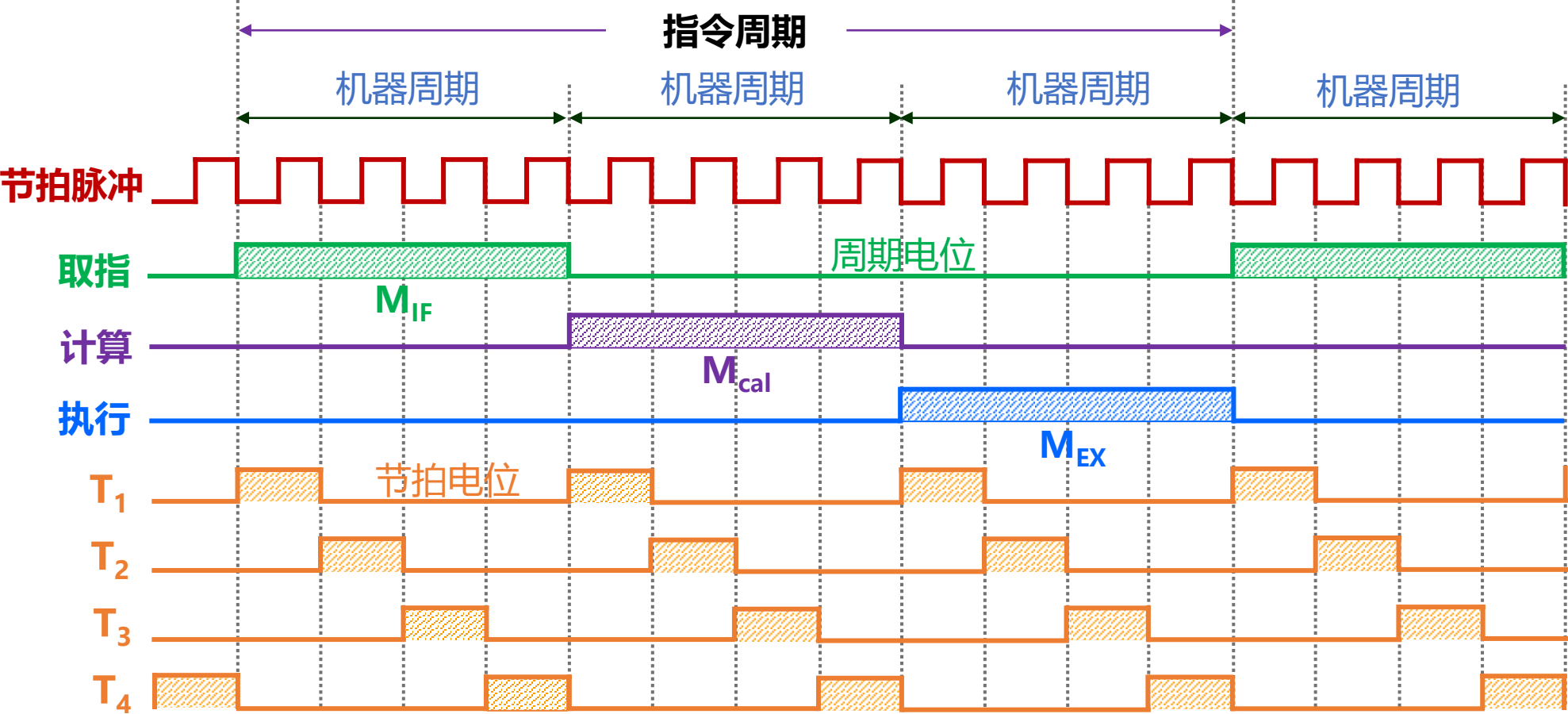
- 无时钟，应答机制

■ 联合控制

- 同步、异步相结合

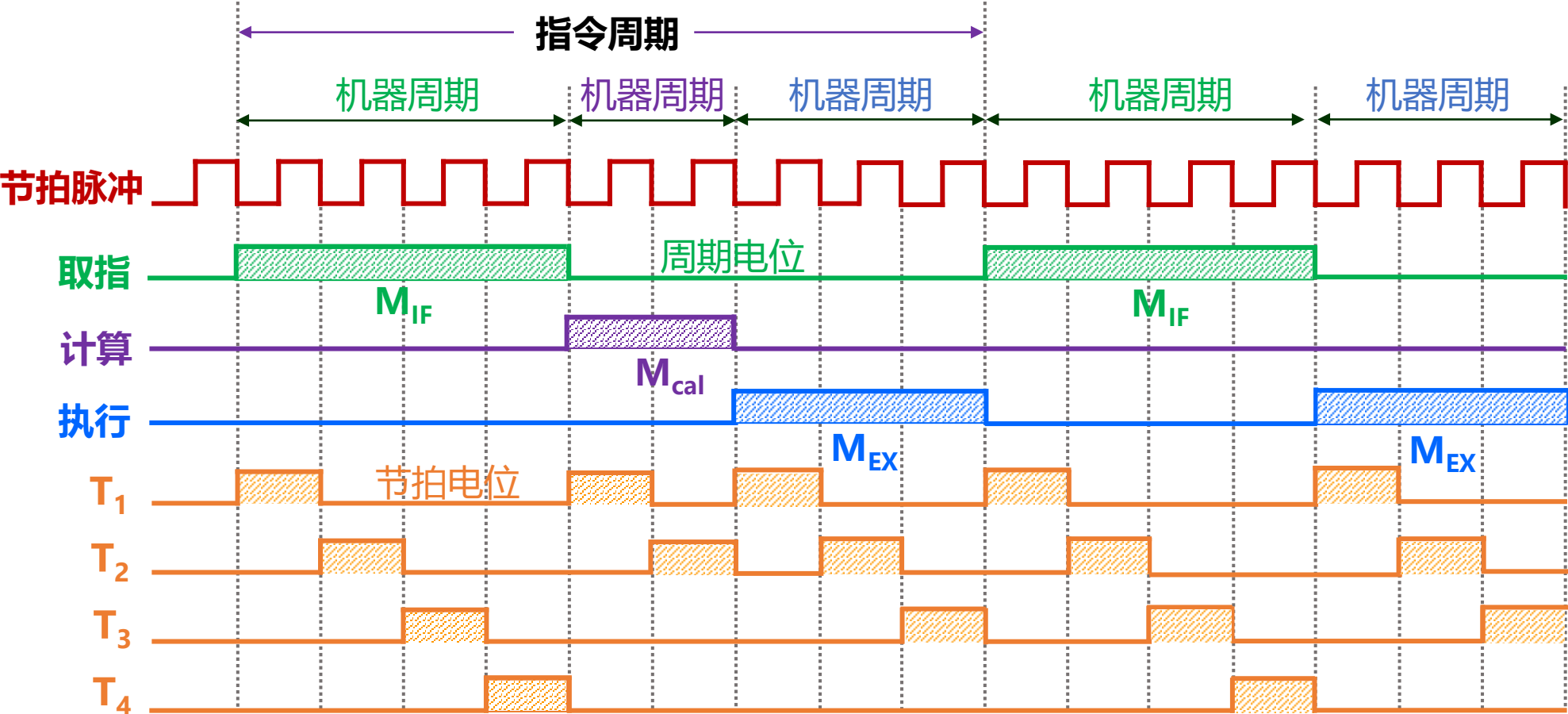


定长指令周期三级时序（同步控制）



机器周期数、节拍数固定，设计简单，效率较低

变长指令周期三级时序（同步控制）



机器周期数可变、节拍数可变，无周期浪费，更加灵活

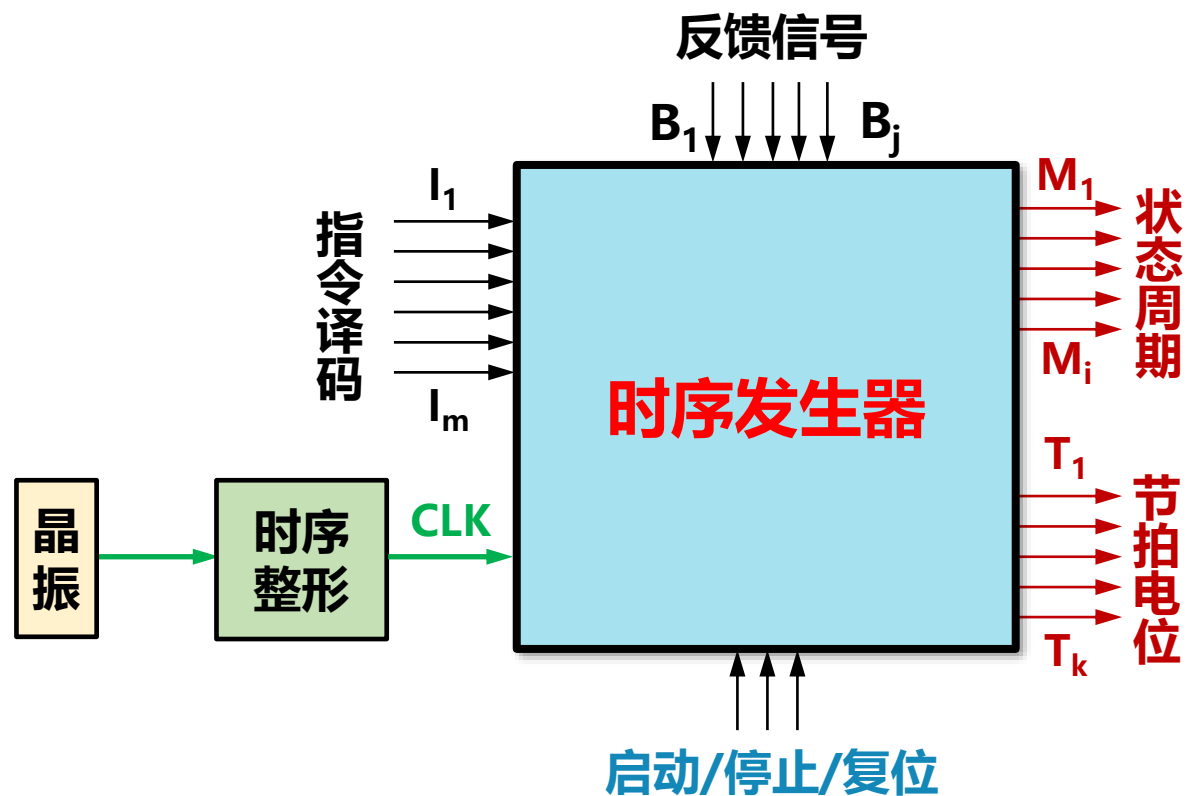
6.4 时序与控制

■ 6.4.1 中央处理器时序

■ 6.4.2 控制方式

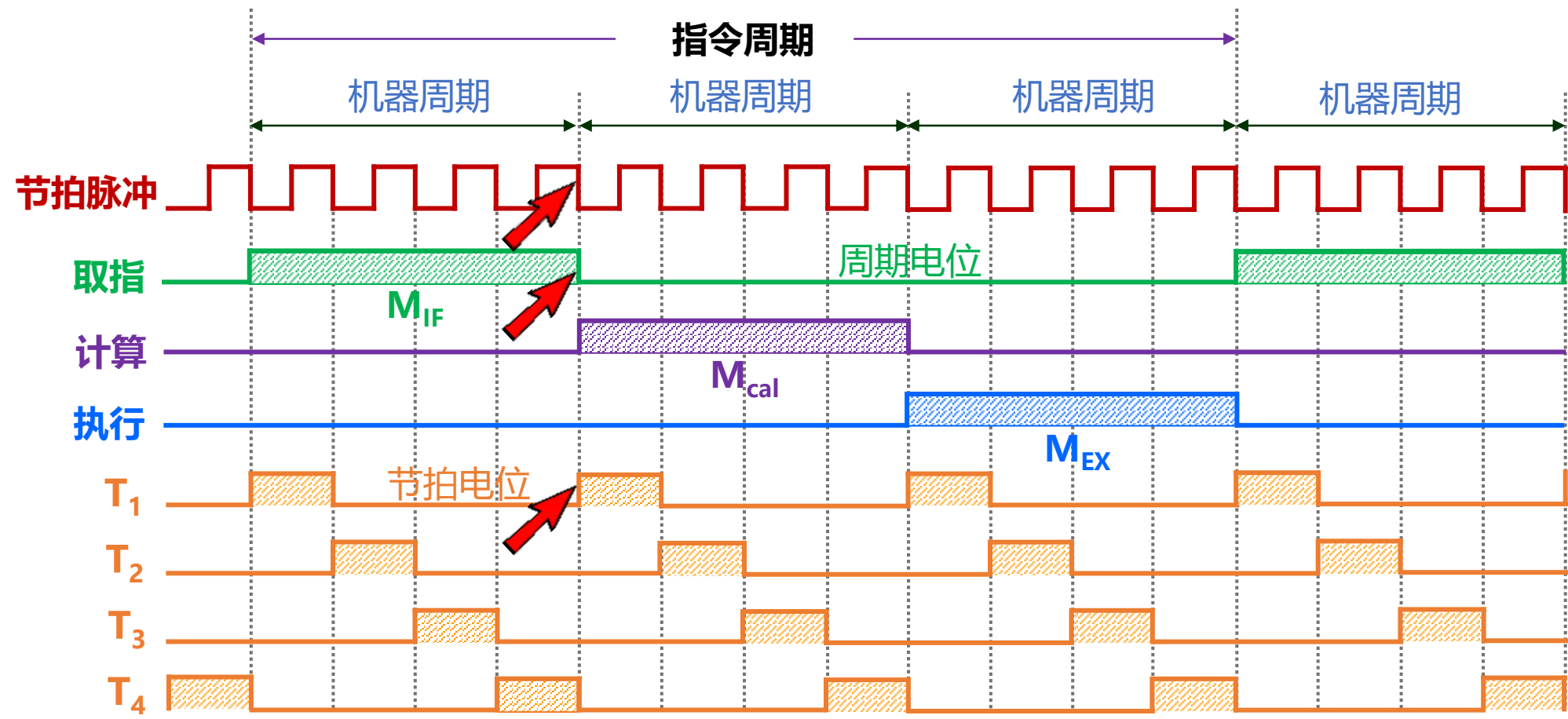
■ 6.4.3 时序发生器

时序发生器



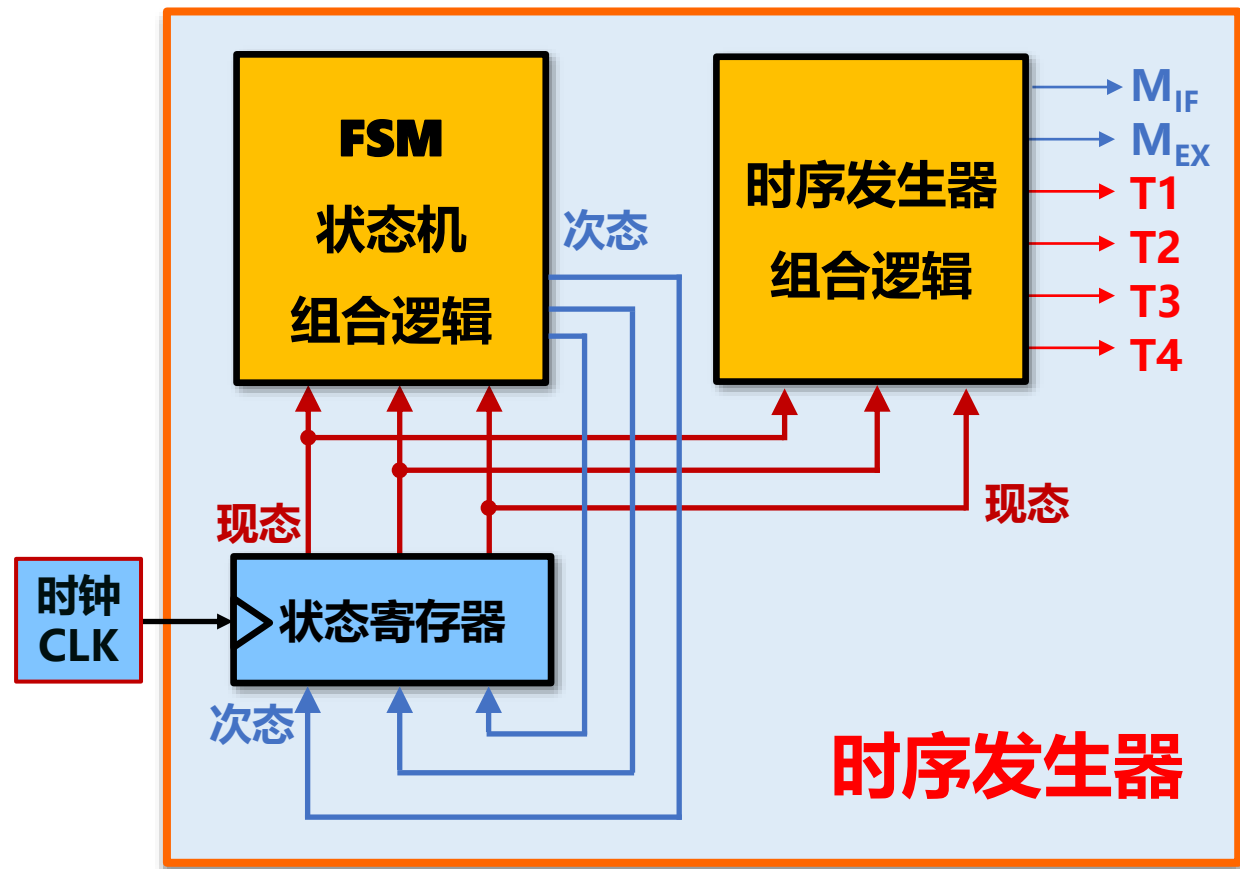
- 根据**时钟脉冲信号**持续不断地产生**状态周期电位**和**节拍电位**，操作控制器利用这些周期、节拍电位信号对操作控制信号进行时序的调制，生成控制信号序列
- 将状态周期电位信号、节拍电位信号与指令译码信号、反馈信号等进行适当的逻辑与操作，从而限定操作控制信号的开始时间和持续时间

定长指令周期的三级时序发生器

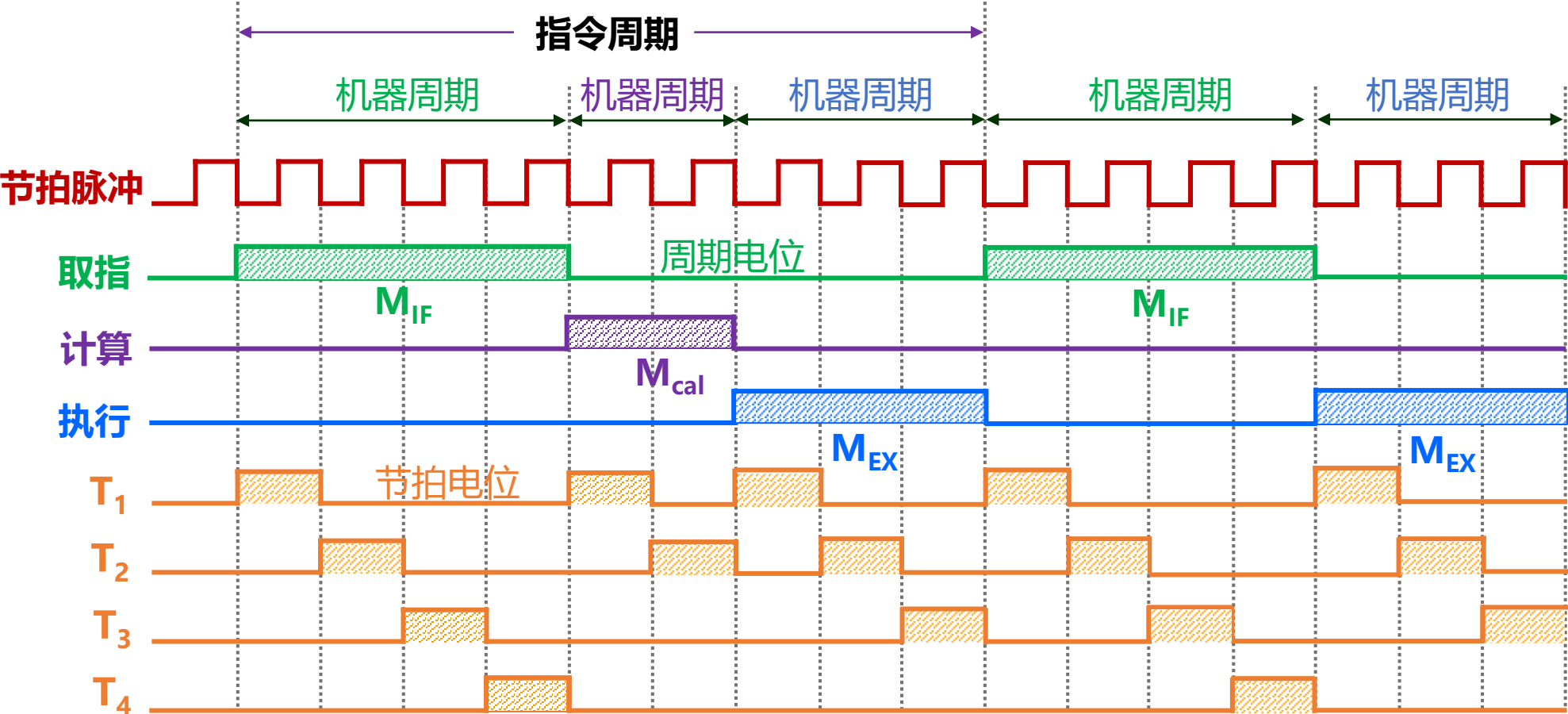


在定长指令周期的同步控制方式中，状态周期、节拍电位信号仅仅与时钟脉冲有关

定长指令周期时序发生器设计

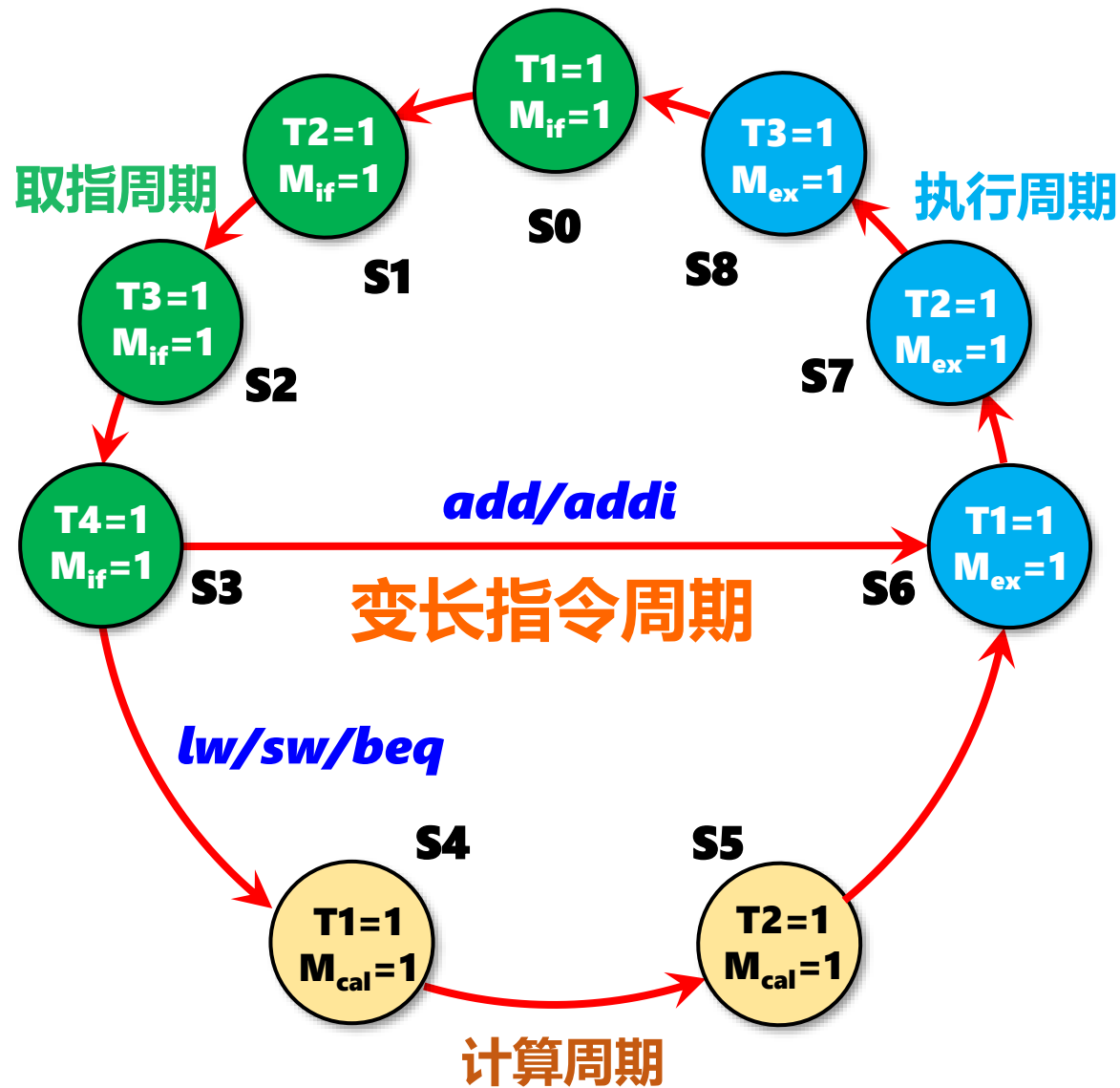
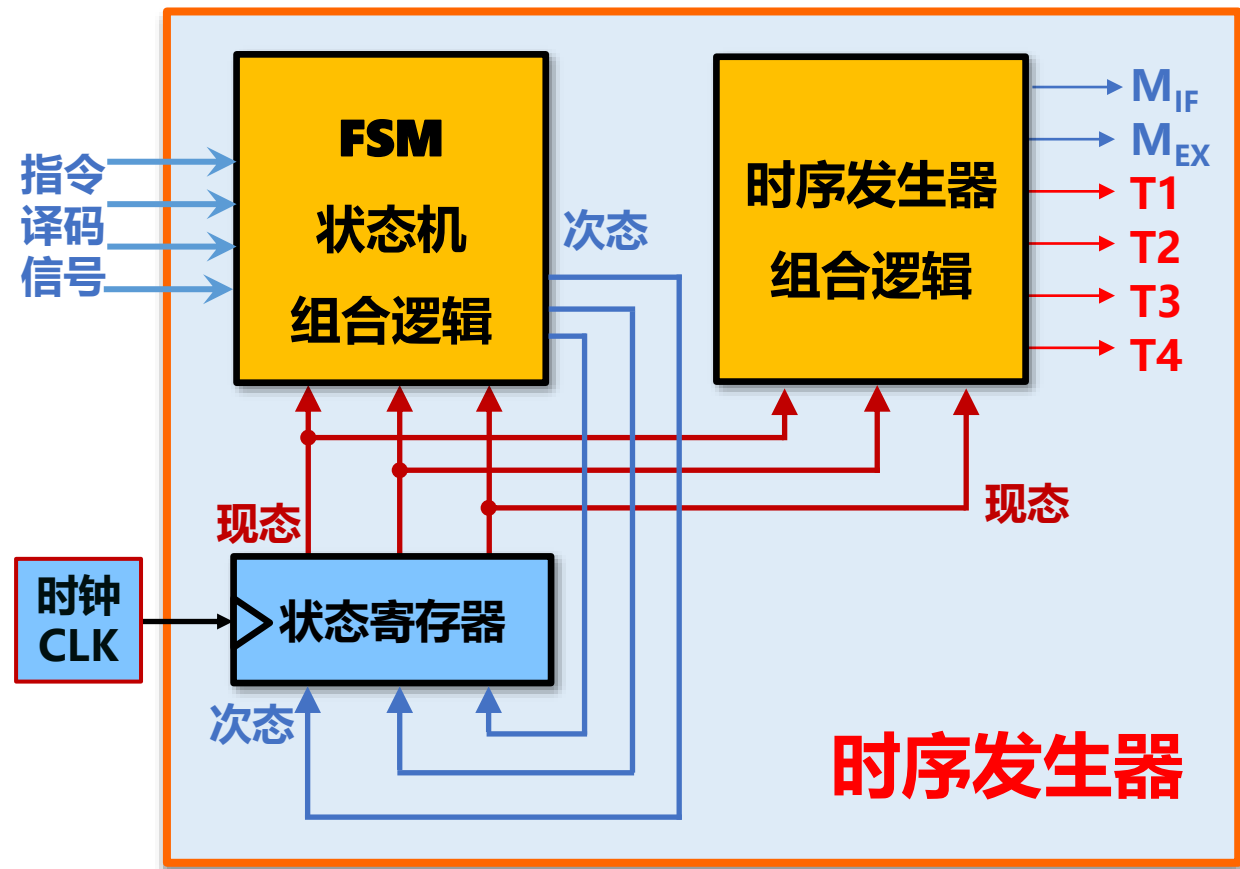


变长指令周期三级时序时序发生器



机器周期数可变、节拍数可变，无周期浪费，更加灵活

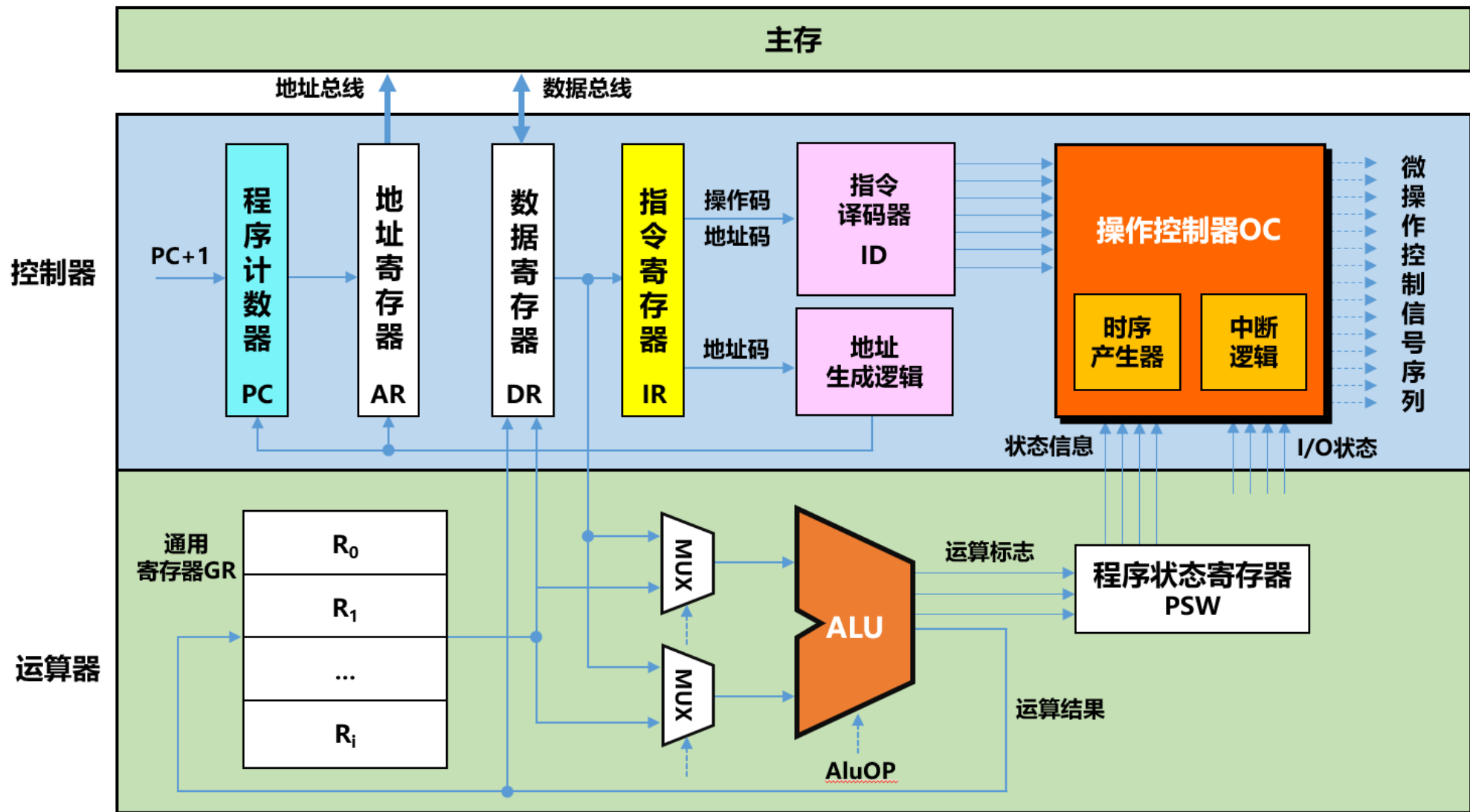
变长指令周期时序发生器设计



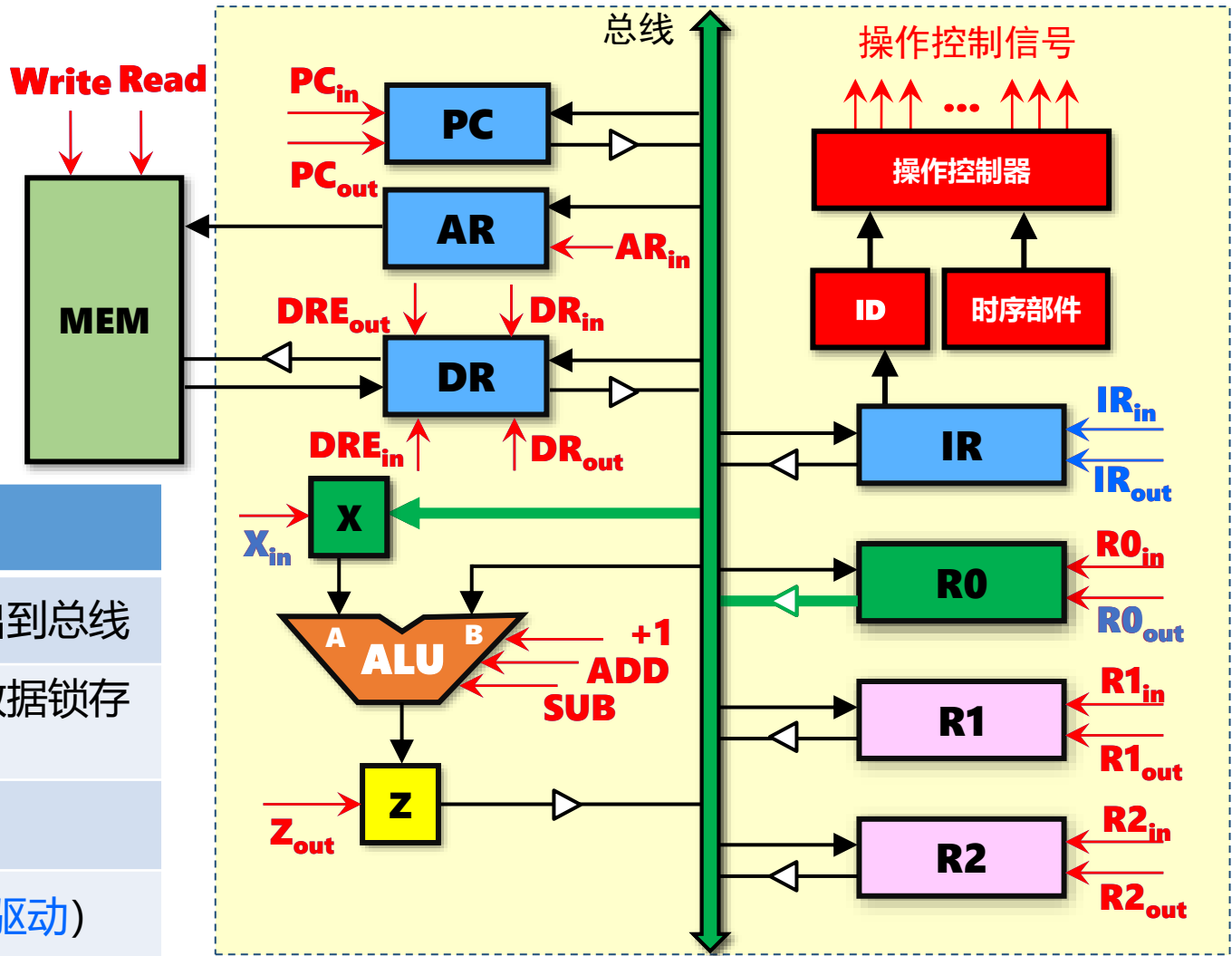
本章主要内容

- 6.1 中央处理器概述
- 6.2 指令周期
- 6.3 数据通路及指令操作流程
- 6.4 时序与控制
- **6.5 硬布线控制器**
- 6.6 微程序控制器
- 6.7 异常与中断处理





控制信号	作用说明
IR_{out} 、 PC_{out} 、 $\dots R1_{out}$	控制三态门将寄存器值输出到总线
IR_{in} 、 PC_{in} 、 $\dots R1_{in}$	控制寄存器使能端将总线数据锁存 (时钟驱动)
$+1$ 、 ADD 、 SUB	运算控制信号
$Write$ 、 $Read$	内存读写控制信号 (时钟驱动)



—————→ : 数据流
 —————→ : 控制流

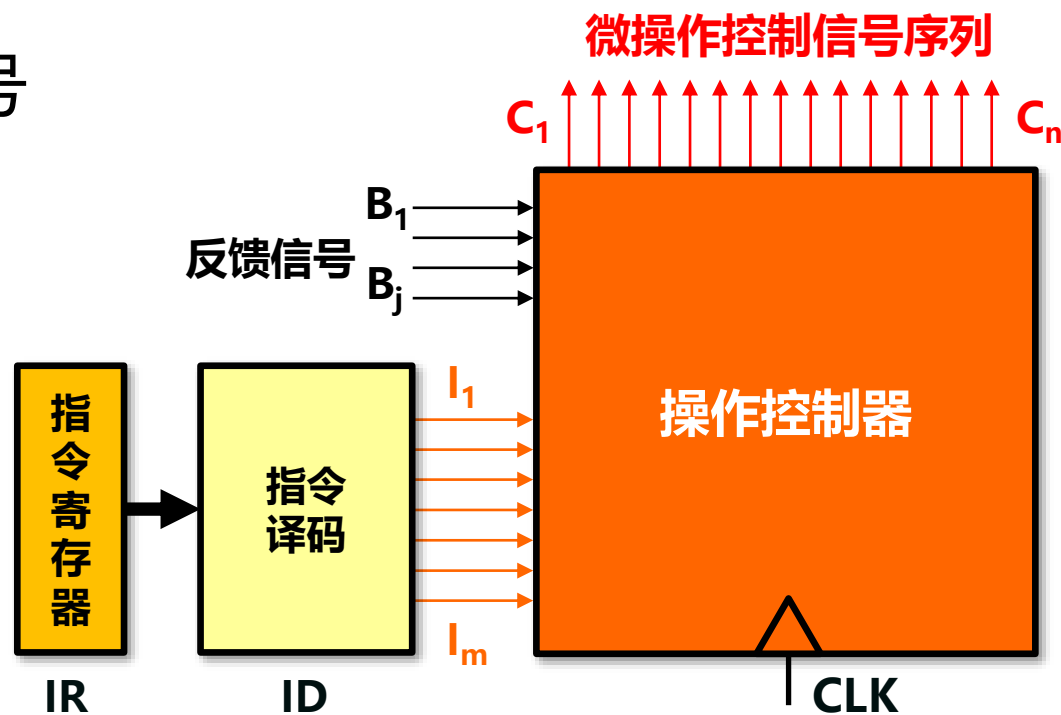
- 硬布线控制器 (时序逻辑型) (硬件实现)
- 微程序控制器 (存储程序型) (软件实现)

硬布线控制器

又称为组合逻辑控制器，这种控制器的电路直接由各种类型的逻辑门电路和触发器等构成

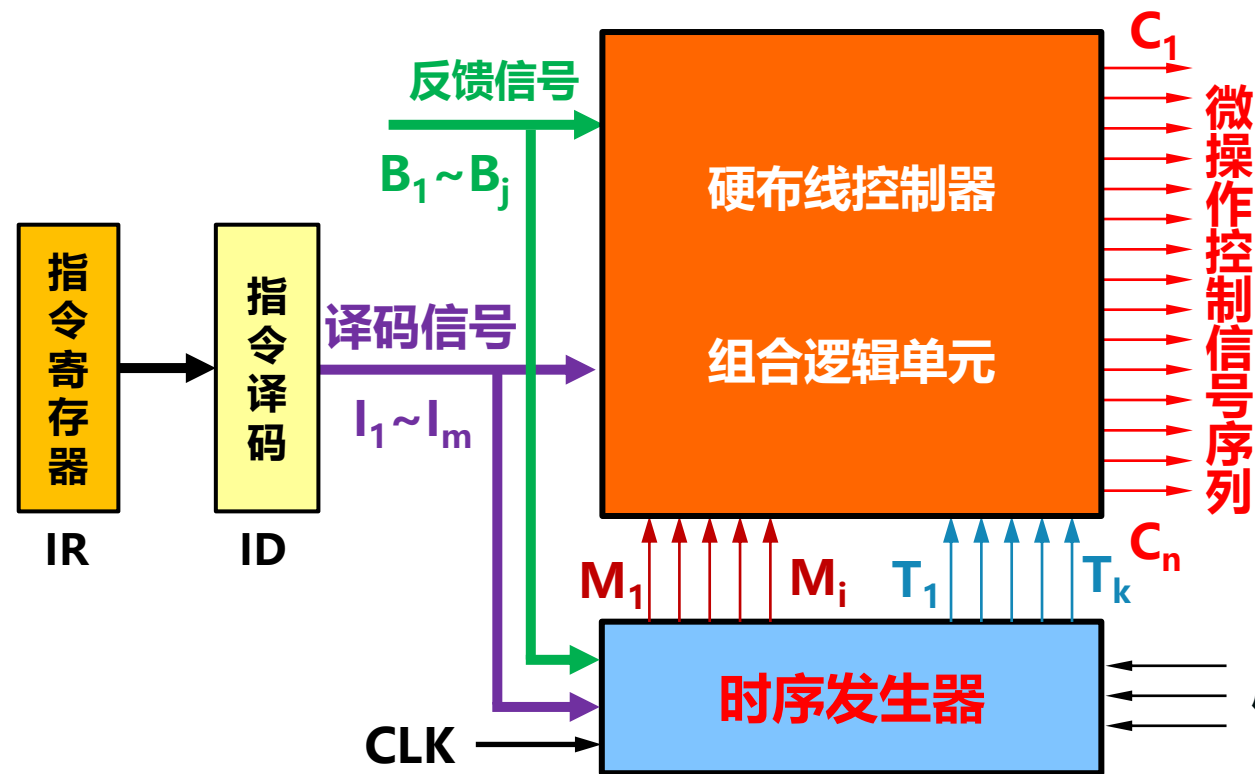
基本原理

- **控制器**：产生控制信号序列的逻辑电路
- **输入信号**：指令译码，时钟，反馈信号
- **输出信号**：功能部件控制信号序列
- **设计目标**：最少元件，最快速度
- **理论基础**：布尔代数
- **组成器件**：门电路，触发器



机器指令字 → 控制器信号序列

三级时序硬布线控制器

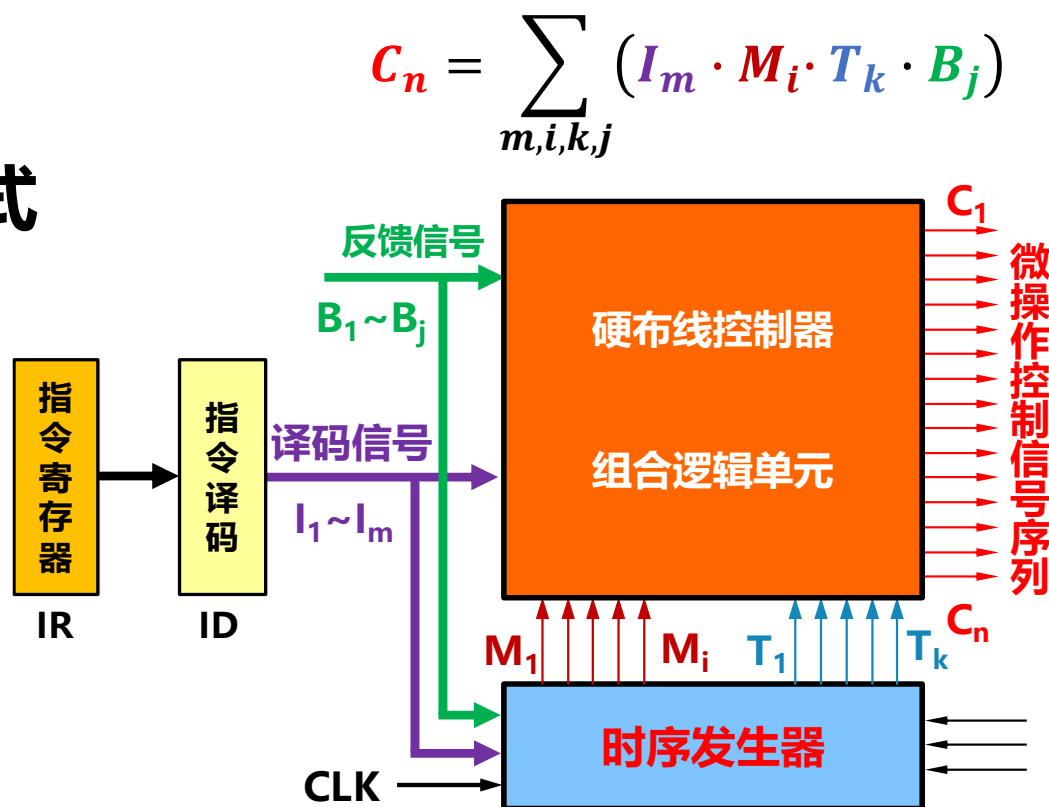


$$C_n = \sum_{m,i,k,j} (I_m \cdot M_i \cdot T_k \cdot B_j)$$

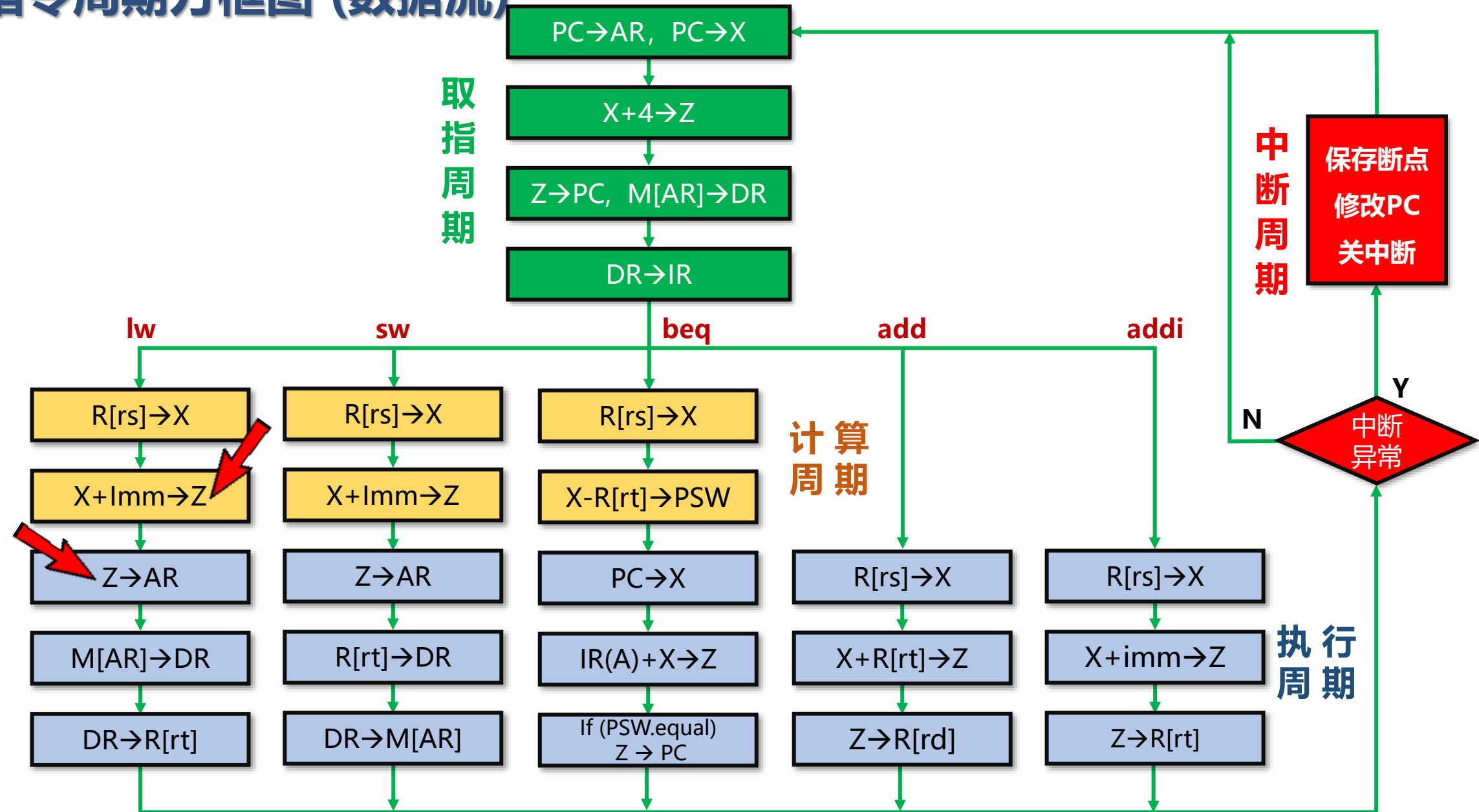
时序产生器循环产生周期电位、节拍电位，供控制器对信号进行时间调制

三级时序硬布线控制器设计步骤

1. 分析数据通路，画指令周期流程图，明确各节拍控制信号
2. 设计时序发生器：根据机器周期、节拍划分构建状态图
3. 找出同一微操作控制信号产生条件
4. 写出各微操作控制信号的逻辑表达式
5. 利用组合逻辑电路实现



指令周期方框图 (数据流)



单总线结构CPU指令周期（控制流）

取指令周期

节拍	控制信号(4 cycles)
T1	PC _{out} , AR _{in} , X _{in}
T2	+4
T3	Z _{out} , PC _{in} , DRE _{in} , Read
T4	DR _{out} , IR _{in}

$$C_n = \sum_{m,i,k,j} (I_m \cdot M_i \cdot T_k \cdot B_j)$$

■ Read = M_{IF} · T3 + L_W · M_{EX} · T2

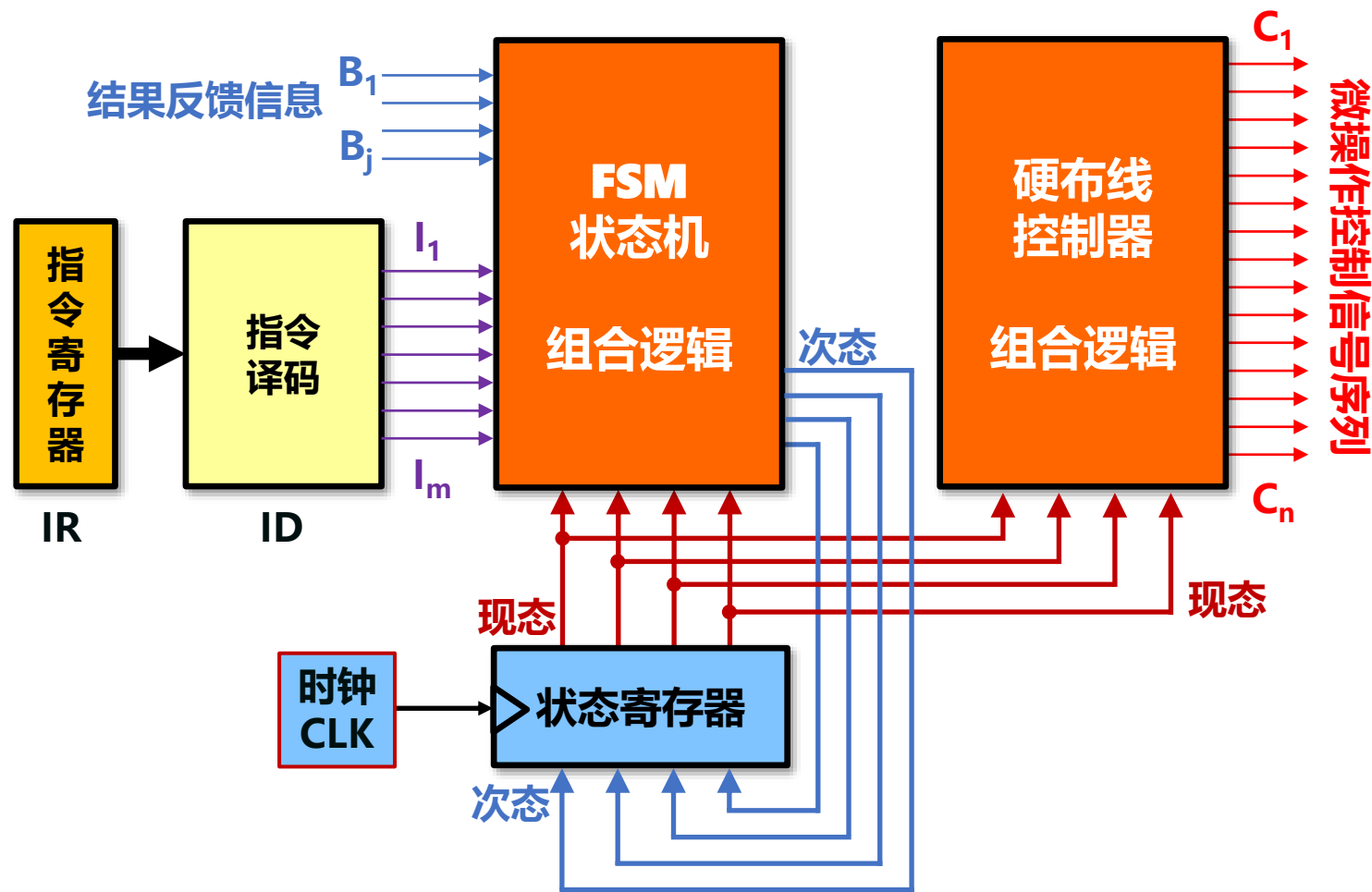
■ IR(I)_{out} = (L_W+S_W) · M_{cal} · T2 + addi · M_{EX} · T2

计算周期

节拍	lw (5 cycles)	sw (5 cycles)	beq (5 cycles)	add (3 cycles)	addi (3 cycles)
T1	R _{out} , X _{in}	R _{out} , X _{in}	R _{out} , X _{in}		
T2	IR(I) _{out} , ADD	IR(I) _{out} , ADD	R _{out} , Rs/Rt, SUB, PSW _{in}		
T1	Z _{out} , AR _{in}	Z _{out} , AR _{in}	PC _{out} , X _{in}	R _{out} , X _{in}	R _{out} , X _{in}
T2	DRE _{in} , Read	R _{out} , Rs/Rt, DR _{in}	IR(A) _{out} , ADD	Rs/Rt, R _{out} , ADD	IR(I) _{out} , ADD
T3	DR _{out} , R _{in}	DRE _{out} , Write	Z _{out} , C _{in} =PSW.equal	Z _{out} , R _{in} , RegDst	Z _{out} , R _{in}

执行周期

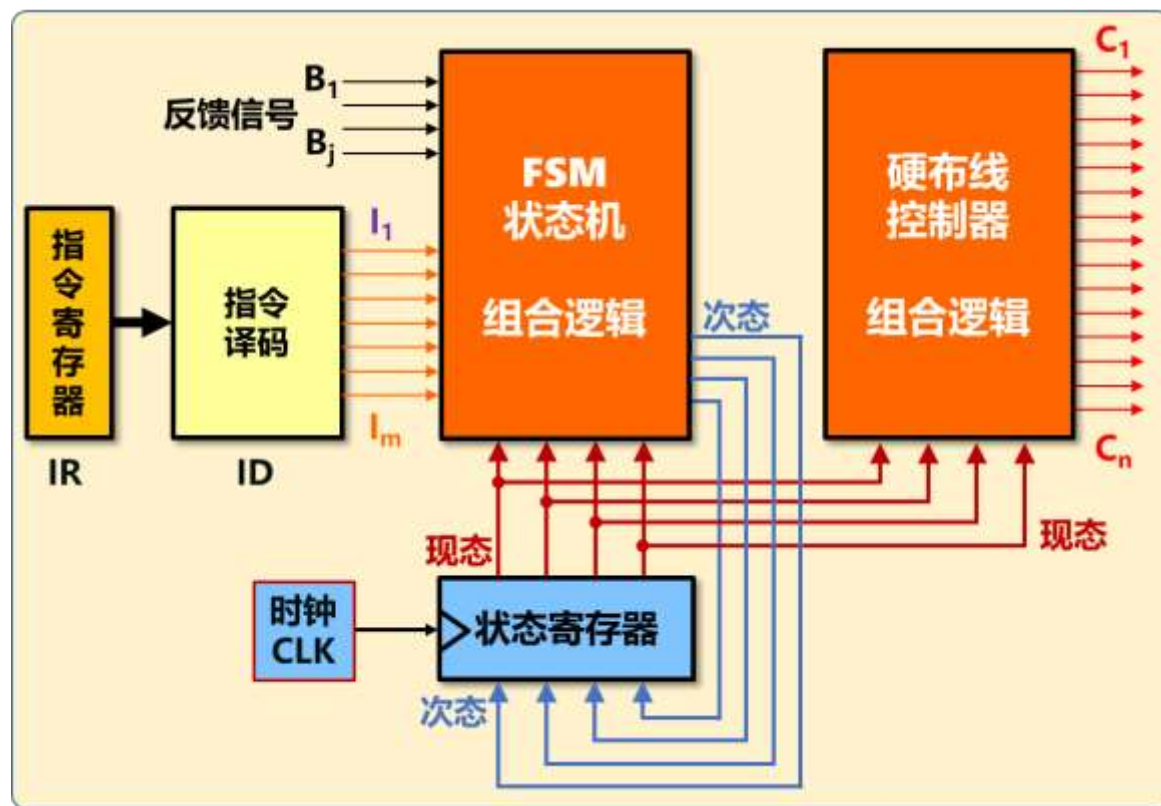
现代时序硬布线控制器



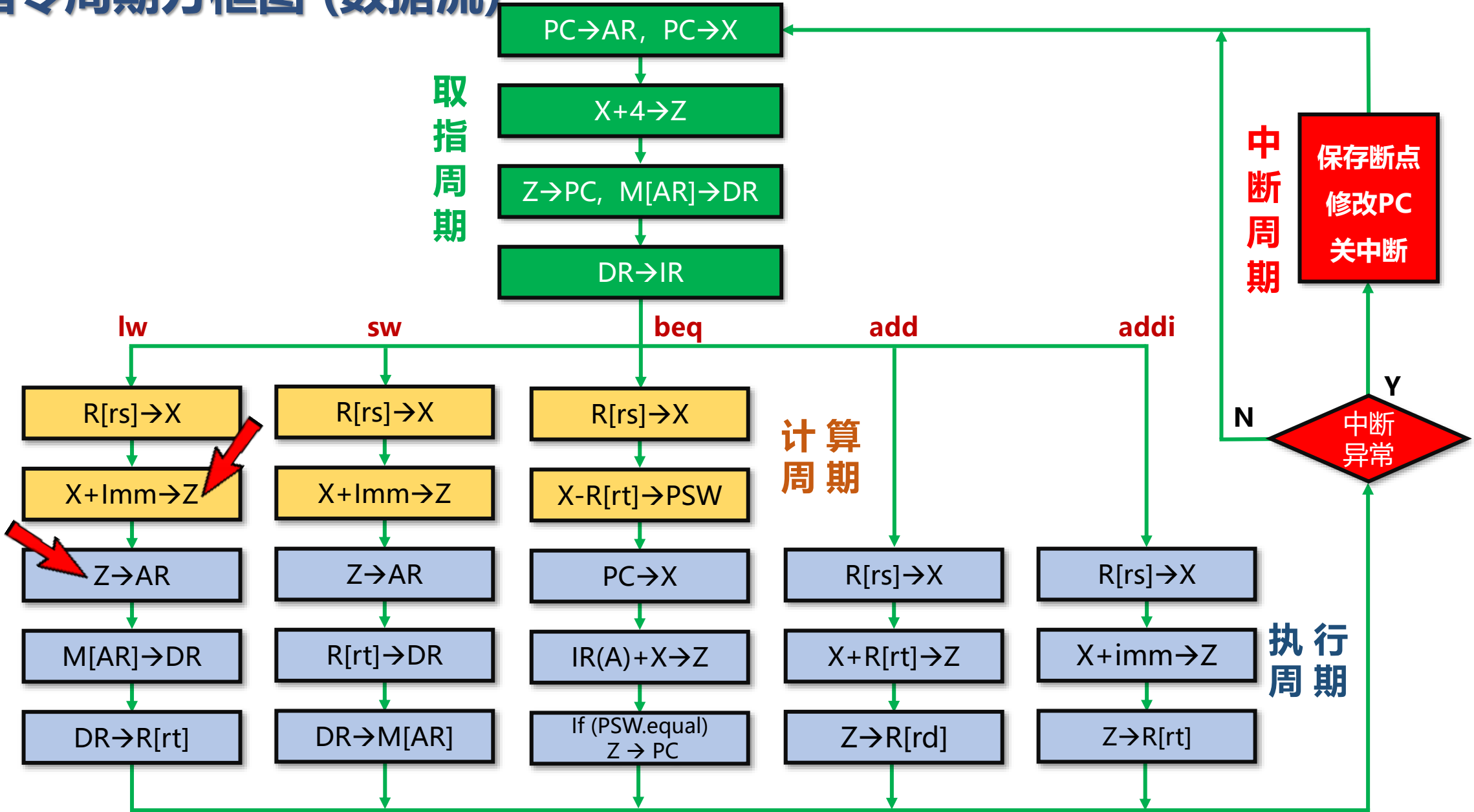
- 无时序发生器、操作控制信号仅仅与状态寄存器现态有关
- 有限状态机(FSM)来描述指令的执行过程，将不同指令执行的每个时钟周期均对应一个状态，每一个状态会对应特定的微操作控制信号

现代时序硬布线控制器设计步骤

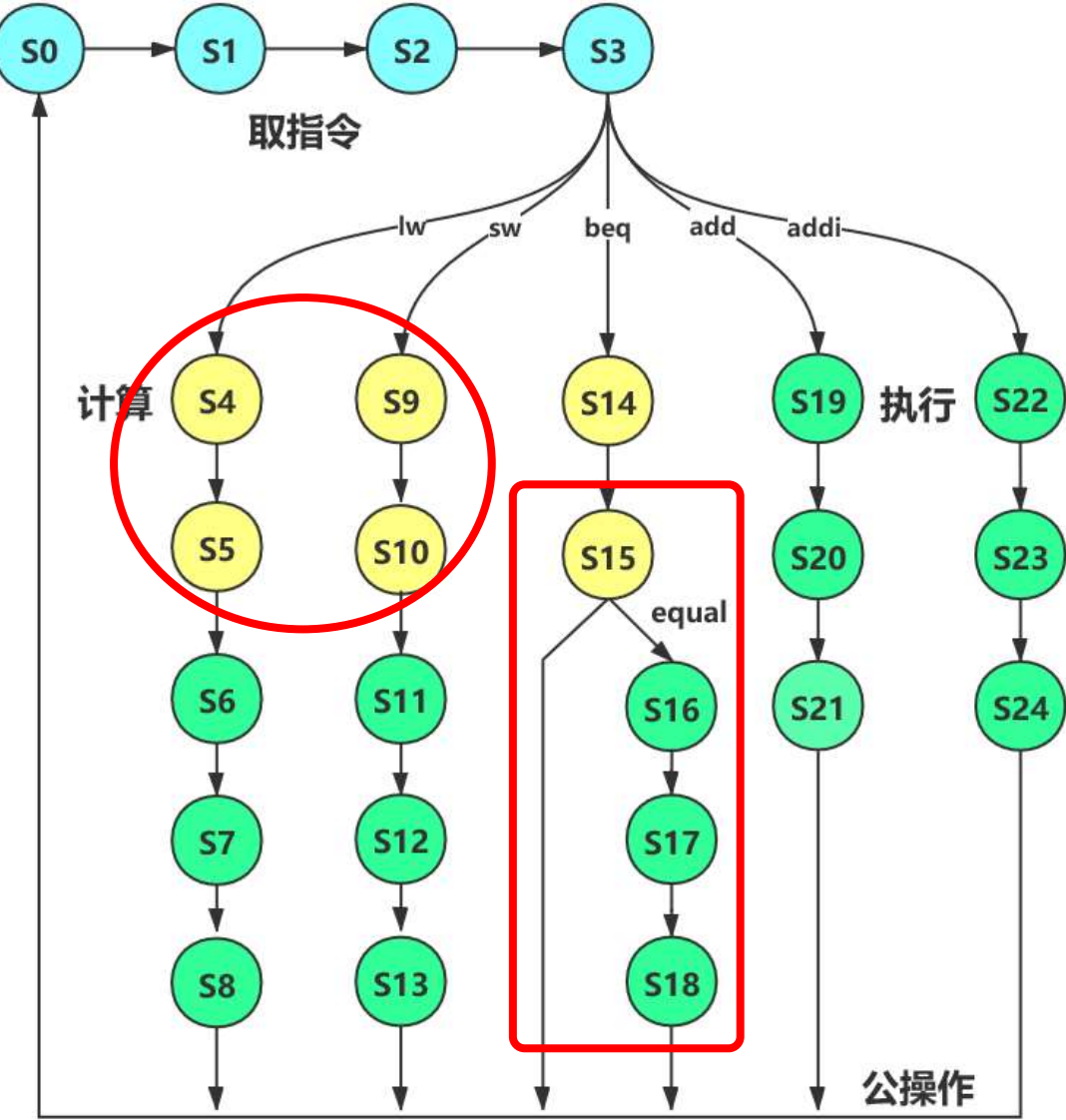
1. 分析数据通路，画指令周期流程图，明确各节拍控制信号
2. 绘制指令执行状态转换图
3. 构建状态转换表
4. 实现有限状态机组合逻辑
5. 实现控制器组合逻辑电路



指令周期方框图 (数据流)



指令执行状态转换图 → 状态转换表

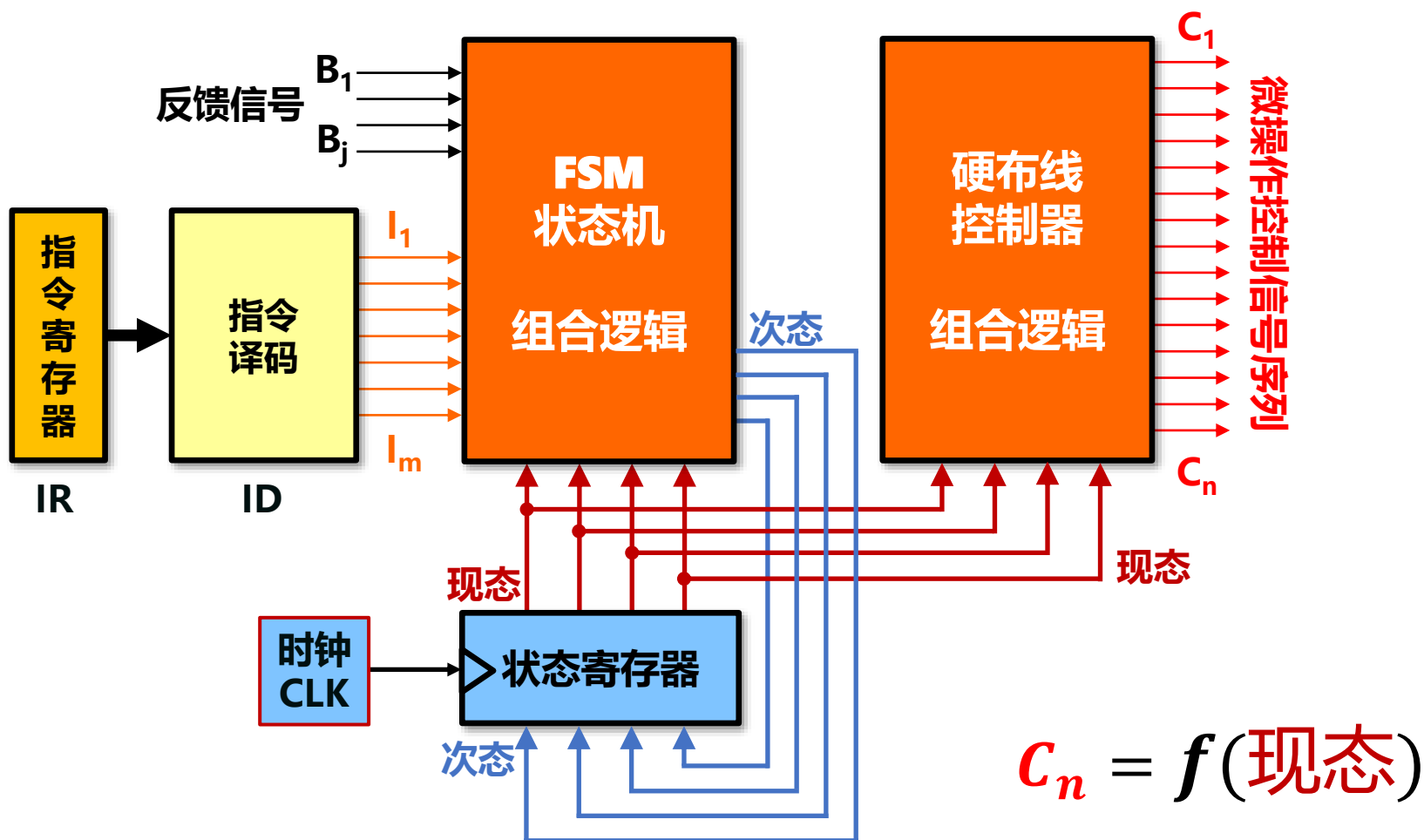


现态	lw	sw	beq	add	addi	equal	次态
S0	x	x	x	x	x		S1
S1	x	x	x	x	x		S2
S2	x	x	x	x	x		S3
S3	1						S4
S3		1					S9
S3			1				S14
S3				1			S19
S3					1		S22

有限状态机FSM 状态转换表

取指令周期状态		现态	lw	sw	beq	add	addi	Equal	次态
		S0	x	x	x	x	x	x	S1
		S1	x	x	x	x	x	x	S2
		S2	x	x	x	x	x	x	S3
		S3	1						S4
		S3		1					S9
		S3			1				S14
		S3				1			S19
		S3					1		S22
lw		S4	x	x	x	x	x	x	S5
		S5	x	x	x	x	x	x	S6
		S6	x	x	x	x	x	x	S7
		S7	x	x	x	x	x	x	S8
		S8	x	x	x	x	x	x	S0
beq		...							...
		S15						1	S16
		S15						0	S0
		S16							S17

现代时序系统硬布线控制器



操作控制信号仅与状态寄存器现态有关

单总线结构CPU指令周期（控制流）

取指令周期

节拍	控制信号(4 cycles)	
T1	S₀	PC _{out} , AR _{in} , X _{in}
T2	S₁	+4
T3	S₂	Z _{out} , PC _{in} , DRE _{in} , Read
T4	S₃	DR _{out} , IR _{in}

■ 传统三级时序

◆ 机器周期同步，慢、设计简单

■ 现代时序

◆ 时钟周期同步，快，设计复杂

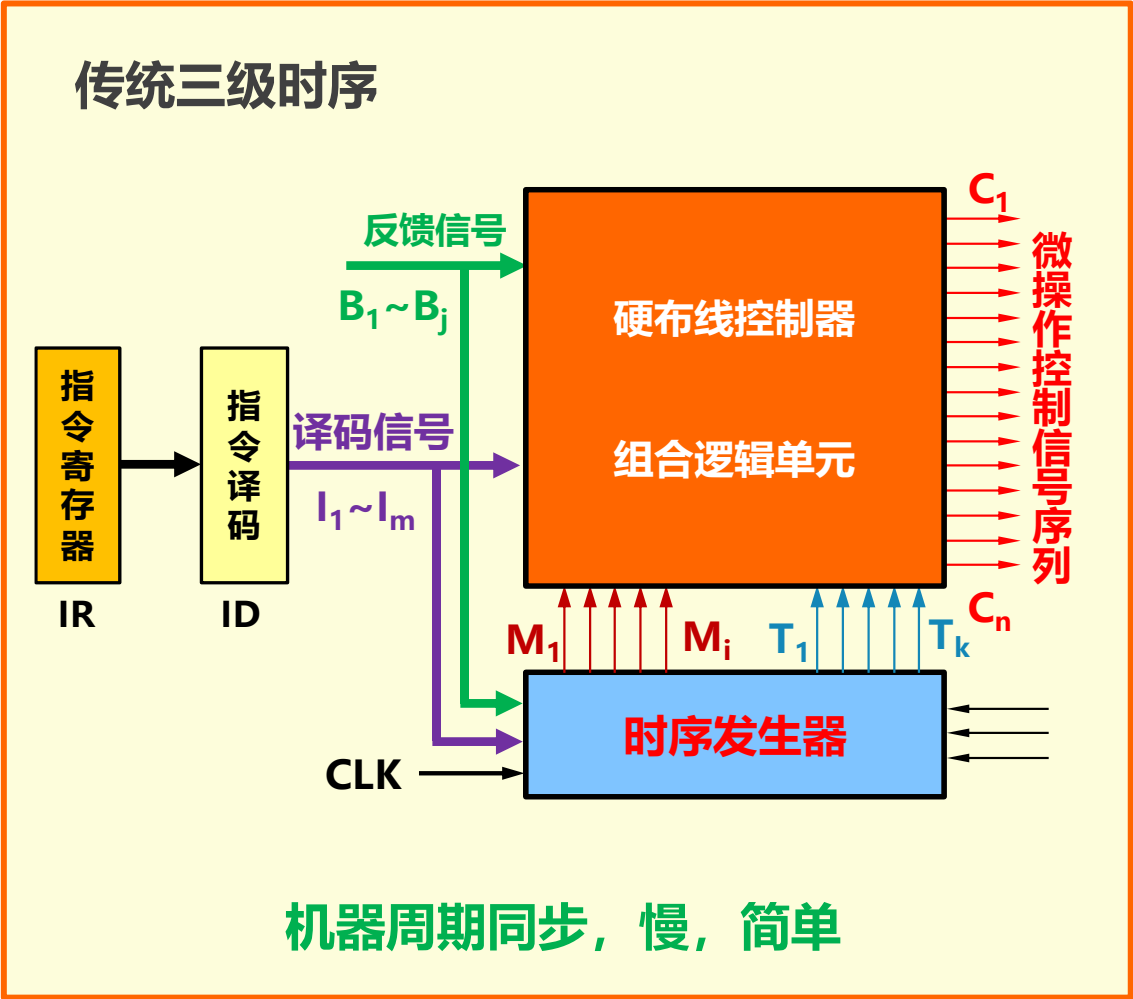
计算周期

执行周期

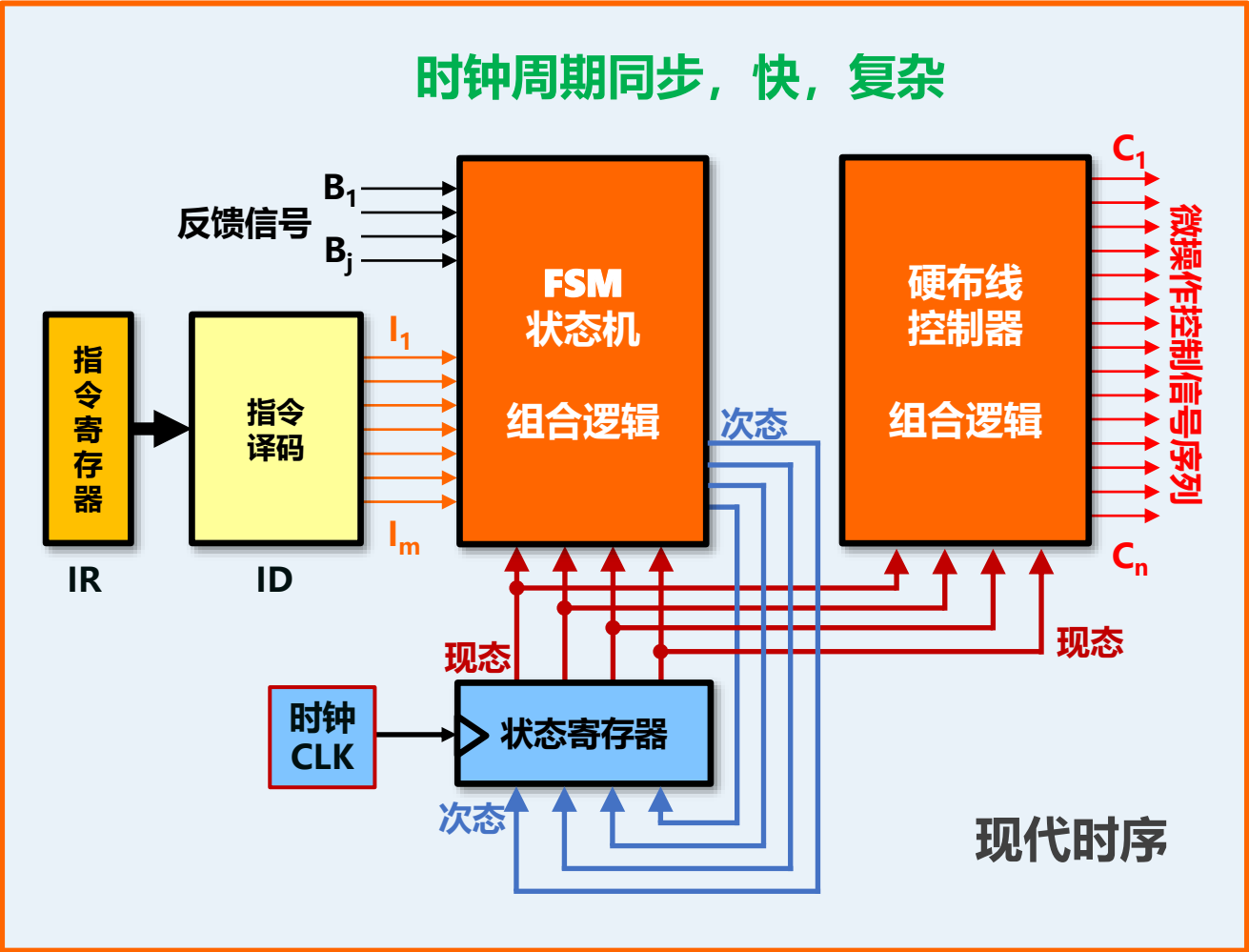
节拍	lw (5 cycles)		sw (5 cycles)		beq (5 cycles)		add (3 cycles)		addi (3 cycles)	
T5	S₄	R _{out} , X _{in}	S₉	R _{out} , X _{in}	S₁₄	R _{out} , X _{in}				
T6	S₅	IR(I) _{out} , ADD	S₁₀	IR(I) _{out} , ADD	S₁₅	R _{out} , Rs/Rt, SUB, PSW _{in}	不同状态对应不同指令阶段			
T7	S₆	Z _{out} , AR _{in}	S₁₁	Z _{out} , AR _{in}	S₁₆	PC _{out} , X _{in}				
T8	S₇	DRE _{in} , Read	S₁₂	R _{out} , Rs/Rt, DR _{in}	S₁₇	IR(A) _{out} , ADD	S₁₉	R _{out} , X _{in}	S₂₂	R _{out} , X _{in}
T9	S₈	DR _{out} , R _{in}	S₁₃	DRE _{out} , Write	S₁₈	Z _{out} , C _{in} =PSW.equal	S₂₀	Rs/Rt, R _{out} , ADD	S₂₃	IR(I) _{out} , ADD
							S₂₁	Z _{out} , R _{in} , RegDst	S₂₄	Z _{out} , R _{in} , RegDst

传统三级时序与现代时序对比

传统三级时序

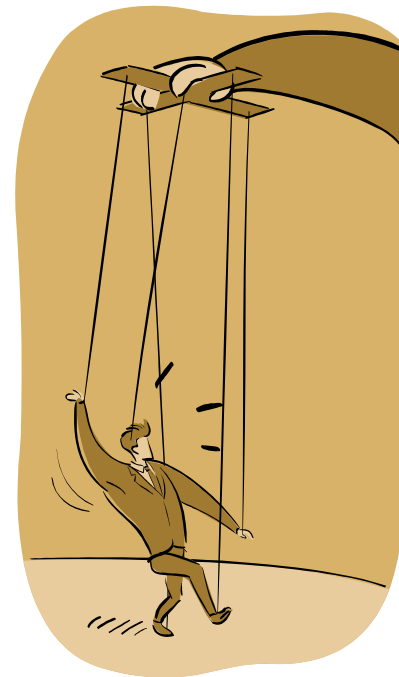


时钟周期同步, 快, 复杂



硬布线控制器特点

- 结构复杂，无规则；
- 设计和调试困难；
- 不可改变指令系统和指令功能
- 适用于VLSI
- 速度快



本章主要内容

- 6.1 中央处理器概述
- 6.2 指令周期
- 6.3 数据通路及指令操作流程
- 6.4 时序与控制
- 6.5 硬布线控制器
- 6.6 微程序控制器
- 6.7 异常与中断处理



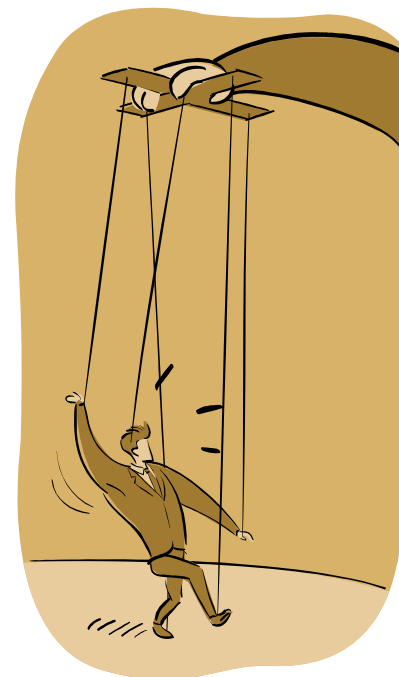
微程序控制器基本思想

■ 硬布线：同步逻辑、繁，快，贵，难改

- 一条指令多个时钟周期
- 一个时钟周期一个状态
- 一个状态对应一组并发信号

■ 微程序：存储逻辑、简、慢、廉，易改

- 控制信号流存储为微指令
- 一个状态对应一条微指令
- 一条指令对应多条微指令
- 状态字等同与存储器地址



微程序控制器工作原理

■ 微程序是利用**软件思想设计硬件**的技术

□ 控制信号序列像程序一样存储起来

- ◆ 控制信号序列分解为若干节拍
- ◆ 一个节拍的信号编成一条微指令
- ◆ 多个节拍对应多条微指令，形成一段微程序

□ 依序执行微程序即可生成控制信号序列

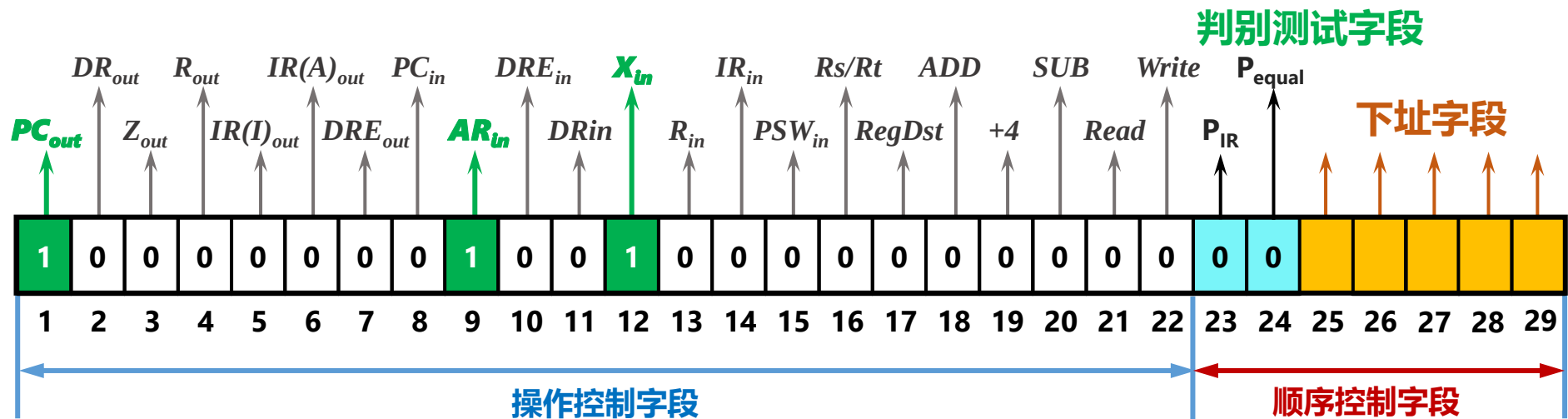
- ◆ 指令取指执行 → 微程序执行 → 微指令执行 → 生成控制信号

■ **存储技术和程序设计**相结合，回避复杂时序逻辑设计



微命令与微操作

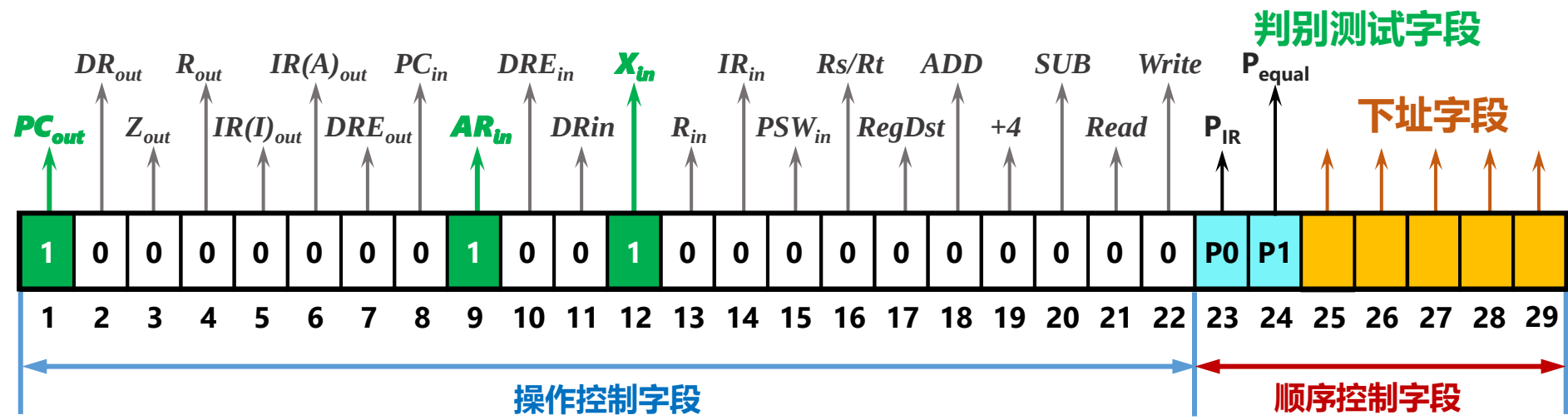
- **微命令**：控制部件向执行部件发出的各种控制命令
- **微操作**：执行部件收到微命令后所进行的操作
 - **相容性微操作**：同时或在同一个周期内并行执行的微操作，例如从总线向寄存器锁存数据的使能信号 PCin、ARin、IRin、Xin、Rin等
 - **互斥性微操作**：不能在同一个周期并行执行的微操作，例如Read、Write 就属于互斥性微操作



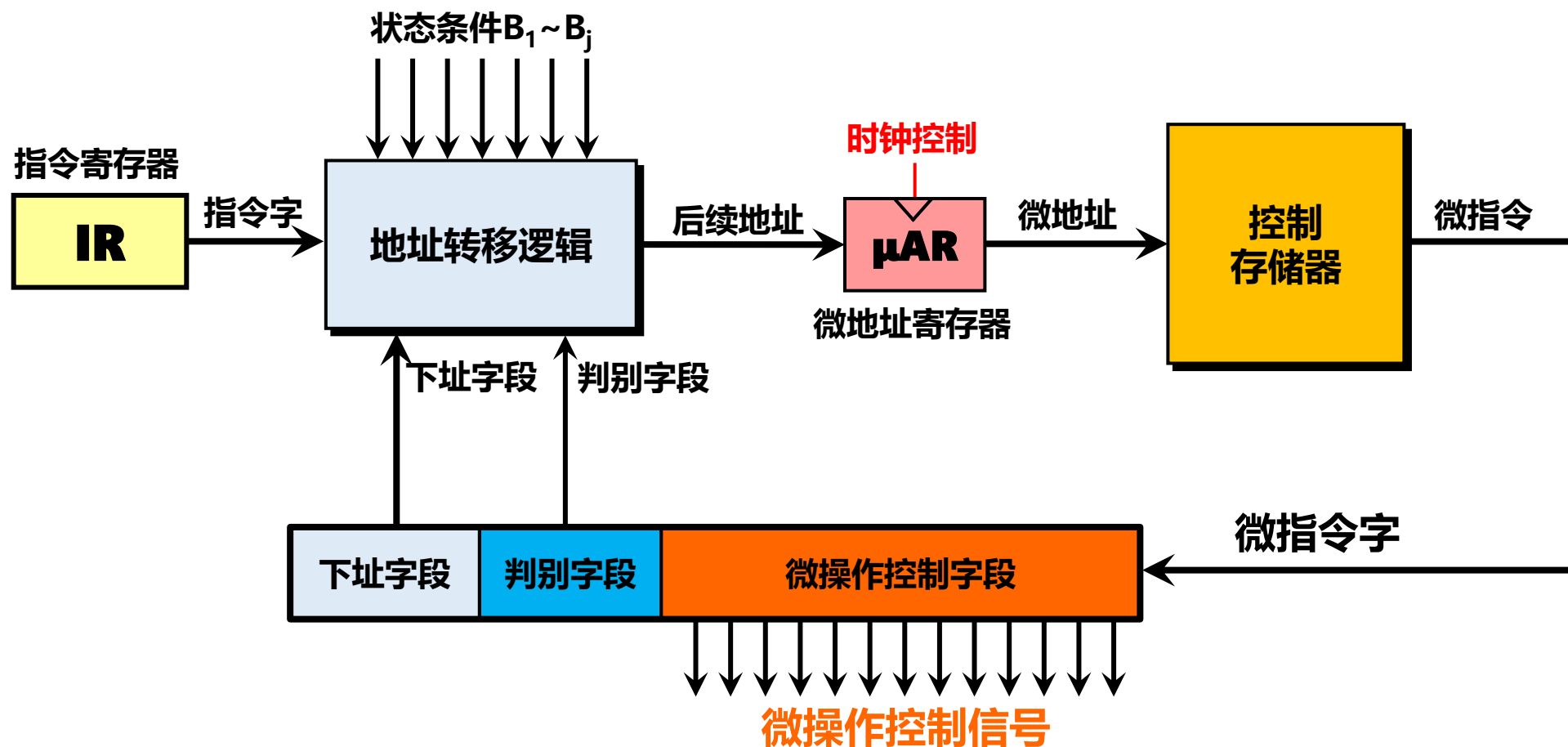
- 一条微指令对应一个时钟周期
- 微指令操作控制字段的信号在当前时钟周期内有效，为1有效
- 指令需要多少时钟周期就包括多少微指令

单总线CPU微指令构造

- 操作控制字段： 存储操作控制信号
 - ◆ 每一位对应一个控制信号，也称微命令，可同时给出多个操作信号
- 顺序控制字段： 用于控制微程序的执行顺序
 - ◆ 判别逻辑为零，下一条微指令地址从下址字段获取，否则按约定规则生成



微程序控制器组成原理框图



- **初始化: $\mu AR=0$, 0号地址为取指微程序入口**
- **顺序执行取指微程序, 最后一条微指令判别字段非零, 地址跳转**

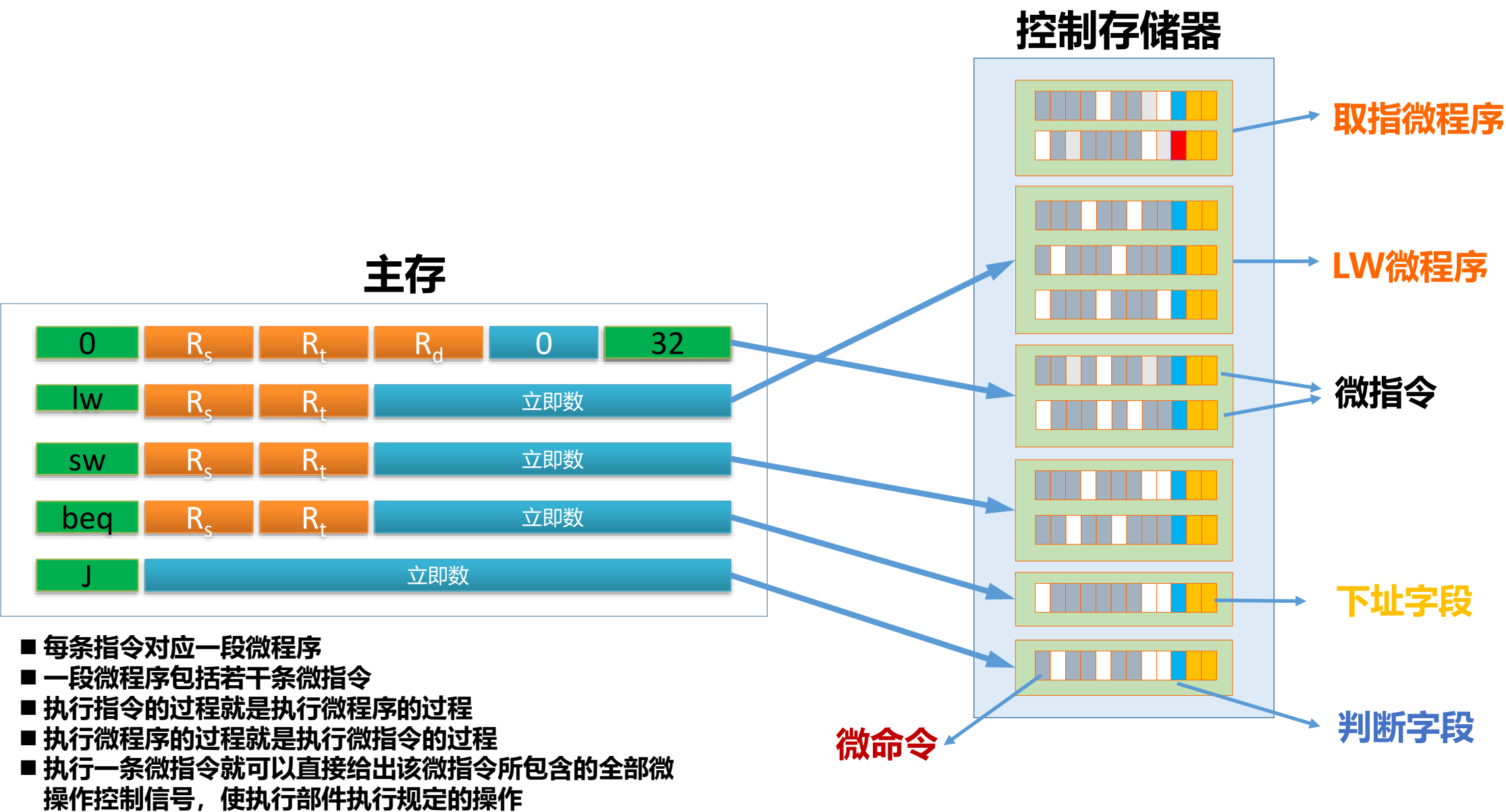
控制存储器

■ 用于存放微程序

■ 控制存储器与主存差异？

	存放内容	存储容量	存储字长	存储速度	介质
主存	数据与程序	大	机器字长	慢	RAM
控制存储器	微程序	小	微指令长度	快	ROM/RAM

程序、微程序、指令、微指令对应关系



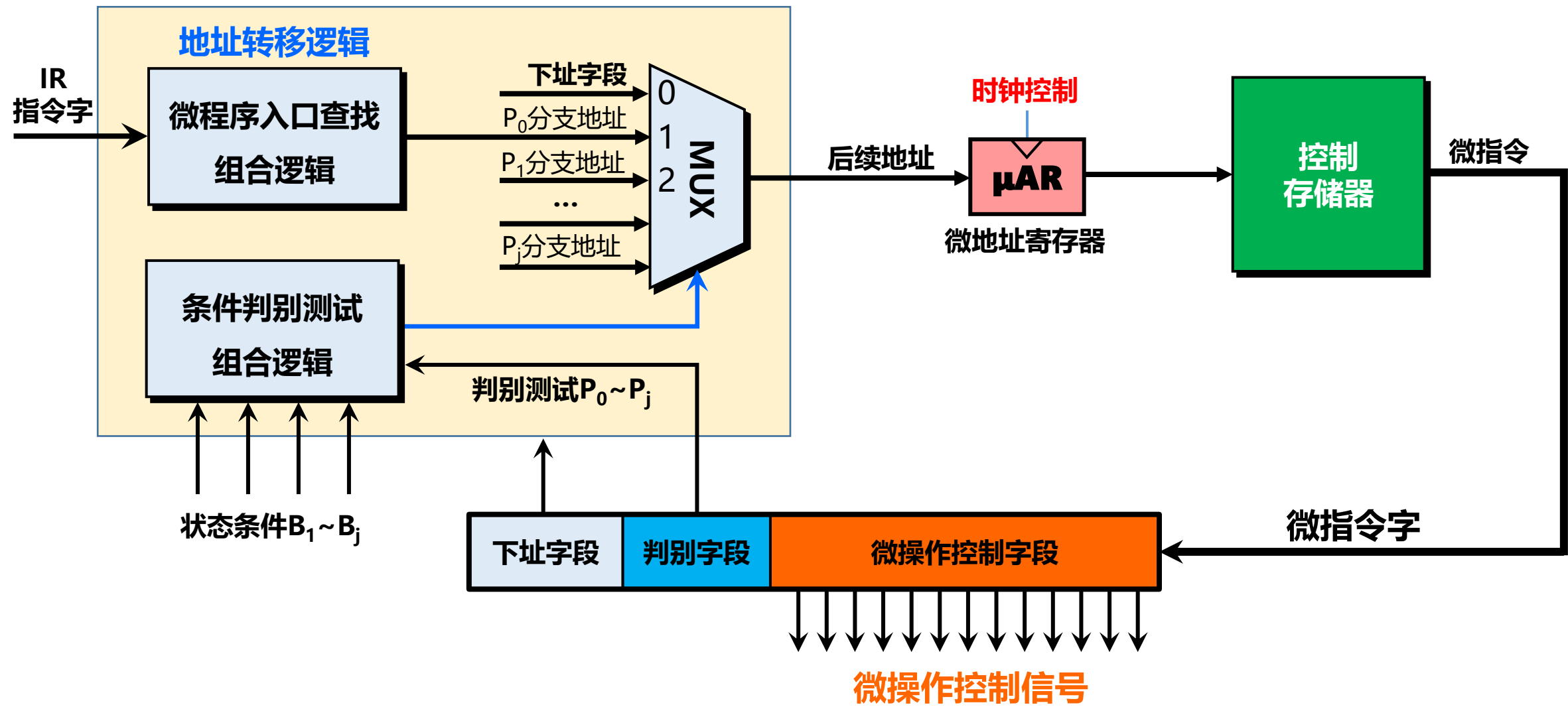
- 为控制存储器提供微指令地址，初始化时为0

初始化： $\mu AR=0$ ，0号地址为取指微程序入口

- μAR 输入来源为地址转移逻辑的输出
- 靠时钟控制其地址更新，每一次时钟控制端的触发都会重新锁存新的微地址，从而取出并执行下一条微指令

- 产生后续地址(下一条微指令的地址)
 - 下址字段法
 - 计数器法
- 地址转移逻辑根据以下因素共同决定微地址寄存器的输入
 - 指令字的译码情况、
 - 外部状态条件、
 - 微指令判别测试字段、
 - 下址字段
- 生成后续微指令的地址并送入 μAR ，时钟触发到来时 μAR 更新为后续地址的值。

地址转移逻辑

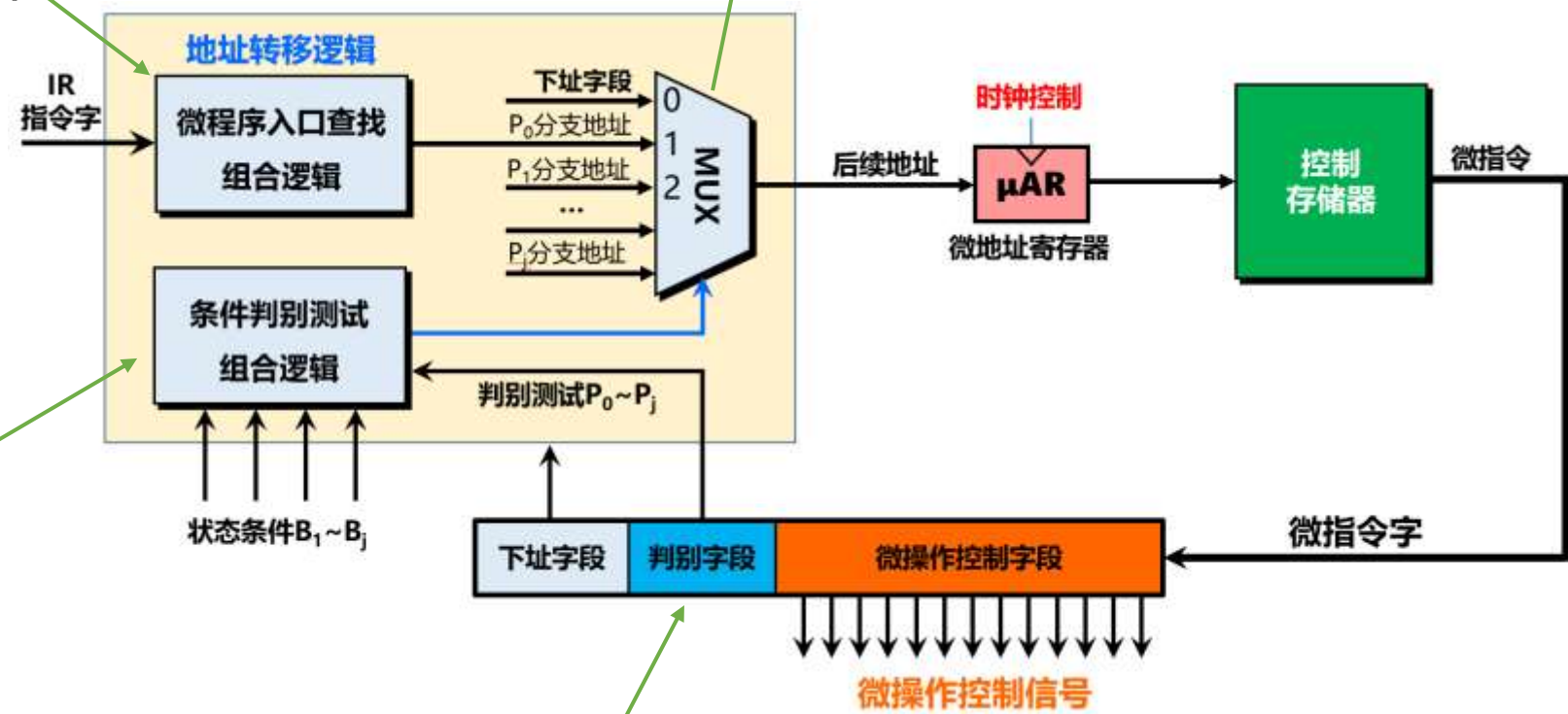


地址转移逻辑

- 将IR指令字直接转换成微程序入口地址，入口地址固定的

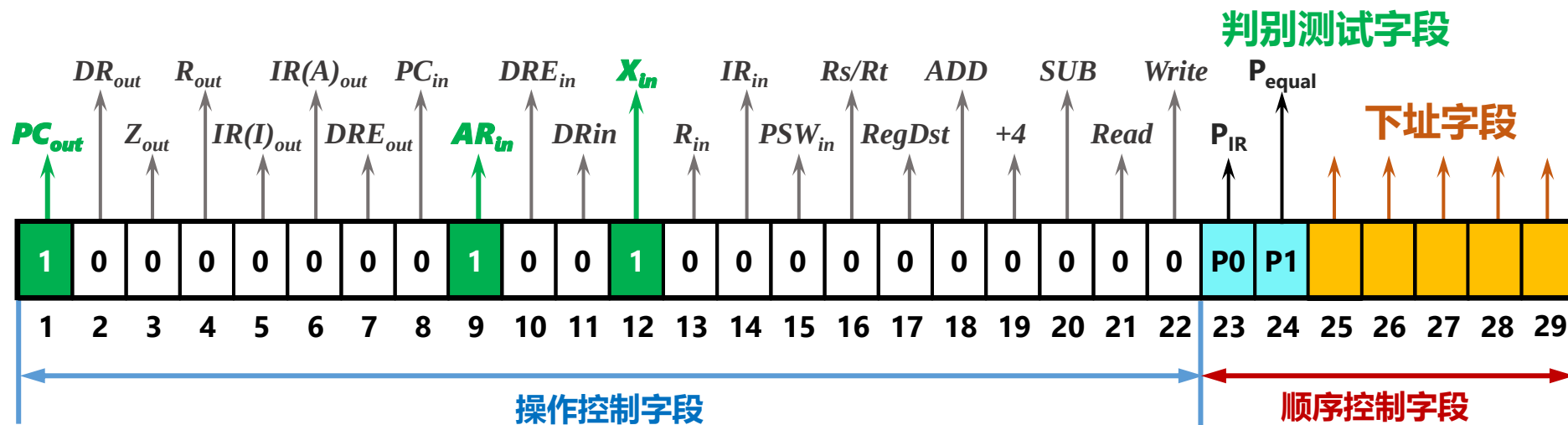
- 将所有可能的后续地址连接到多路选择器中输入，其中下址字段放在0号单元输出后续地址送入微地址寄存器 μAR ，多路选择器的选择控制端由条件判别测试逻辑根据判别测试字段和状态条件自动生成

- 列出判别测试条件与输出选择信号之间的关系



- 判别字段中的每一位均对应一个判别标志，判别测试条件成立微程序会分支跳转到预先设置的该判别标志的微程序分支地址处。

微指令设计



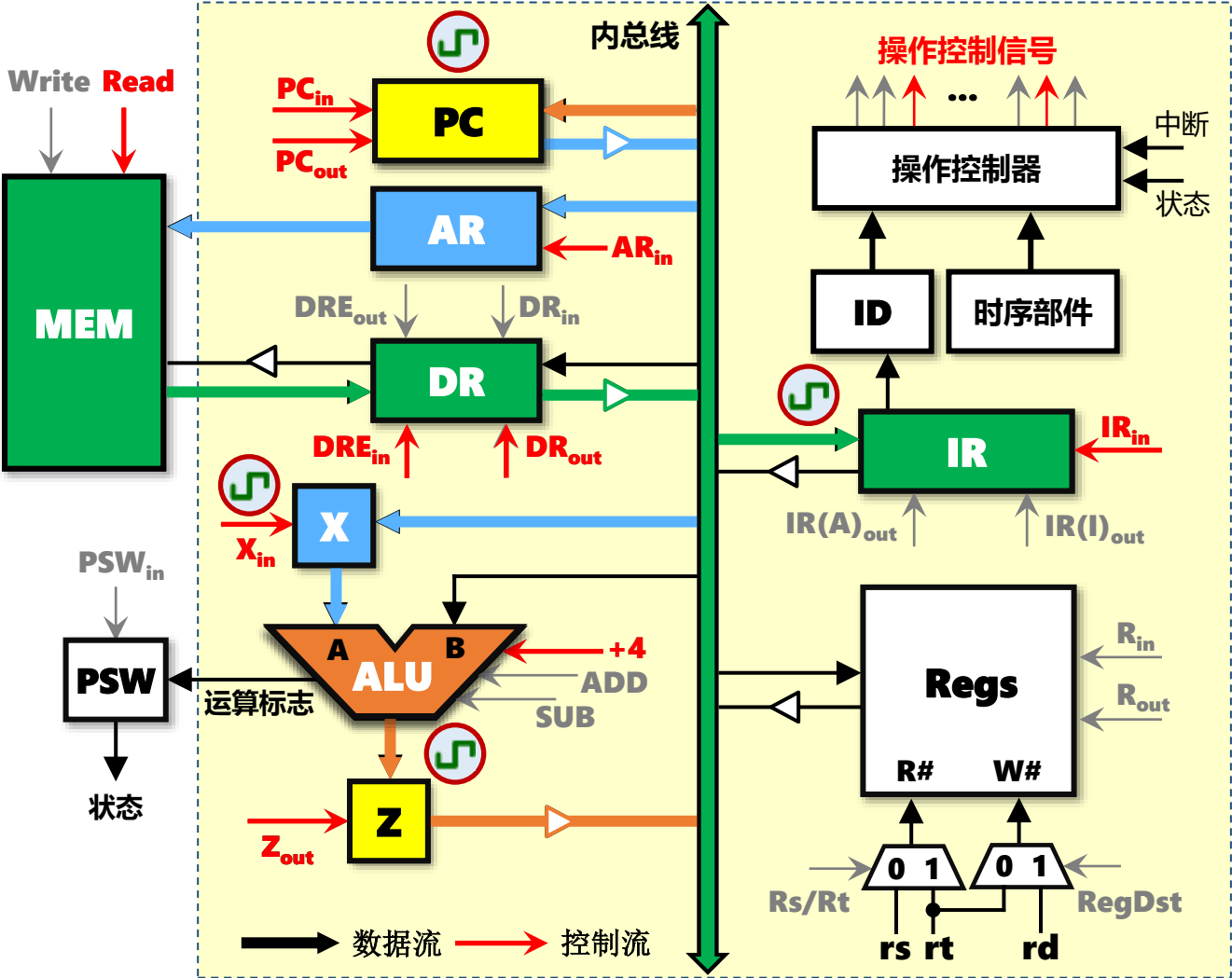
- 22个控点，操作控制字段应该包括22位，每一位对应一个控制信号
- beq指令需要使用条件状态寄存器 PSW 的 equal标志，P1判别测试位来对应 equal状态标志
- 下址字段的长度与控制存储器的容量有关，以及与所有微程序的规模相关，这里暂定为5位，控制存储器最多能支持32条微指令

微程序控制器设计

$M[PC++]\rightarrow IR$

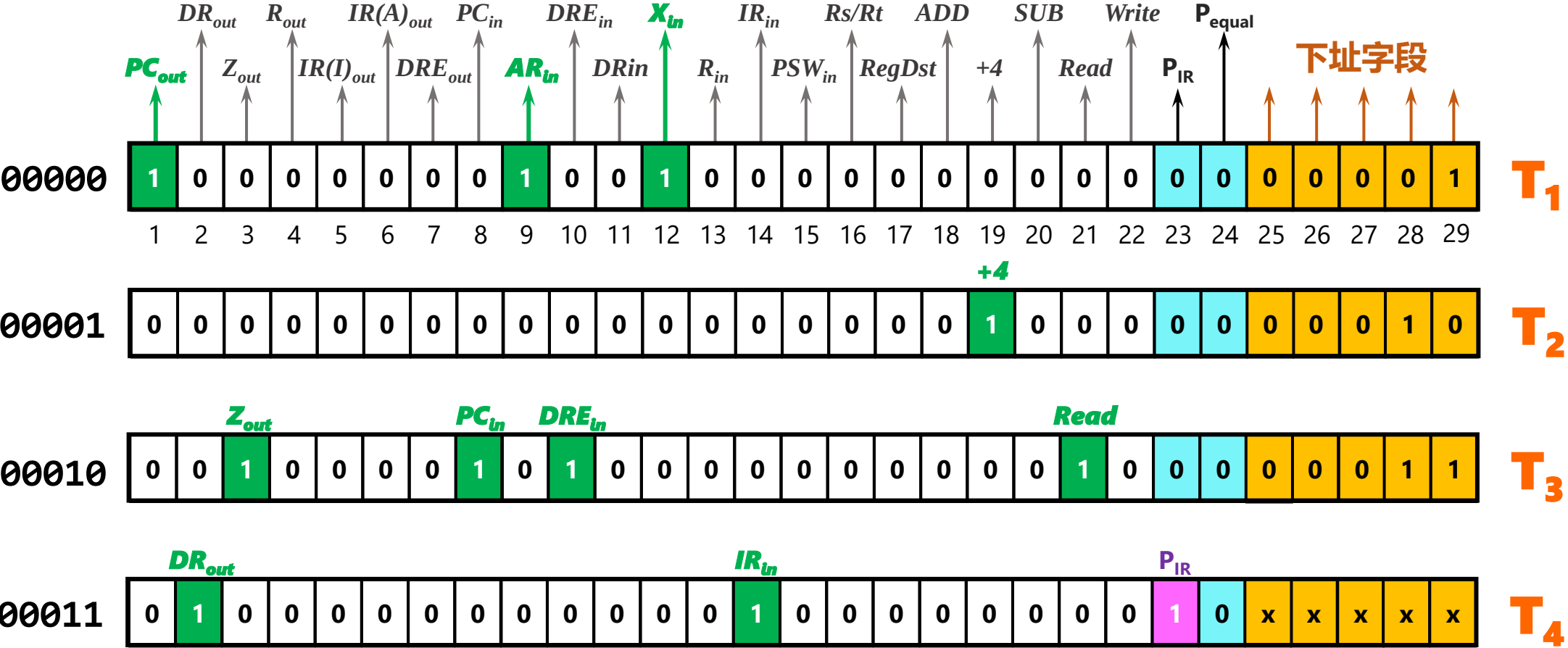
- 4个时钟周期
- 四条微指令

节拍	数据通路 (数据流)	控制信号(控制流)
T1	PC→AR, PC→X	PC _{out} , AR _{in} , X _{in}
T2	X+4→Z	+4
T3	Z→PC, M[AR]→DR	Z _{out} , PC _{in} , DRE _{in} , Read
T4	DR→IR	DR _{out} , IR _{in}



取指微程序

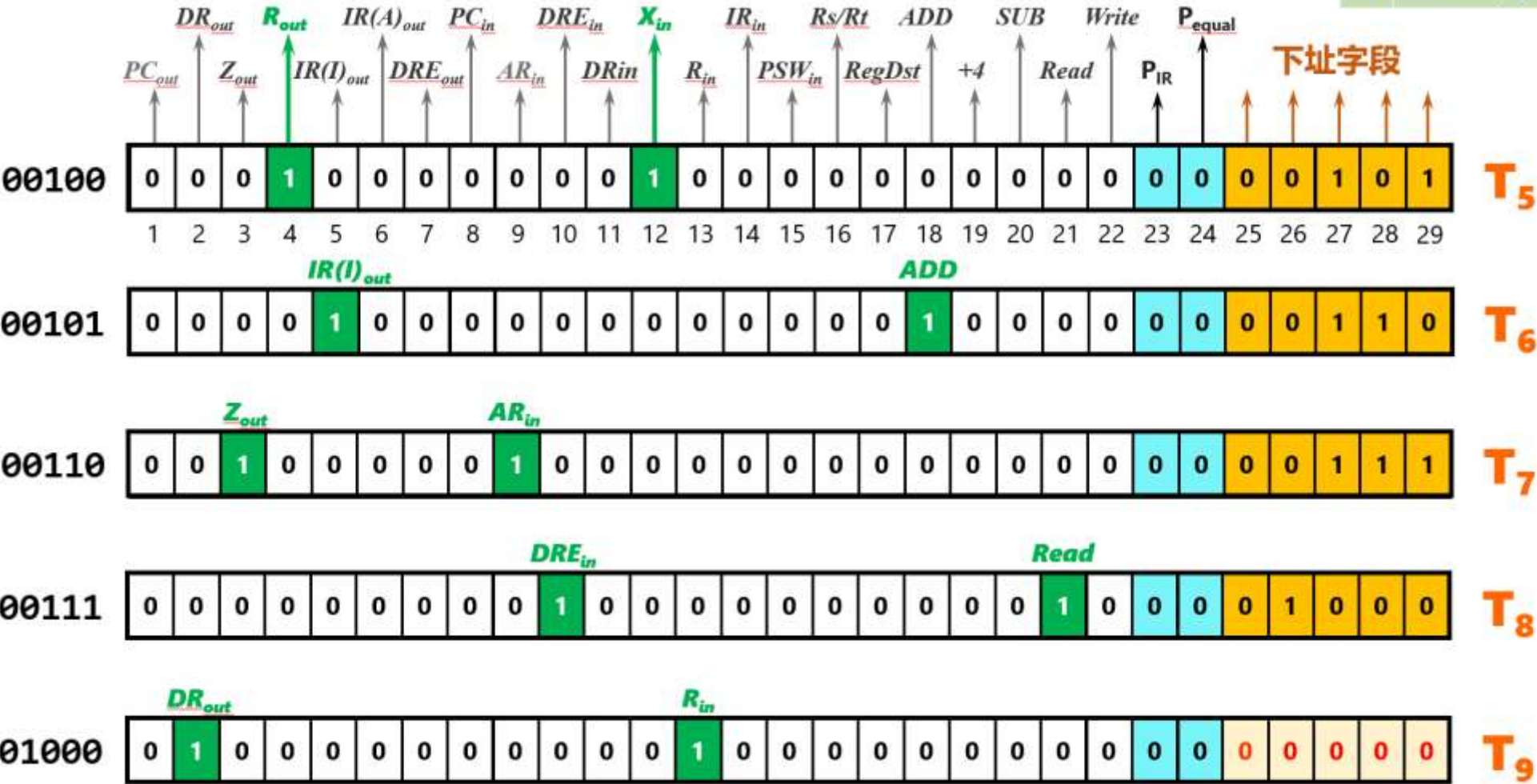
节拍	数据通路 (数据流)	控制信号(控制流)
T1	PC→AR, PC→X	PC _{out} , AR _{in} , X _{in}
T2	X+4→Z	+4
T3	Z→PC, M[AR]→DR	Z _{out} , PC _{in} , DRE _{in} , Read
T4	DR→IR	DR _{out} , IR _{in}



S3状态根据指令译码情况进行分支，所以3号微指令下址字段无效，同时应该设置指令译码测试位P0为1，表明执行这条微指令时要根据指令译码情况进行微程序分支

lw指令微程序

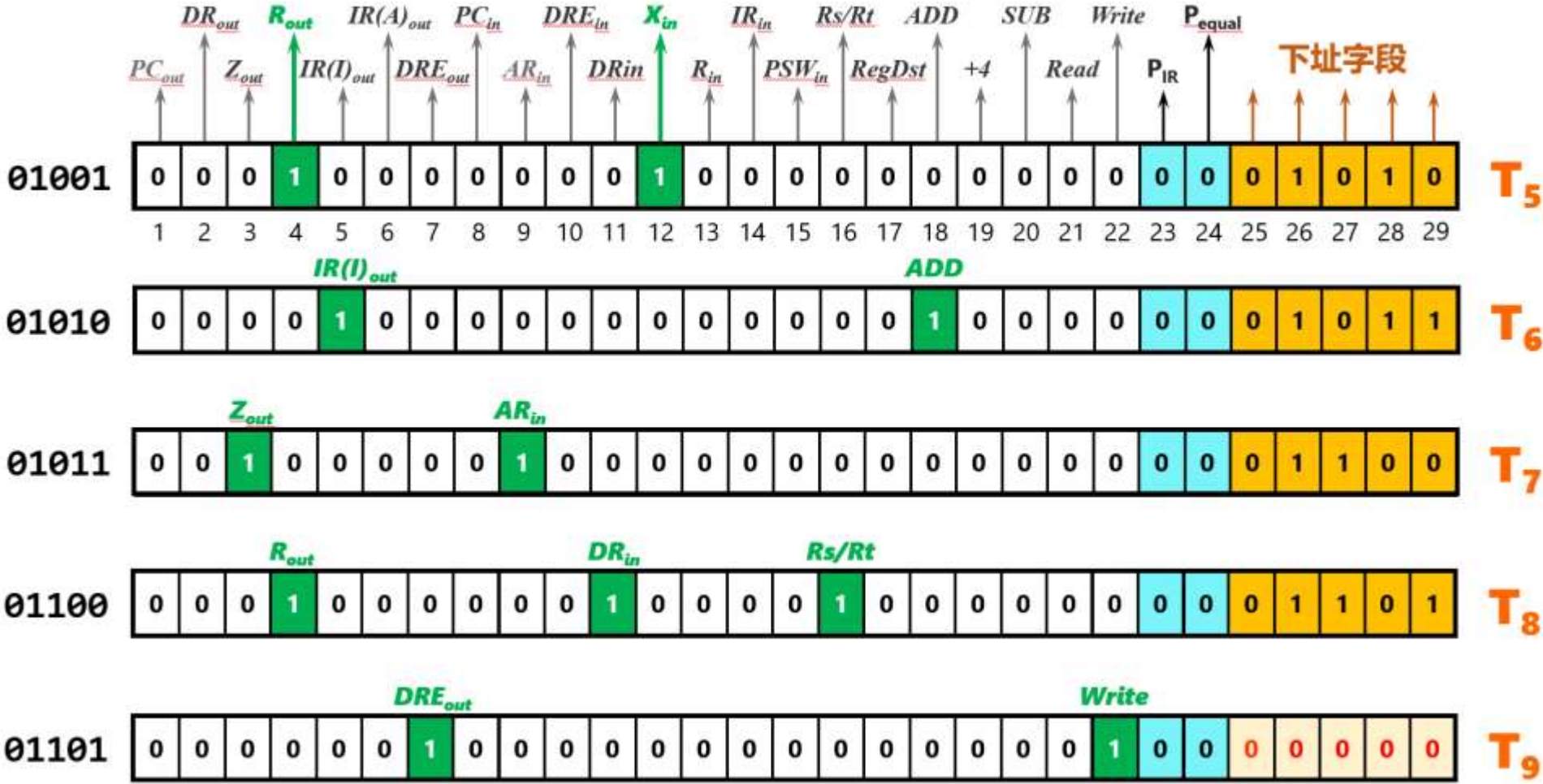
节拍	数据通路 (数据流)	控制信号 (控制流)
T5	$R[rs] \rightarrow X$	R_{out}, X_{in}
T6	$X + imm \rightarrow Z$	$IR(I)_{out}, ADD$
T7	$Z \rightarrow AR$	Z_{out}, AR_{in}
T8	$M[AR] \rightarrow DR$	$DRE_{in}, Read$
T9	$DR \rightarrow R[rt]$	DR_{out}, R_{in}



lw 指令微程序最后一条微指令(8号微指令)执行完毕后应该跳转回取指微程序, 其下址字段应该是取指微程序的入口地址, 这里设置为0

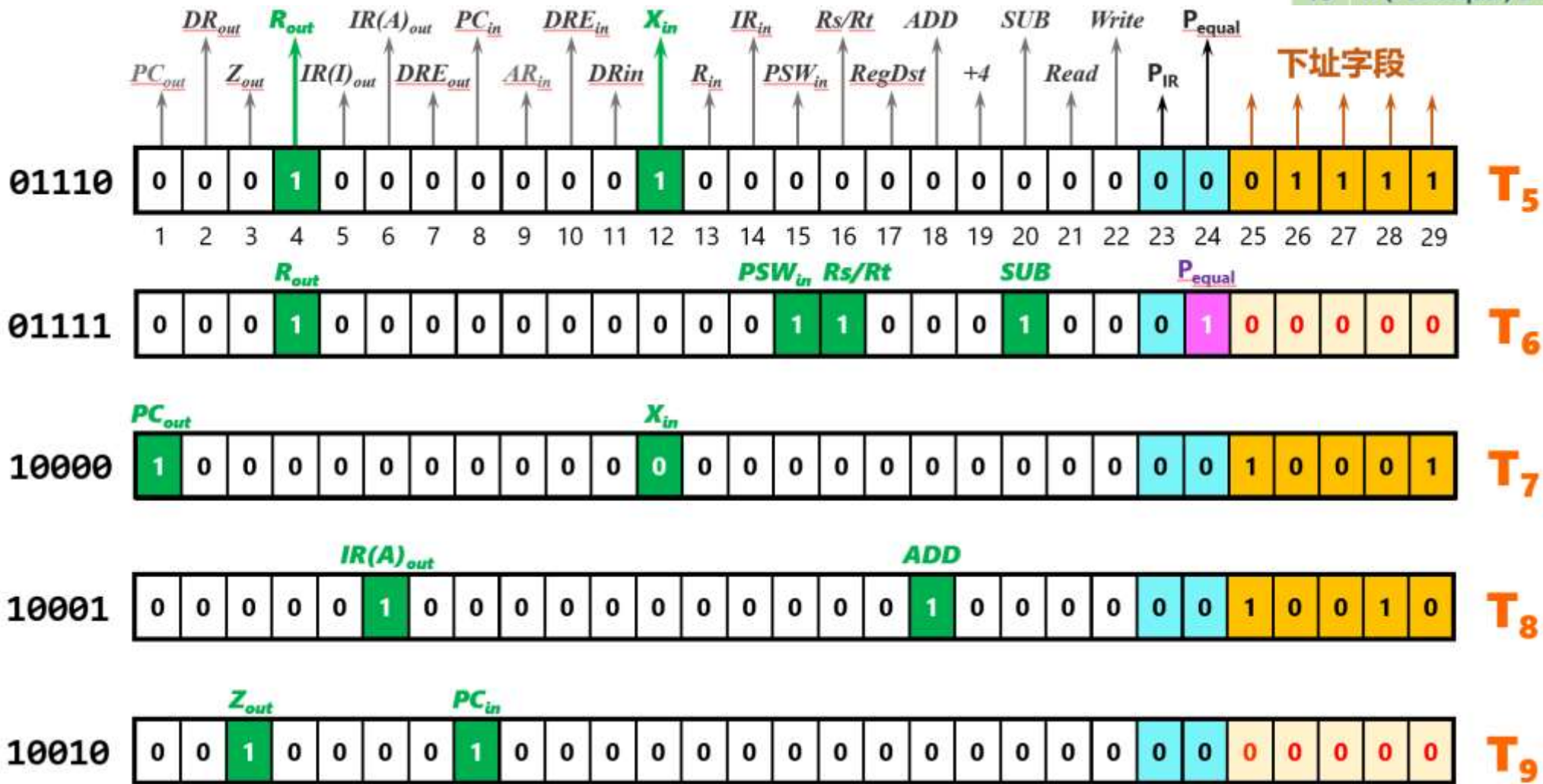
sw指令微程序

节拍	数据通路 (数据流)	控制信号 (控制流)
T5	$R[rs] \rightarrow X$	R_{out}, X_{in}
T6	$X + imm \rightarrow Z$	$IR(I)_{out}, ADD$
T7	$Z \rightarrow AR$	Z_{out}, AR_{in}
T8	$R[rt] \rightarrow DR$	$R_{out}, Rs/Rt, DR_{in}$
T9	$DR \rightarrow M[AR]$	$DRE_{out}, Write$



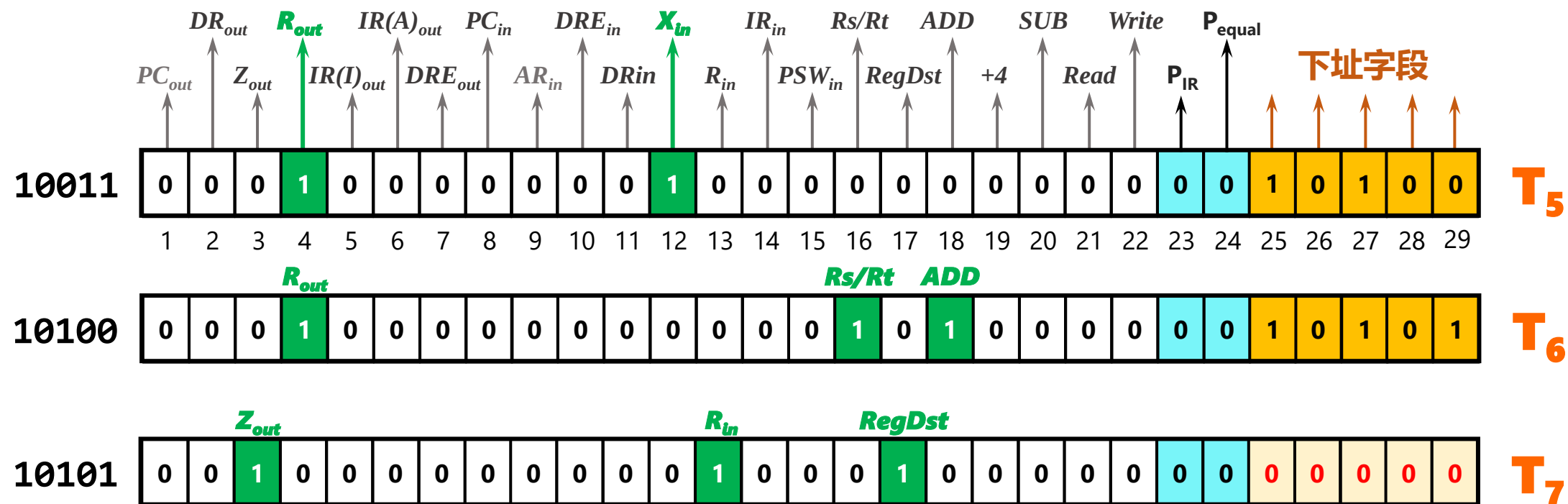
beq指令微程序

节拍	数据通路 (数据流)	控制信号(控制流)
T5	$R[rs] \rightarrow X$	R_{out}, X_{in}
T6	$X - R[rt] \rightarrow PSW$	$R_{out}, Rs/Rt, SUB, PSW_{ir}$
T7	$PC \rightarrow X$	PC_{out}, X_{in}
T8	$IR(A) + X \rightarrow Z$	$IR(A)_{out}, ADD$
T9	If (PSW.equal) $Z \rightarrow PC$	$Z_{out}, PC_{in} = PSW.equal$

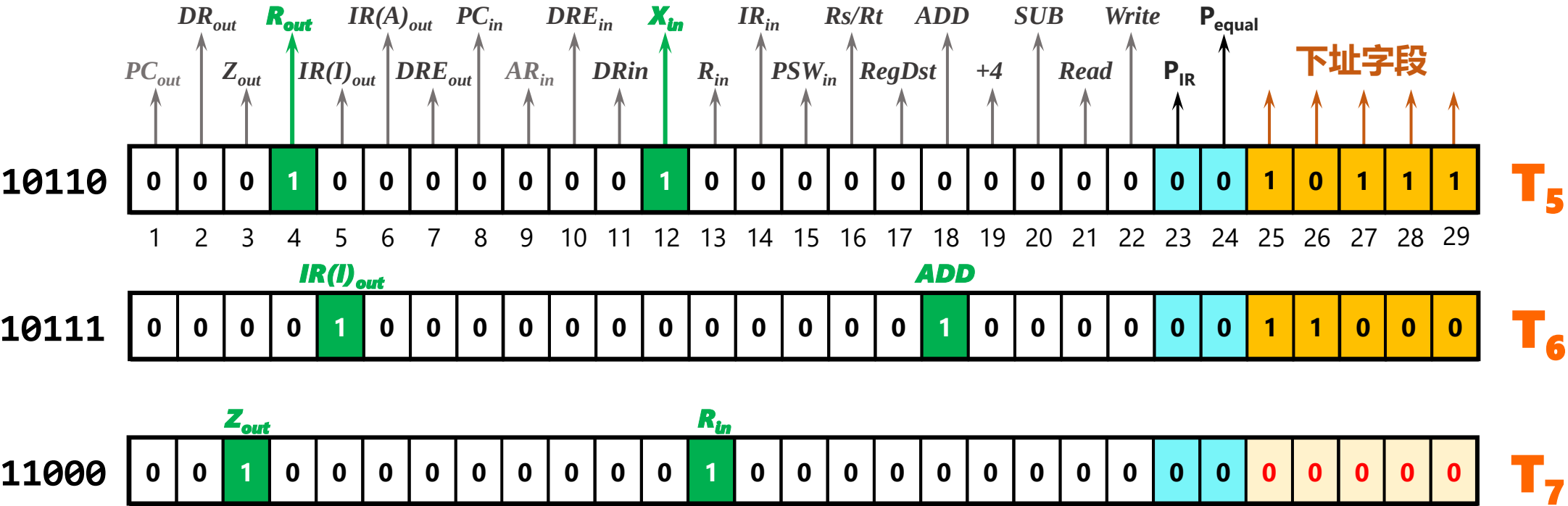


判别测试字段P
代表equal 测试

add指令微程序



addi指令微程序

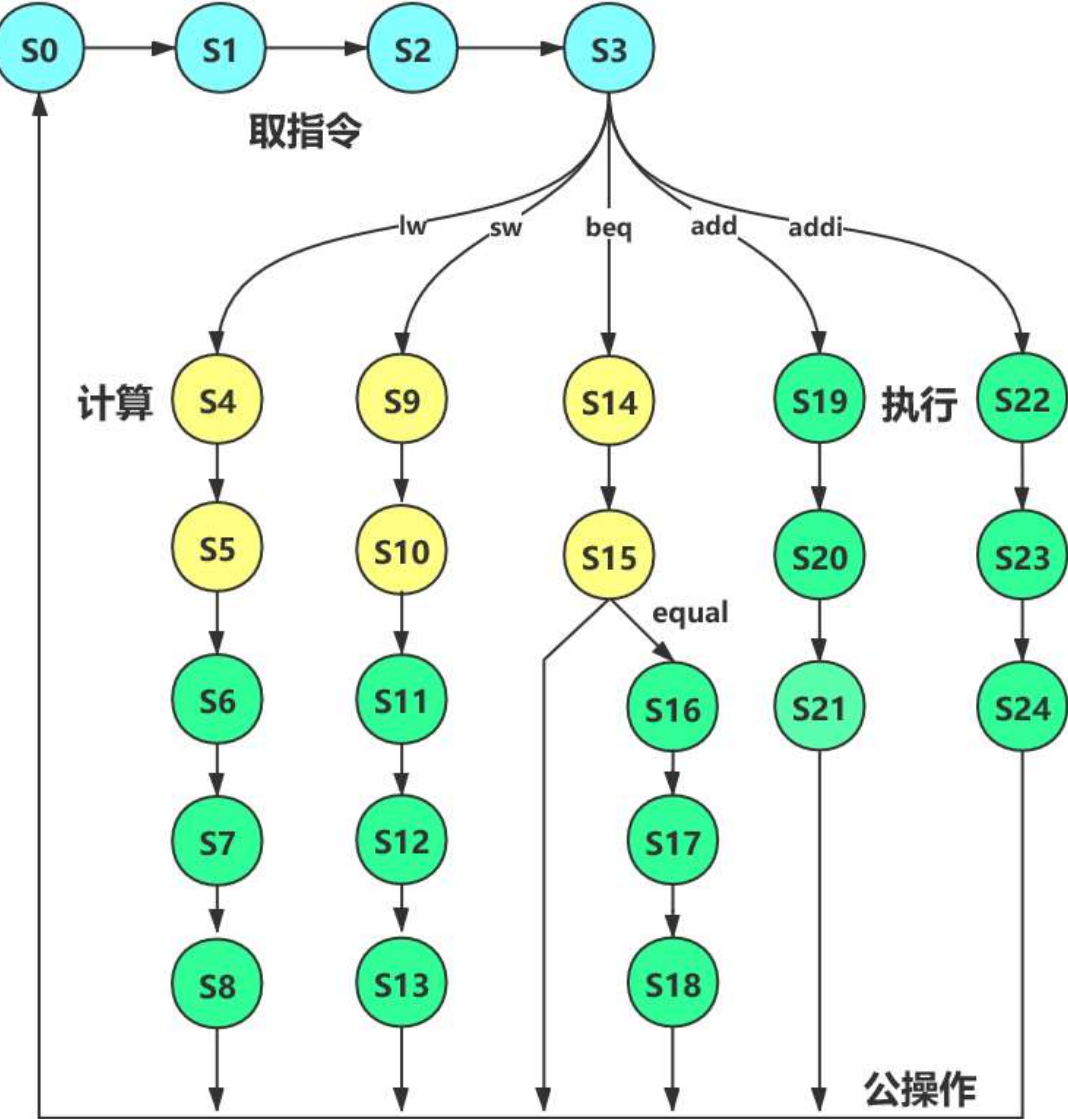


单总线CPU微程序

微地址与状态等价！

状态	微地址	操作控制字段																顺序控制字段											
0	00000	1								1			1										0	0	0	0	1	取指微程序	
1	00001																1						0	0	0	1	0		
2	00010			1					1		1								1				0	0	0	1	1		
3	00011		1												1							1		x	x	x	x		x
4	00100				1								1										0	0	1	0	1	lw	
5	00101					1											1						0	0	1	1	0		
6	00110			1						1													0	0	1	1	1		
7	00111										1									1			0	1	0	0	0		
8	01000		1											1									0	0	0	0	0	sw	
9	01001				1								1										0	1	0	1	0		
10	01010					1											1						0	1	0	1	1		
11	01011			1						1													0	1	1	0	0		
12	01100				1							1					1						0	1	1	0	1	beq	
13	01101							1												1			0	0	0	0	0		
14	01110				1								1										0	1	1	1	1		
15	01111				1										1	1				1			1	0	0	0	0		
16	10000	1											1											1	0	0	0	1	add
17	10001						1										1						1	0	0	1	0		
18	10010			1					1														0	0	0	0	0		
19	10011				1								1										1	0	1	0	0		
20	10100				1										1		1						1	0	1	0	1	addi	
21	10101			1										1			1						0	0	0	0	0		
22	10110				1								1										1	0	1	1	1		
23	10111					1											1						1	1	0	0	0		
24	11000			1									1										0	0	0	0	0		

指令执行状态转换图 → 状态转换表



现态	lw	sw	beq	add	addi	equal	次态
S0	x	x	x	x	x		S1
S1	x	x	x	x	x		S2
S2	x	x	x	x	x		S3
S3	1						S4
S3		1					S9
S3			1				S14
S3				1			S19
S3					1		S22

程序执行动态?

主存

#	指令
1	lw rt,imm(rs)
2	beq rs,rt,imm
3	add rd,rs,rt
4	sw rt,imm(rs)

地址	微程序	微指令
0	取指微程序	1
1		2
2		3
3		4
4	lw微程序	1
5		2
6		3
7		4
8		5
9	sw微程序	1
10		2
11		3
12		4
13		5
14	beq微程序	1
15		2
16		3
17		4
18		5
19	add微程序	1
20		2
21		3

控存

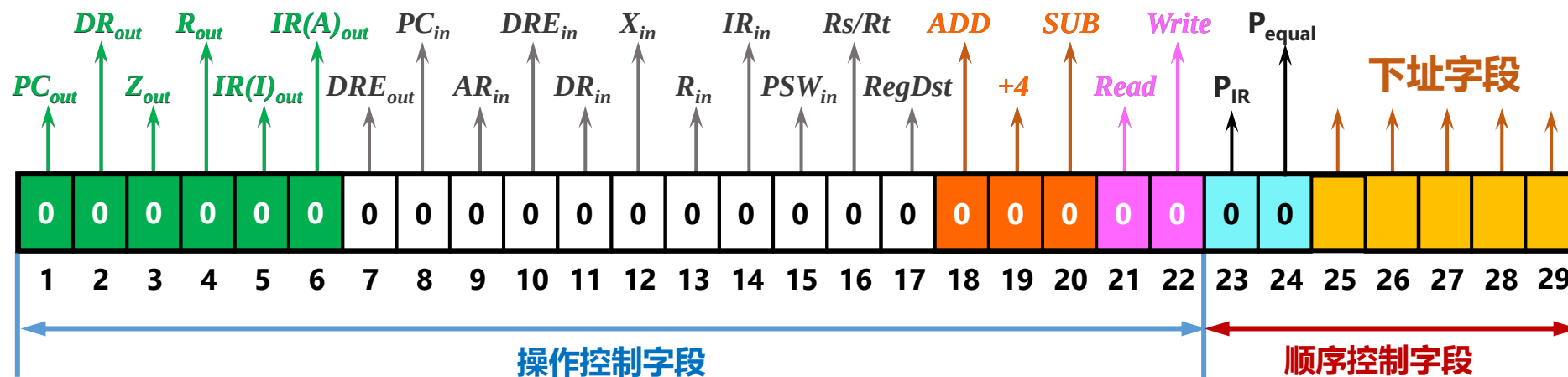
微指令设计原则

- 有利于缩短微指令字长度
- 有利于减少控制存储器容量
- 有利于提高微程序执行速度
- 有利于对微指令进行修改
- 有利于提高微程序设计的灵活性

微命令编码

- 直接表示法
- 编码表示法
- 混合表示法

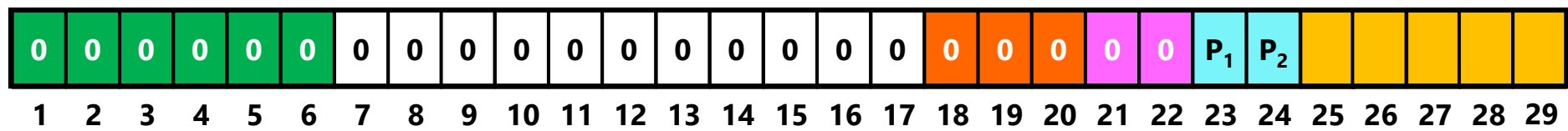
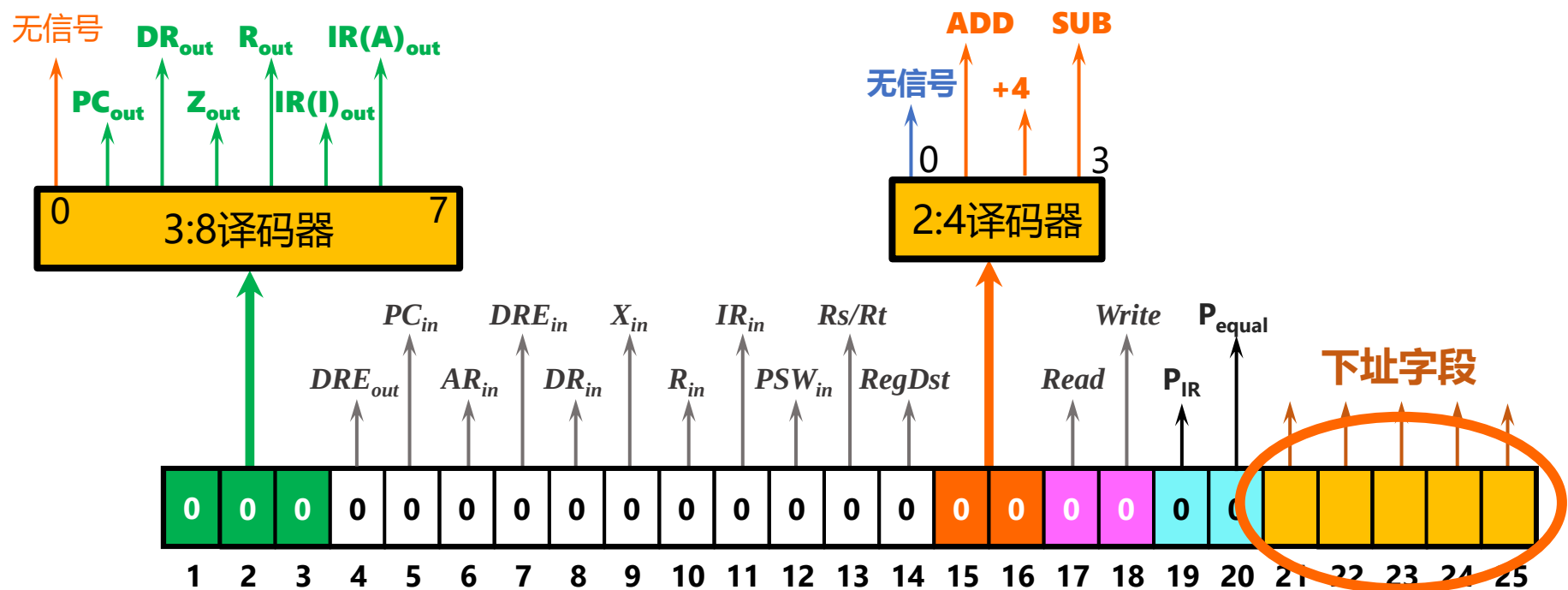
直接表示法



- 简单直观，便于输出控制，微指令长度太长，控存容量大，如何压缩微指令长度？
 - ◆ 将微指令操作控制字段的每个二进制位定义为一个微命令
 - ◆ “1” 或 “0” 表示相应的微命令的 “有” 或 “无”

编码表示法

■ 如果互斥的输出控制信号8个，编码后长度多少？



每个译码器的输出状态中需要预留一个状态，表示当前微指令不使用本组互斥性微命令中的任何一个

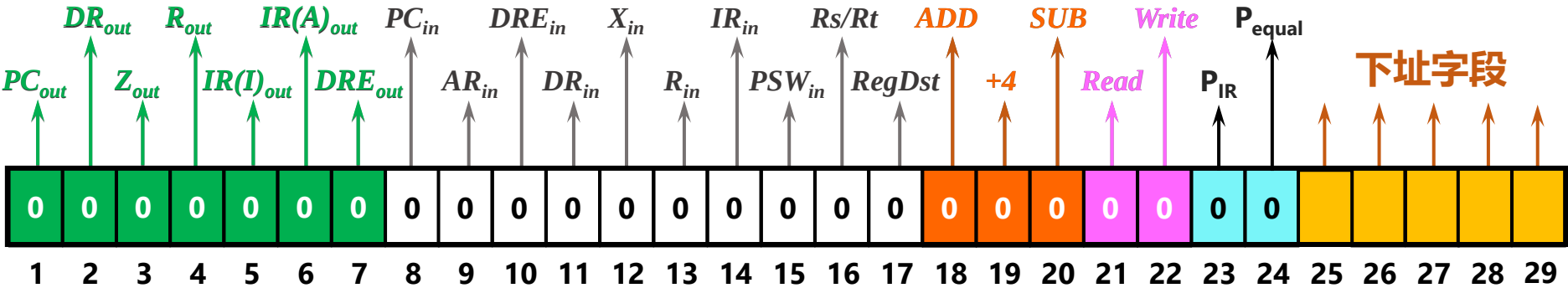


■ 例题6.6

水平型微指令编码效率

- 在一个微指令周期内能同时给出多个微命令的微指令称为水平型微指令
 - 直接表示法
 - 编码表示法
 - 混合表示法

状态	微地址	操作控制字段																顺序控制字段							
0	00000	1																	0	0	0	0	0	1	
1	00001																		0	0	0	0	1	0	
2	00010																		0	0	0	0	1	1	
3	00011																		0	0	0	0	1	1	
4	00100																		0	0	1	0	1	0	
5	00101																		0	0	1	1	1	0	
6	00110																		0	0	1	1	1	1	
7	00111																		0	1	0	0	0	0	
8	01000																		0	0	0	0	0	0	
9	01001																		0	1	0	1	0	0	
10	01010																		0	1	0	1	1	0	
11	01011																		0	1	1	0	0	0	
12	01100																		0	1	1	0	1	0	
13	01101																		0	0	0	0	0	0	
14	01110																		0	1	1	1	1	1	
15	01111																		0	0	0	0	0	0	
16	10000																		1	0	0	0	1	0	
17	10001																		1	0	0	1	0	0	
18	10010																		0	0	0	0	0	0	
19	10011																		1	0	1	0	0	0	
20	10100																		1	0	1	0	1	0	
21	10101																		0	0	0	0	0	0	
22	10110																		1	0	1	1	1	1	
23	10111																		1	1	0	0	0	0	
24	10100																		0	0	0	0	0	0	



垂直型微指令

■ 数据通路分类

- ◆ 寄存器传输 `MOV reg1, reg2` `MOV AR, PC`
- ◆ 运算类型 `ALU_OP reg` `INC、ADD R1`
- ◆ 访存类型 `Mem` `LOAD、STORE`
- ◆ 顺序控制 `Branch` `Branch P1`

如采用扩展操作码，需13位字长

MOV	源寄存器(6)	目的寄存器(6)
INC	无操作数	
ADD	源寄存器编号 (6位)	
Mem	Read/Write	
Branch	判断条件	

节拍	取指令	lw	sw	beq	add	addi
T1	PC→AR, PC→X	R[rs]→X	R[rs]→X	R[rs]→X	R[rs]→X	R[rs]→X
T2	X+4→Z	X+imm→Z	X+imm→Z	X-R[rt]→PSW	X+R[rt]→Z	X+imm→Z
T3	Z→PC, M[AR]→DR	Z→AR	Z→AR	PC→X	Z→R[rd]	Z→R[rt]
T4	DR→IR	M[AR]→DR	R[rt]→DR	IR(A)+X→Z		
T5		DR→R[rt]	DR→M[AR]	If (PSW.equal) Z→PC		

垂直型微指令实例

LOAD指令微程序容量:

■ 直接表示法 + 下址字段

◆ 29位*9=261位

■ 编码表示法

■ 25位*9=225位

■ 21位*9=189位

■ 垂直微指令

◆ 13位*13=169位

◆ 容量小，速度慢，垂直微指令执行较复杂，需译码电路

节拍	LOAD指令	#	LOAD	信号
T1	PC→AR, PC→X	C1	MOV AR,PC	Pc _{out} , AR _{in} ,
		C2	MOV X,PC	Pc _{out} , X _{in}
T2	X+4→Z	C3	ADD 4	+4
T3	Z→PC, M[AR]→DR	C4	MOV PC,Z	Z _{out} , PC _{in}
		C5	LOAD	DRE _{in} , Read
T4	DR→IR	C6	MOV IR,DR	DR _{out} , IR _{in}
		C7	Branch P ₀	
T5	R[rs]→X	C8	MOV X, R[rs]	R _{out} , X _{in}
T6	X+imm→Z	C9	ADD IR(I)	IR(I) _{out} , ADD
T7	Z→AR	C10	MOV AR, Z	Z _{out} , AR _{in}
T8	M[AR]→DR	C11	LOAD	DRE _{in} , Read
T9	DR→R[rt]	C12	MOV R[rt], DR	DR _{out} , R _{in}
		C13	BRANCH P _{end}	

水平型微指令与垂直型微指令

■ 水平型微指令

- ◆ 并行操作能力强，效率高，执行时间短、灵活性强，
- ◆ 微指令字较长，微程序短，控存容量大，性能佳

■ 垂直型微指令

- ◆ 字长短，微程序长，控存容量小，性能差
- ◆ 垂直型与指令相似，易于掌握
- ◆ 基本被淘汰

微程序优势与劣势

- **速度慢 访存频繁、成本低廉**
- **设计规整，设计简单，易于修改、扩展指令系统功能**
 - 适合CISC等功能较复杂的系列机 X86、IBM S/360、DEC VAX
 - 可写控存方便修复出厂故障 Intel Core 2 、 Intel Xeon
- **硬布线控制器执行速度快，但设计复杂，代价昂贵，不便于修改**
 - 适合RISC计算机，如MIPS，ARM

设计处理器的步骤

1. **选定指令系统**，将指令功能用RTL来表示
2. **设计部件与通路**，根据指令功能设计功能部件，并考虑如何互连
3. **设计控点**，确定所有功能部件，数据通路所需控制信号
4. **分析所有指令的控制信号序列**，生成指令与控制信号关系表
5. **设计控制器**
 - 设计微指令，微程序，地址转移逻辑，设计微程序控制器
 - 生成每个控制信号的逻辑表达式，设计硬布线控制器

硬件设计四原则

■ 简单性来自规则性

- Simplicity favors regularity
- 指令越规整设计越简单

■ 越小越快

- Smaller is faster
- 面积小，传播路径小，门延迟少

■ 加快经常性事件

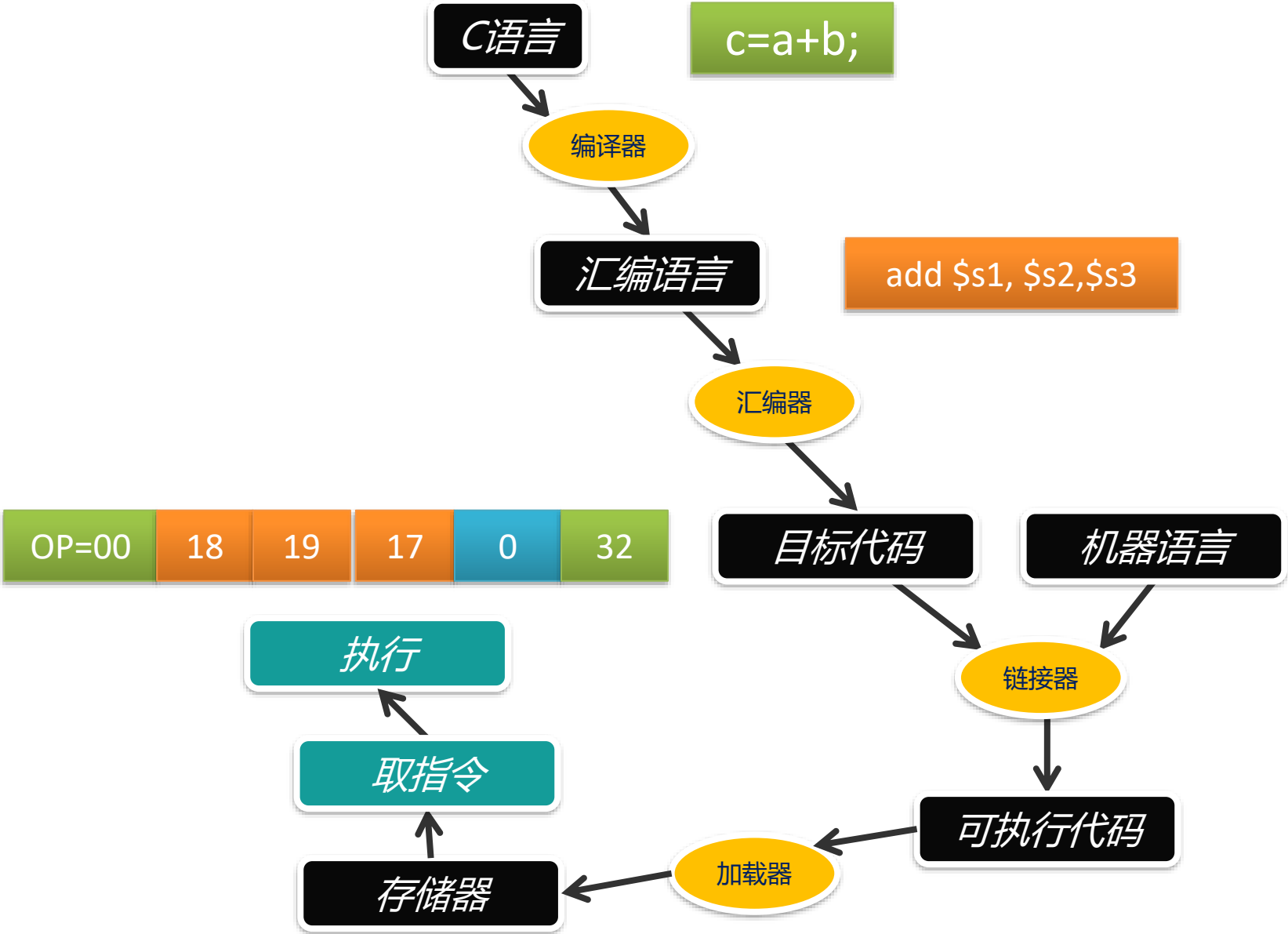
- Make the common case fast

■ 好的设计需要适度的折衷

- Good design demands good compromises.

程序员角度理解

高级语言如何执行成二值逻辑



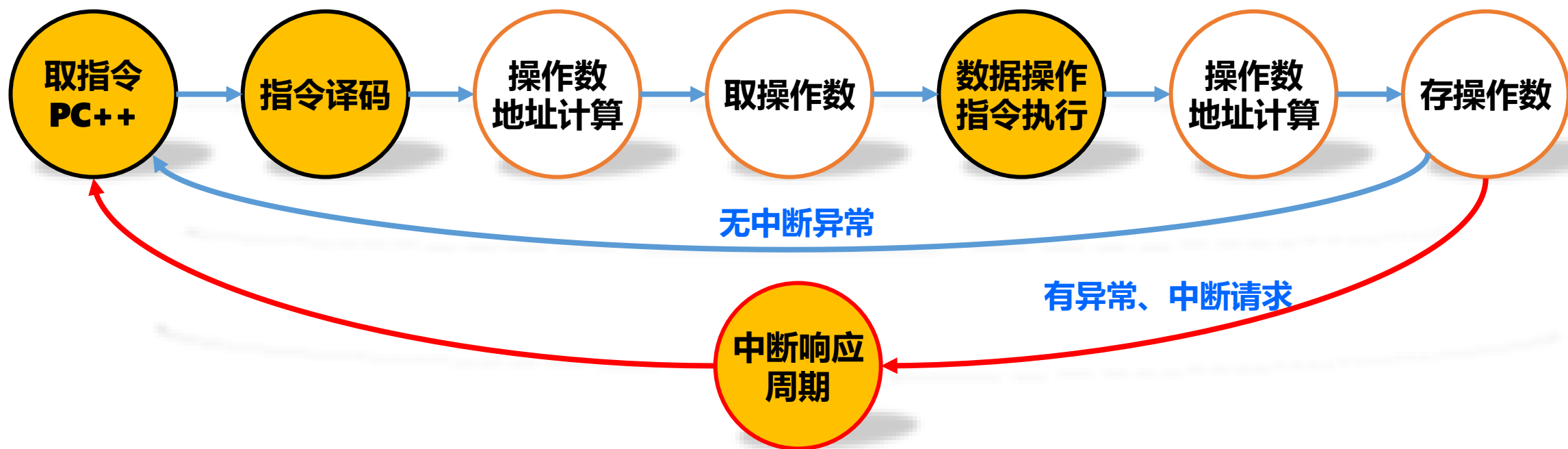
本章主要内容

- 6.1 中央处理器概述
- 6.2 指令周期
- 6.3 数据通路及指令操作流程
- 6.4 时序与控制
- 6.5 硬布线控制器
- 6.6 微程序控制器
- 6.7 异常与中断处理



中断机制实现

- 取指令、执行指令反复循环
- 如何加入中断周期?



异常与中断

■ 内部异常

□ 故障 (Fault)

- ◆ 由指令执行引起的异常，如未定义指令、越权指令、段故障、缺页故障、存储保护违例、数据未对齐、除数为零、浮点溢出、整数溢出等。
- ◆ 可恢复的故障，指令需恢复执行，不可恢复故障，进程被终止

□ 自陷 (trap)

- ◆ 是一种事先安排的“异常”事件，通过在程序中显式的调用自陷指令触发自陷异常，用于在用户态下调用操作系统内核程序，如系统调用、条件陷阱指令。

□ 终止 (Abort)

- ◆ 随机出现的使得 CPU 无法继续执行的硬件故障，和具体指令无关。如机器校验错、总线错误、异常处理中再次异常的双错等，此时当前程序无法继续执行，只能终止执行，由异常服务处理程序来重启系统

■ 外部中断

■ 关中断

临时禁止中断请求，是为了保障中断响应周期以及中断服务程序中保护现场操作的完整性

■ 保存断点

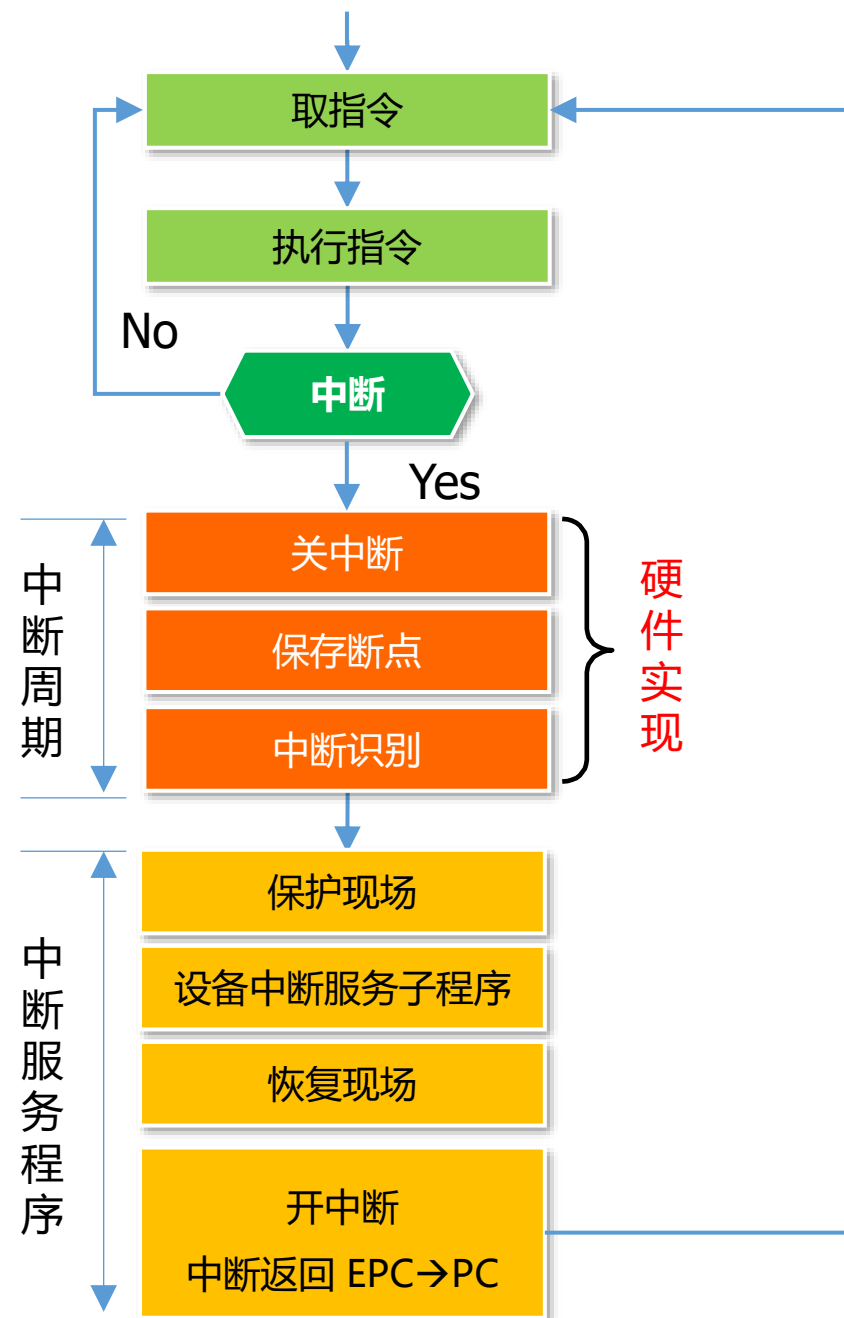
保存将来返回被中断程序的位置，对于已经执行完毕的指令其断点是下一条指令的位置（注意有可能不是顺序指令），对于缺页故障，段错等执行指令引起的故障异常，由于指令并没有执行，所以断点应该是异常指令的 PC 值。

■ 中断识别

就是根据当前的中断请求识别出中断来源，也就是发生了什么中断，并将对应中断的中断服务程序入口地址送程序计数器 PC

单重中断

中断周期
中断隐指令



支持中断异常处理的数据通路升级

■ 开关中断

□ 增加IE寄存器

■ 保存断点

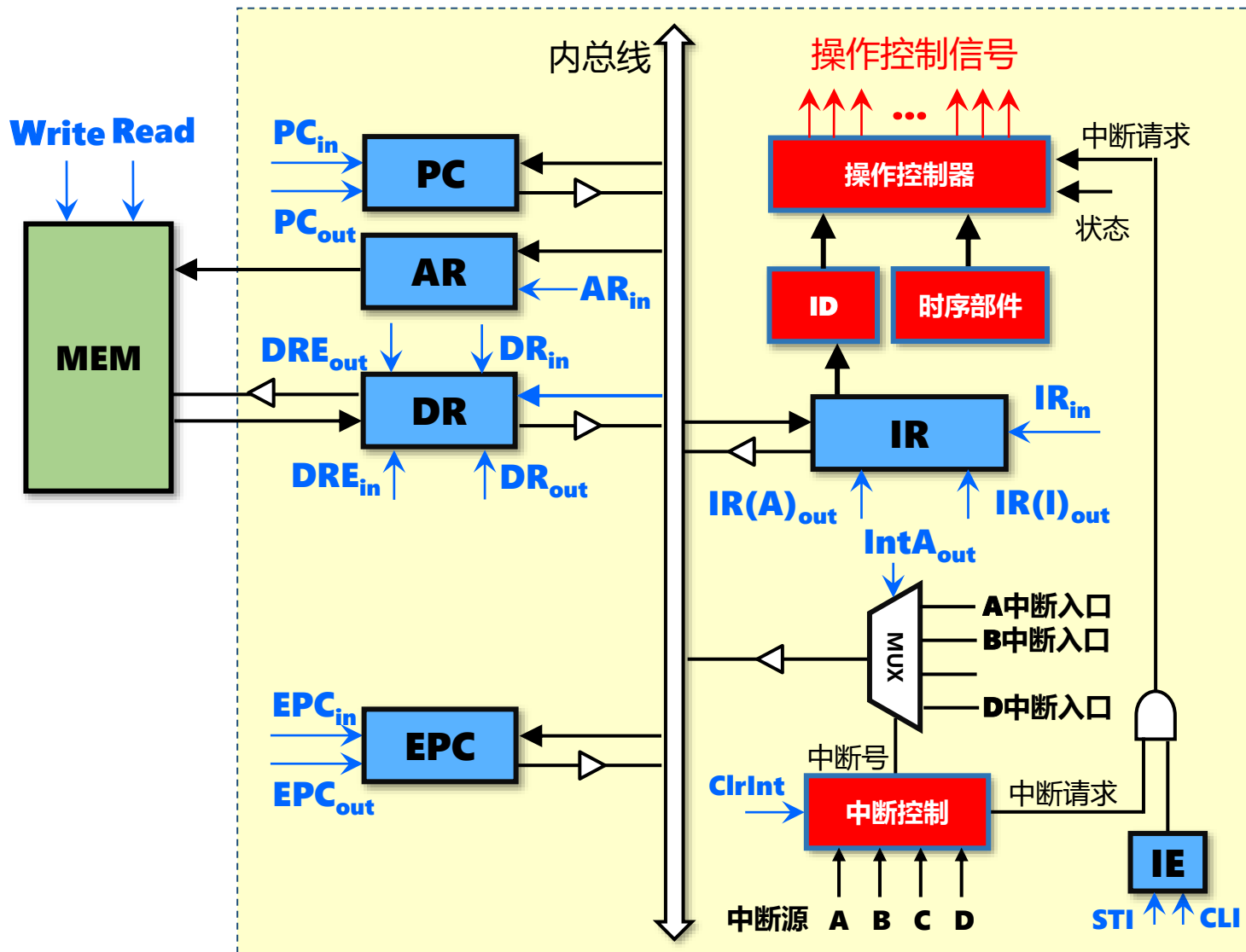
□ 增加EPC寄存器/堆栈

■ 中断识别

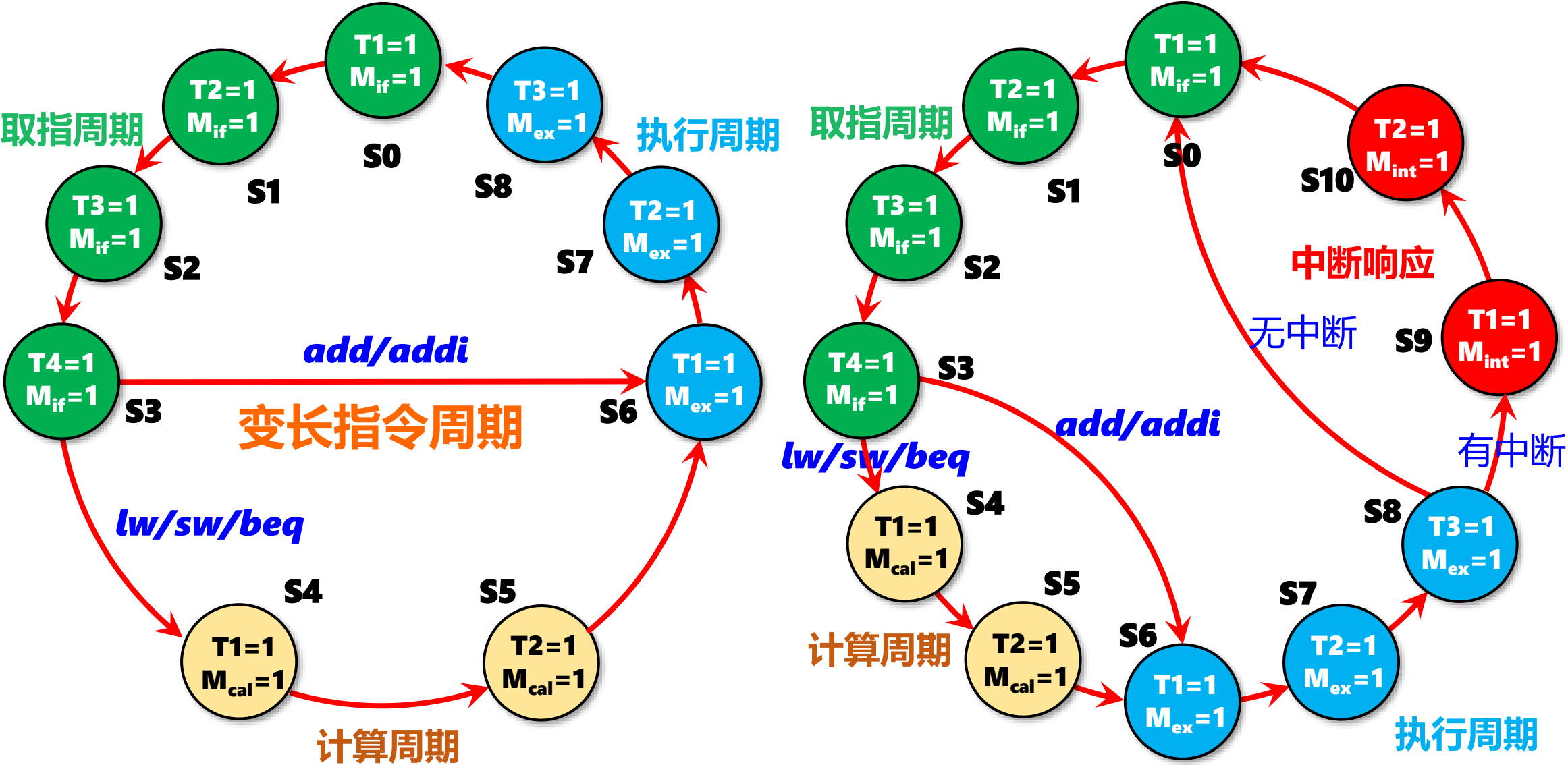
□ 增加中断控制逻辑

■ 软件支持

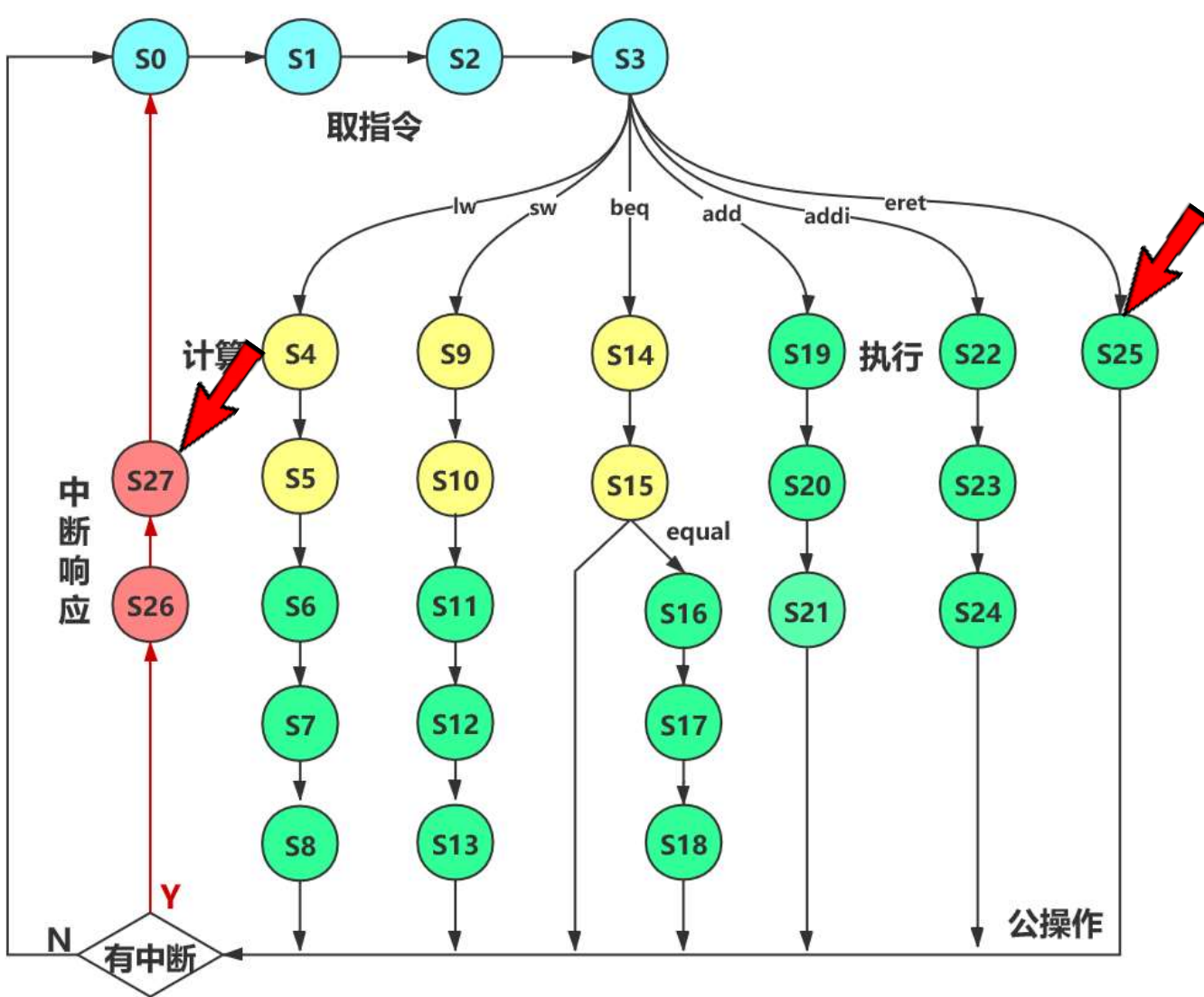
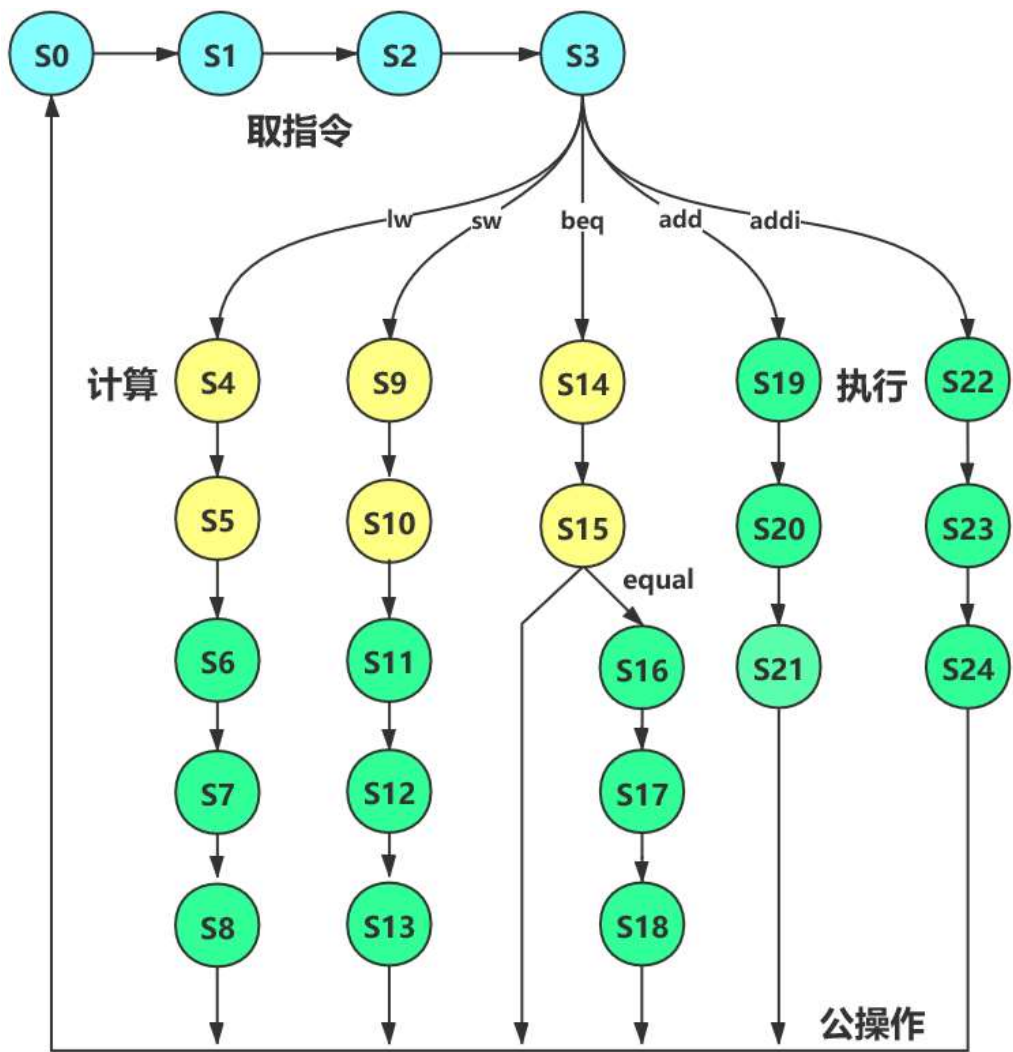
□ eret指令支持



变长指令周期状态机修改



现代时序状态机修改



所有指令的最后一个状态都会进入公操作判断是否有中断请求，如不存在中断请求，直接进入取指令阶段的 S0 状态；如果存在中断请求，则进入 S26~S27 的中断响应周期

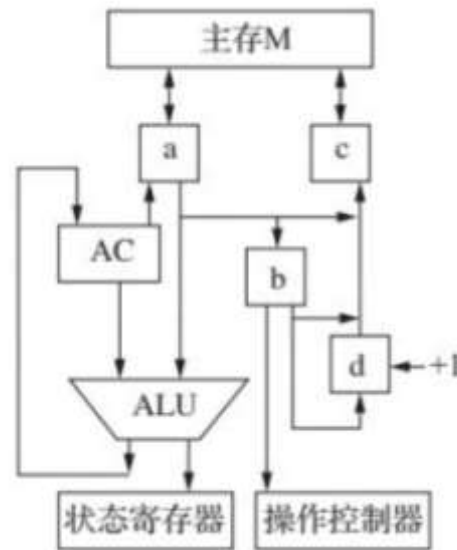


■ 课后习题6.4, 6.20 (1) , 6.21



■ 课后习题6.4答案

- (1) a:DR, b:IR, c:AR, d:PC。
(2) 取指阶段数据通路:PC→AR→主存M→DR→IR, PC→PC+1。
执行阶段数据通路:IR(A)→AR→主存M→DR→AC。





■ 课后习题6.20 (1) 答案

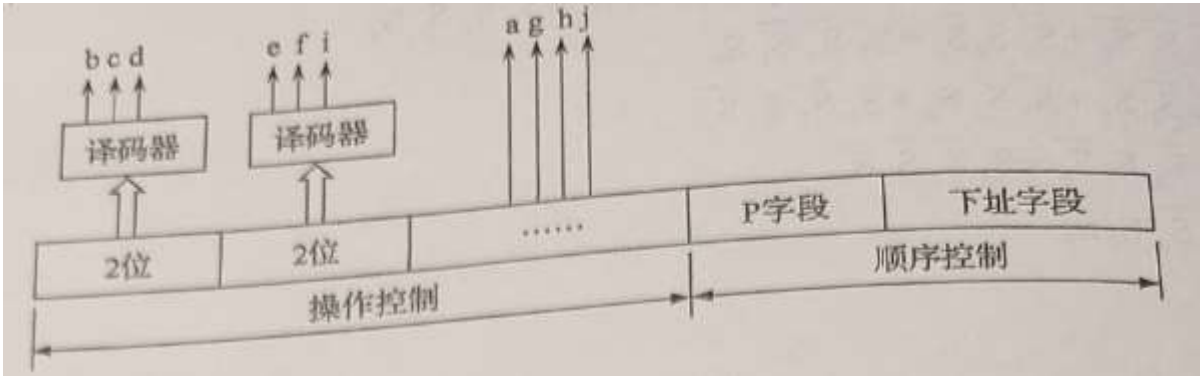
控制存储器容量 128×32 位 $= 2^7 \times 32$ 位
下址字段为7 位
判别测试条件为3
操作控制字段为 $32 - 7 - 3 = 22$ 位。



■ 课后习题6.21答案

控制信号 微指令	a	b	c	d	e	f	g	h	i	j
μL_1	✓		✓		✓		✓			
μL_2	✓			✓		✓		✓		✓
μL_3	✓			✓	✓					
μL_4	✓	✓							✓	
μL_5	✓			✓		✓				✓

从该表中可以发现两个互斥组(b,c,d)和(e,f,i)，用户可以将这两个互斥组采用字段译码法进行编码，其余的a、g、h、j这4个微命令采用直接表示法

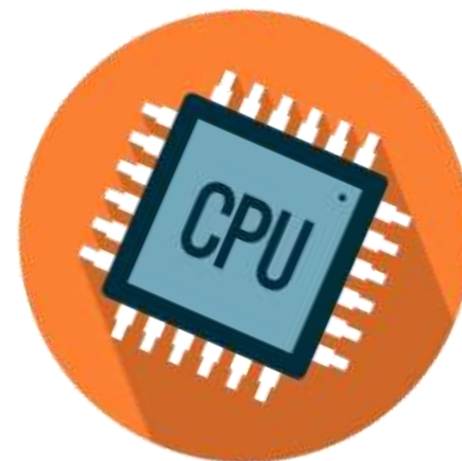




计算机组成原理



附A、单周期MIPS



MIPS CPU控制器设计

■ 定长指令周期：单周期实现

- ◆ 所有指令均在一个时钟周期内完成， $CPI=1$
- ◆ 时钟周期对所有指令等长，因此只能以执行速度最慢的指令作为设计其时钟周期的依据
- ◆ 性能取决于最慢的指令，时钟周期过长

■ 变长指令周期：多周期实现

- 缩短时钟周期，复用器件或数据通路
- 可支持流水操作，提升性能

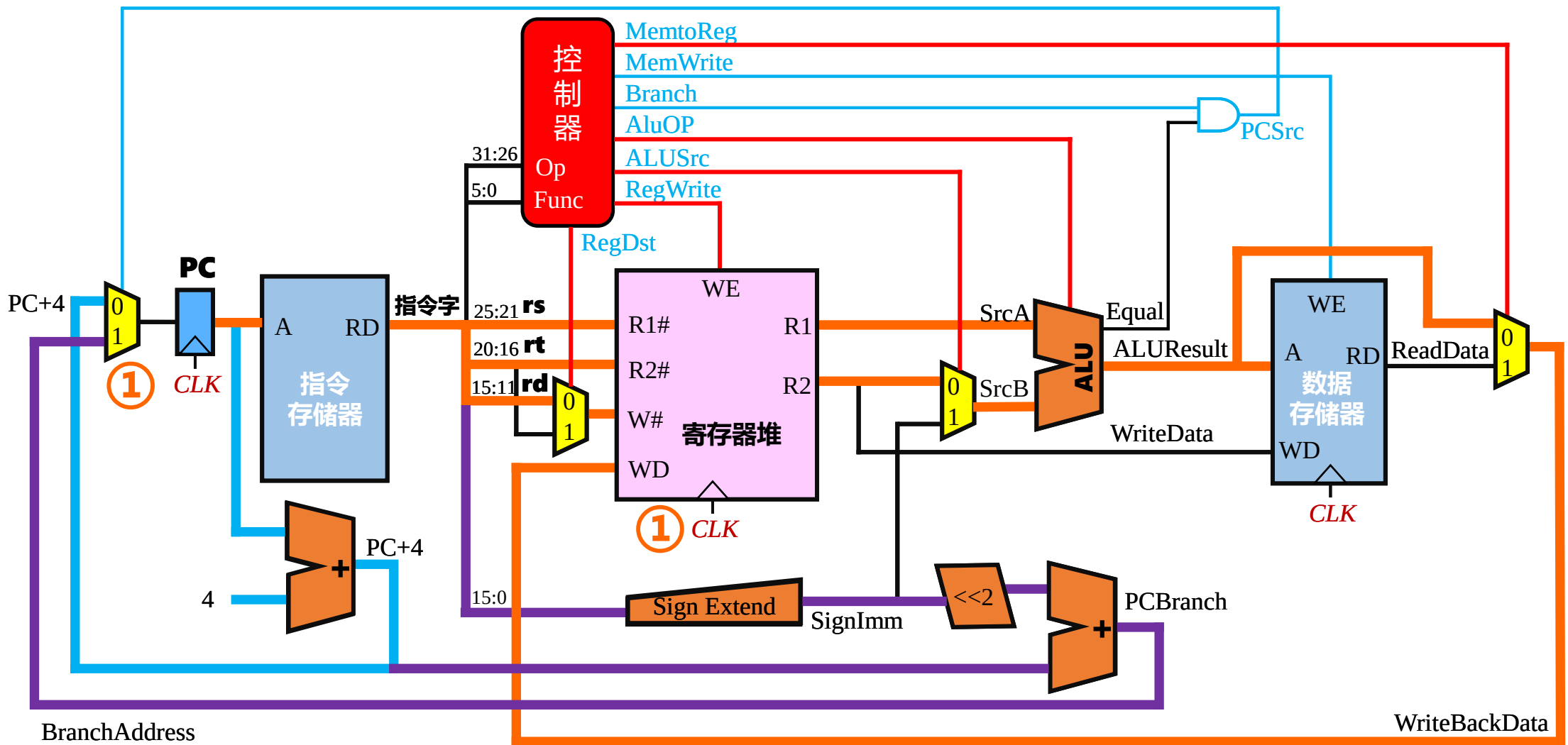
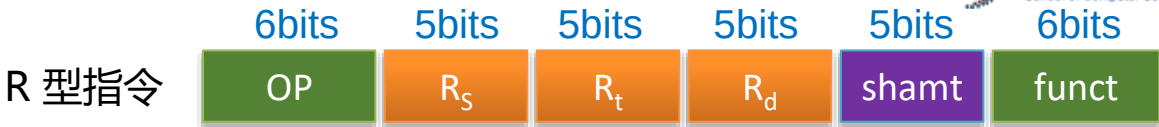


MIPS CPU控制器设计

- 指令执行过程中数据通路的任何资源都不能被重复使用，都应该是**专用的数据通路**，需要被多次使用的资源(如加法器)都需要设置多个
- 取指令和取操作数操作都需要访问存储器，因此将指令和数据分别存放在指令存储器和数据存储器中，以避免资源冲突



专用通路 单周期MIPS CPU

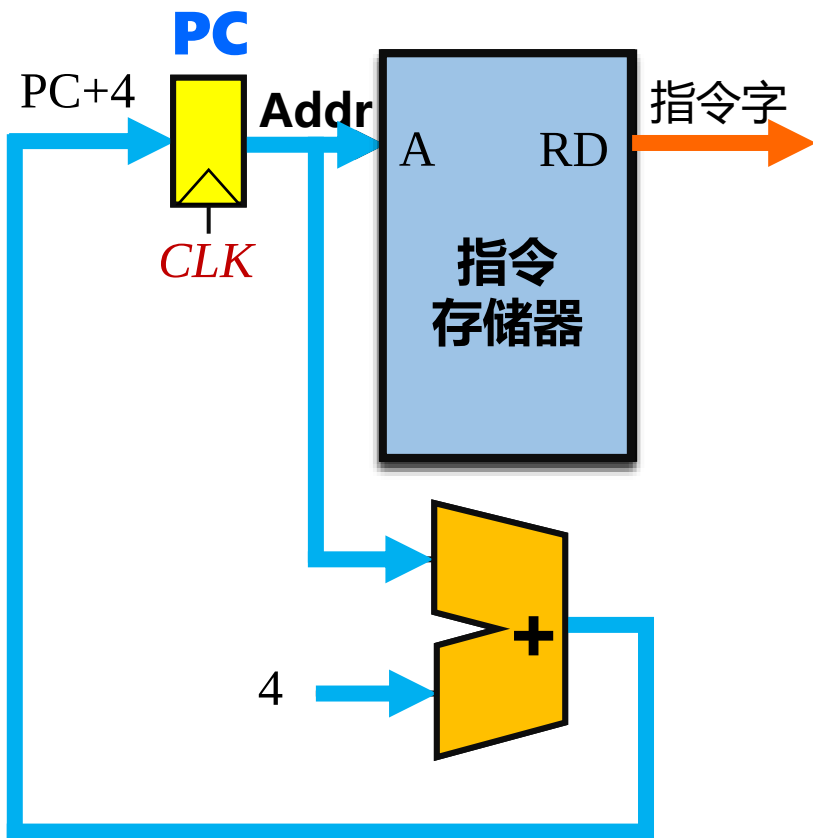


MIPS指令格式



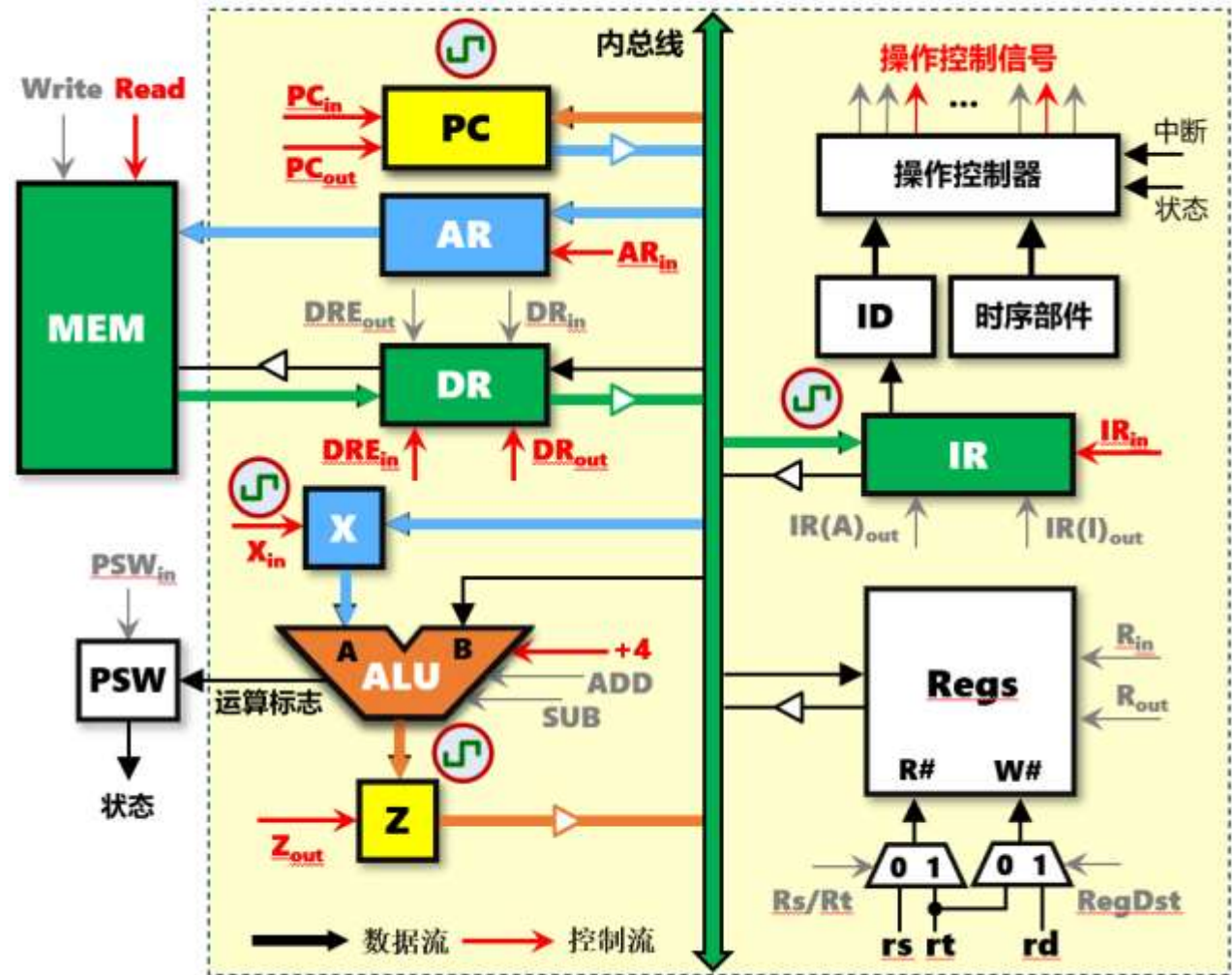
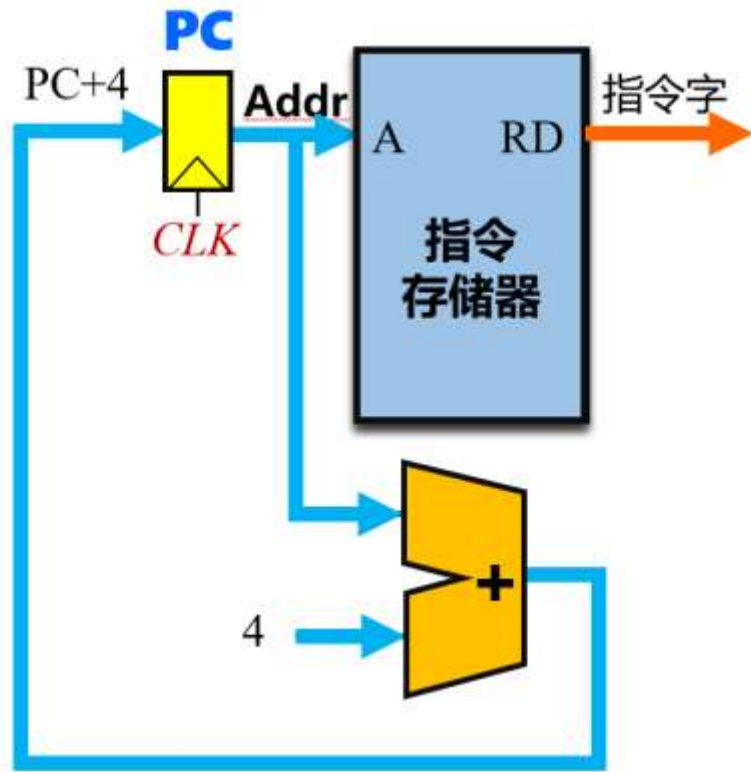
- OP字段固定为6位, funct字段就是扩展操作码、shamt字段是移位变量, 用于移位指令、Imm字段为16位立即数字段.
- rs 、 rt 、 rd 为寄存器操作数字段, 用5位表示, 可以访问32个通用寄存器, R型指令最多可以有3个寄存器操作数, I型指令最多可以有2个寄存器操作数

取指令数据通路

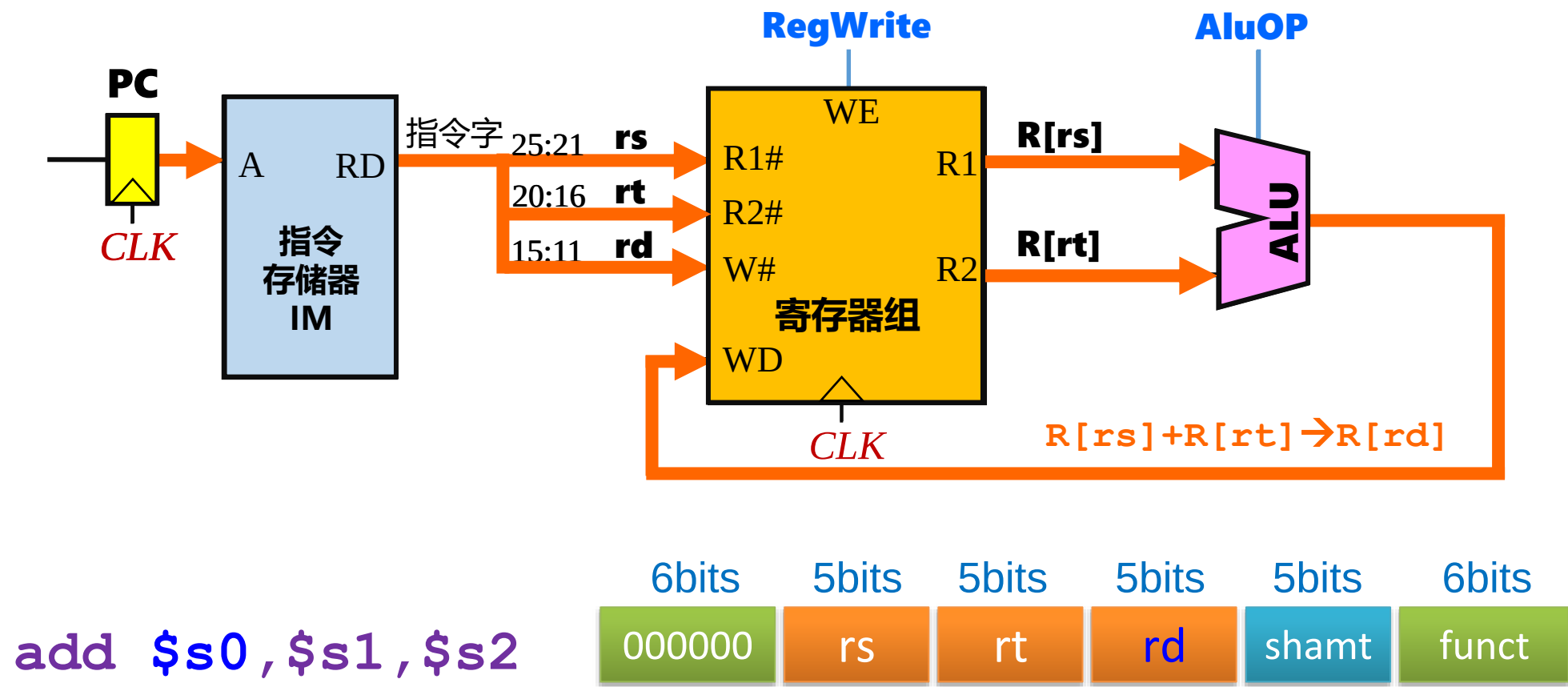


Mem[PC++] → IR

- 程序计数器、**PC**、指令存储器、加法器
- 单周期不能设置**AR, DR, IR**寄存器
- 程序和数据分开存放——哈佛结构
 - ◆ 指令存储器 数据存储器
 - ◆ 指令**cache** 数据**cache**
- 运算器和**PC**累加器分离

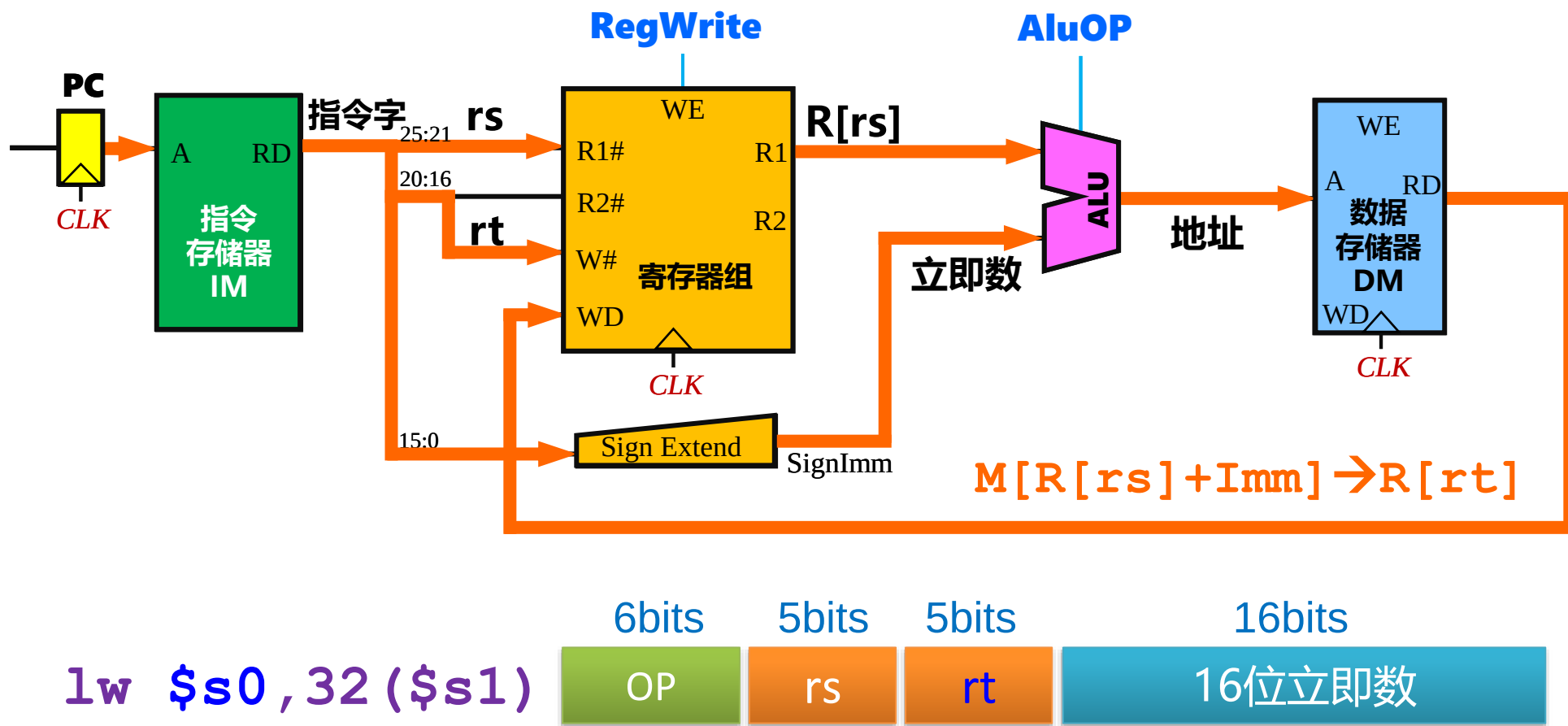


R型指令数据通路



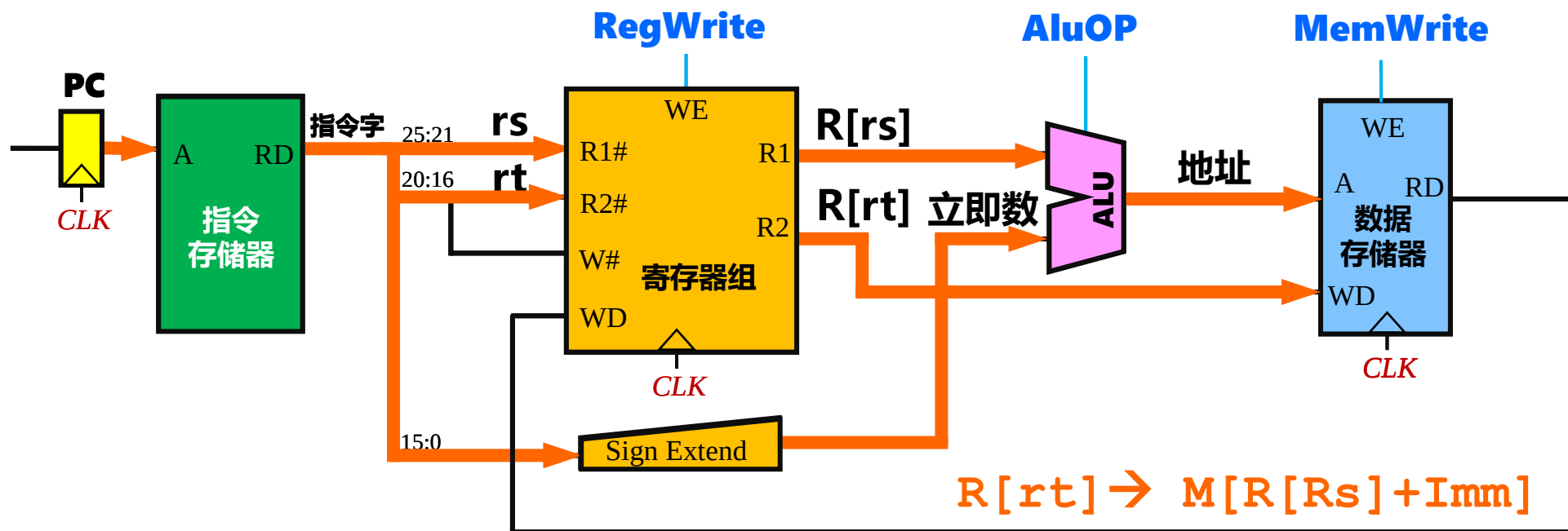
- 寄存器编号端 R1#、R2#、W#，数据端口WD
- 指令字中的funct字段决定AluOp控制 ALU进行相应的运算(这里应选择加法)
- 运算结果被送入寄存器堆的写数据端口WD
- 时钟上跳沿到来时会把运算结果写入目的寄存器rd中

Lw指令数据通路



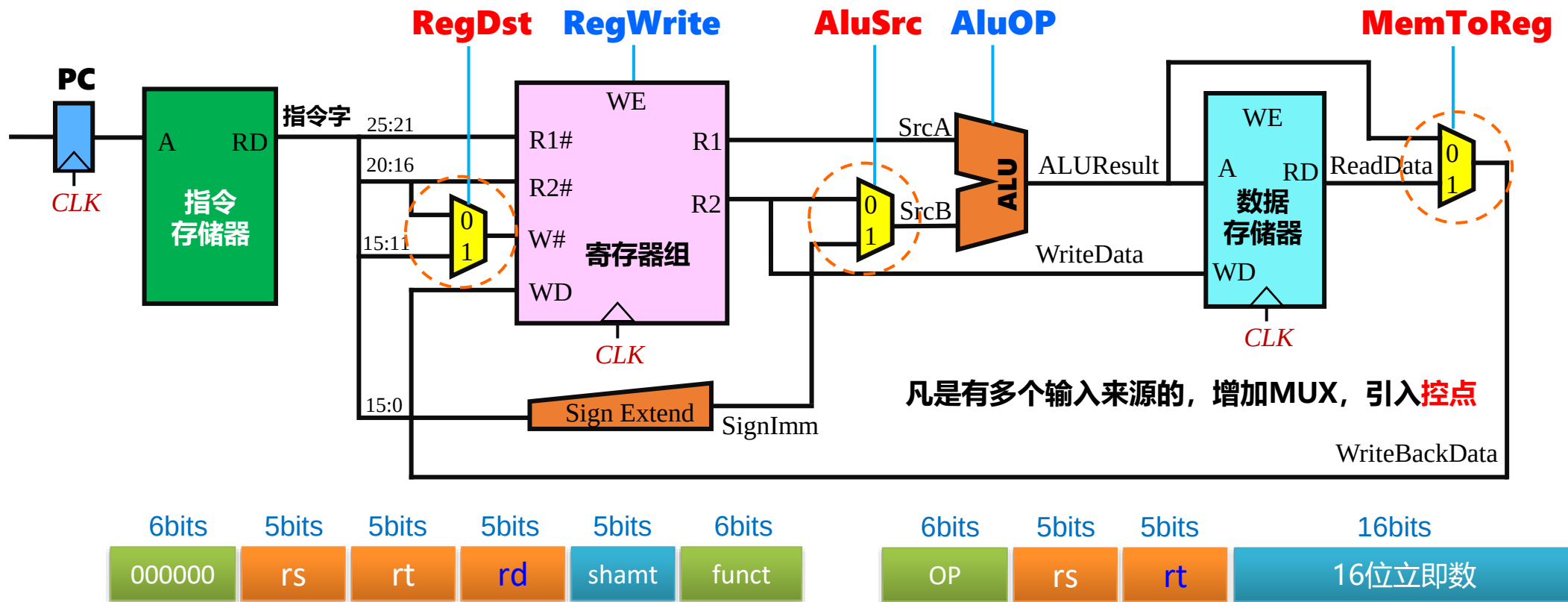
- 其数据通路涉及的功能部件包括指令存储器、寄存器堆、符号扩展单元、ALU、数据存储器
- AluOp设置为加法操作
- WE=0，用于控制存储器进行读操作。

sw指令数据通路



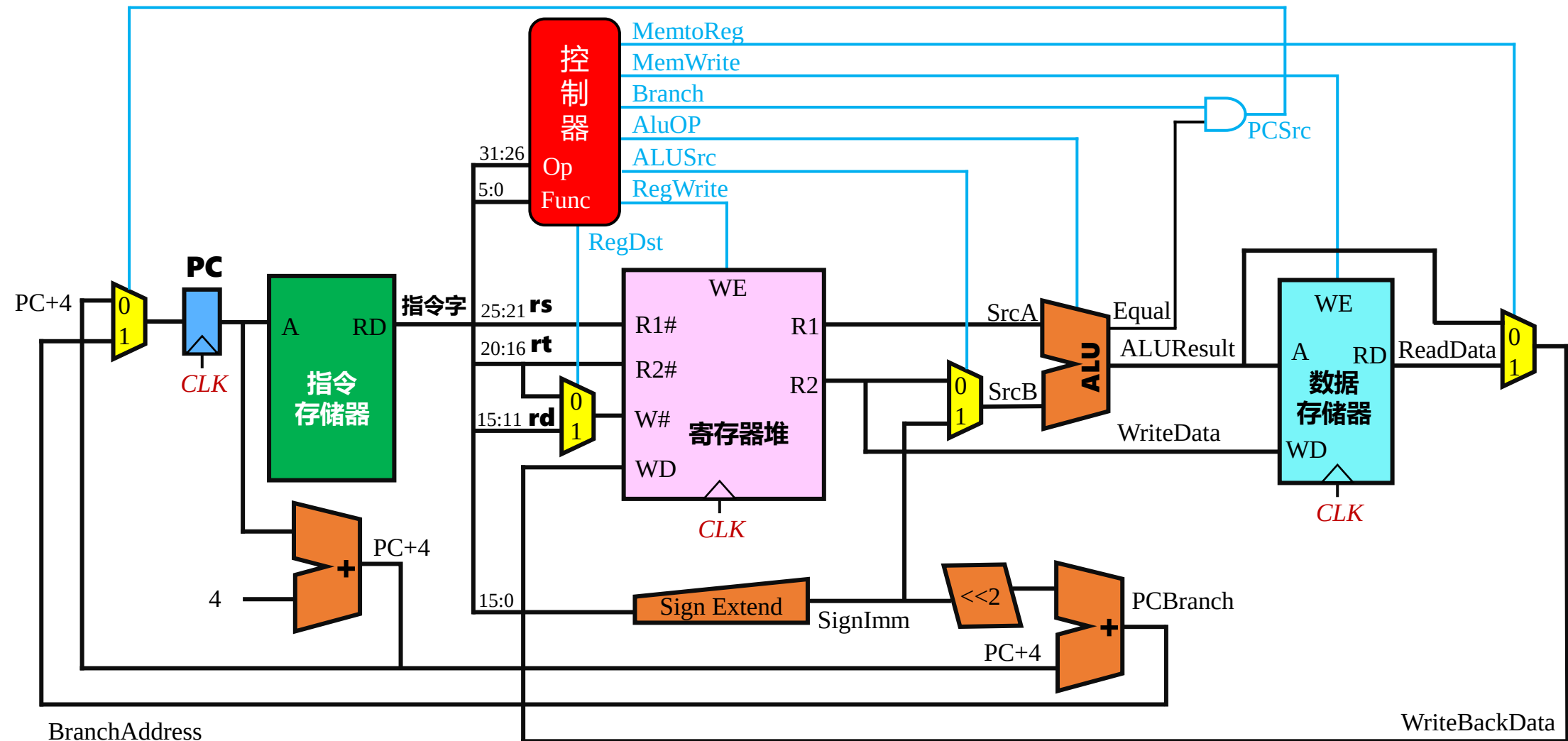
- AluOp应设置为加法操作
- MemWrite=1, 用于控制存储器进行写操作

数据通路综合



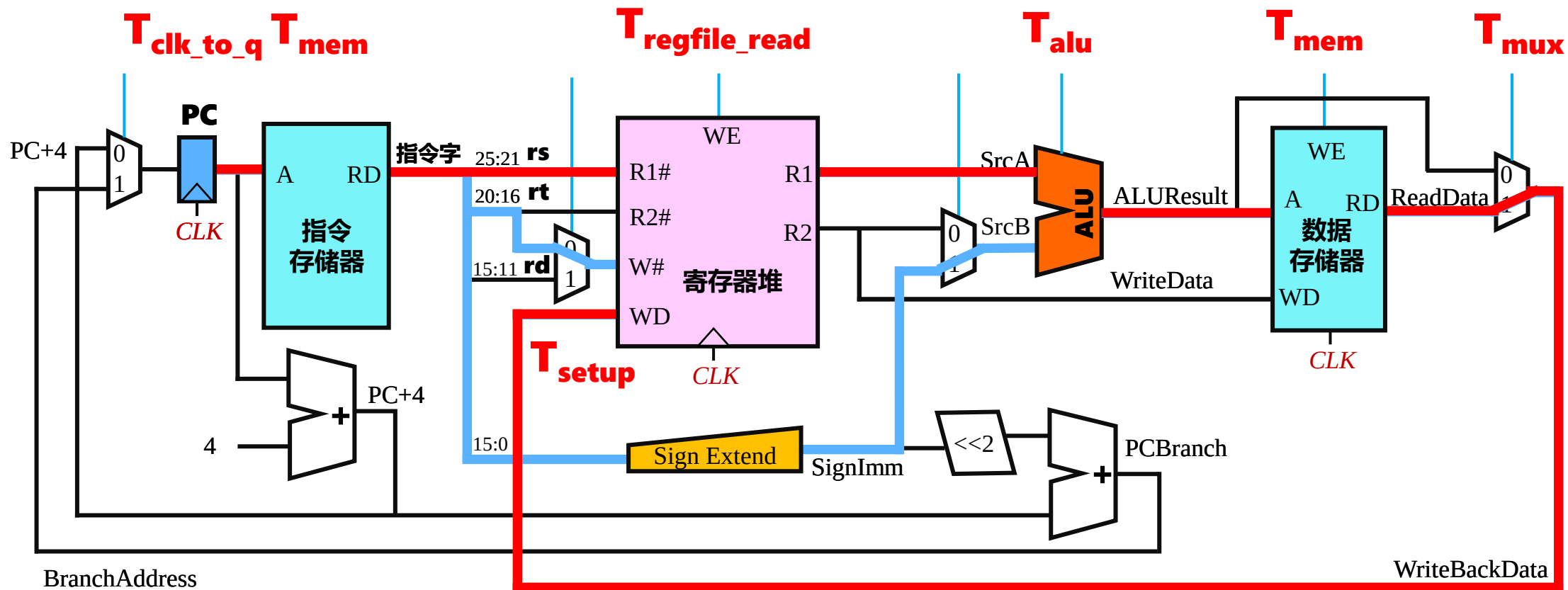
- 增加了RegDst、AluSrc、MemToReg三个控制信号
 - 其中 RegDst 用于决定指令字中的rt、rd 哪个字段作为目的寄存器进行写入
 - AluSrc 用于从寄存器或立即数扩展值中选择一个操作数送入ALU
 - MemToReg用于从ALU的运算结果或主存访问数据中选择一路写回寄存器堆。

单周期MIPS数据通路



将各类指令的数据通路合并，并增加必要的控制线路，得到单周期MIPS32指令系统数据通路

性能分析



- 程序计数器 PC 的触发器延迟 $T_{clk_to_q}$
- 指令存储器访存延迟 T_{mem}
- 寄存器堆的读延迟 $T_{regfile_read}$
- 符号扩展单元延迟 T_{ext}
- SrcB 端的多路选择器延迟 T_{mux}

- 运算器延迟 T_{alu}
- 数据存储器访存延迟 T_{mem}
- 数据存储器输出端多路选择器延迟 T_{mux}
- 写入寄存器的寄存器建立时间 T_{setup}

性能分析

单周期 MIPS 处理器最小时钟周期Tmin为(lw指令延时最长):

$$T_{min} = T_{clk_to_q} + T_{mem} + \max(T_{regfile_read}, T_{ext} + T_{mux}) + T_{alu} + T_{mem} + T_{mux} + T_{setup}$$

单周期数据通路小结

■ 早期的小指令集计算机实现技术

■ 单周期处理器的CPI为多少？

- $CPI = 1$

- CPI越小，则性能越好！

■ 单周期处理器的性能？

- 计算机的性能除CPI外，还取决于时钟周期的宽度

- 单周期处理器的时钟宽度为最复杂指令的执行时间

- 很多指令可以在更短的时间内完成